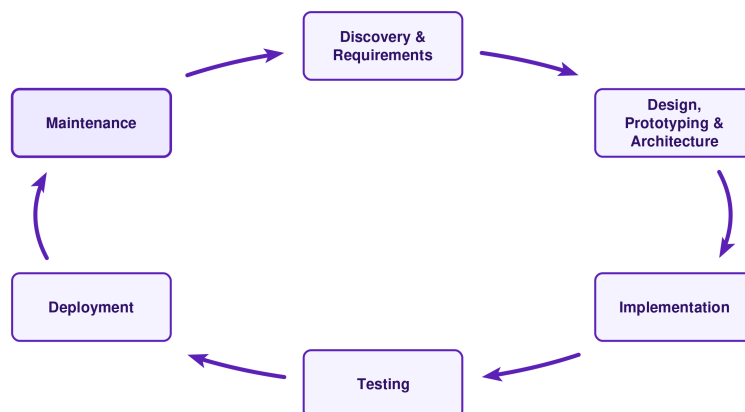




A Process Framework for Human-AI Collaboration in the Software Development Lifecycle

Design and Instantiation of a Supporting Tool

Tomás dos Santos Taborda

Thesis to obtain the Master of Science Degree in

Computer Science and Engineering

Supervisors: Prof. Miguel Leitão Bignolas Mira da Silva
Hugo de Sousa

October 2026

Declaration

I declare that this document is an original work of my own authorship and that it fulfills all the requirements of the Code of Conduct and Good Practices of the Universidade de Lisboa. I acknowledge the use of artificial intelligence tools to support the drafting and refinement of this thesis, with all content produced by these tools carefully reviewed, edited, and validated to ensure accuracy and originality.

Acknowledgments

I would like to thank my parents for their friendship, encouragement and caring over all these years, for always being there for me through thick and thin and without whom this project would not be possible. I would also like to thank my grandparents, aunts, uncles and cousins for their understanding and support throughout all these years.

Quisque facilisis erat a dui. Nam malesuada ornare dolor. Cras gravida, diam sit amet rhoncus ornare, erat elit consectetur erat, id egestas pede nibh eget odio. Proin tincidunt, velit vel porta elementum, magna diam molestie sapien, non aliquet massa pede eu diam. Aliquam iaculis.

Fusce et ipsum et nulla tristique facilisis. Donec eget sem sit amet ligula viverra gravida. Etiam vehicula urna vel turpis. Suspendisse sagittis ante a urna. Morbi a est quis orci consequat rutrum. Nullam egestas feugiat felis. Integer adipiscing semper ligula. Nunc molestie, nisl sit amet cursus convallis, sapien lectus pretium metus, vitae pretium enim wisi id lectus.

Donec vestibulum. Etiam vel nibh. Nulla facilisi. Mauris pharetra. Donec augue. Fusce ultrices, neque id dignissim ultrices, tellus mauris dictum elit, vel lacinia enim metus eu nunc.

I would also like to acknowledge my dissertation supervisors Prof. Some Name and Prof. Some Other Name for their insight, support and sharing of knowledge that has made this Thesis possible.

Last but not least, to all my friends and colleagues that helped me grow as a person and were always there for me during the good and bad times in my life. Thank you.

To each and every one of you - Thank you.

Abstract

Generative Artificial Intelligence and Large Language Models have moved from autocomplete assistants to active contributors across the Software Development Life Cycle, enabling development in which practitioners express requirements in natural language and supervise the resulting artefacts. Adoption, however, has outpaced method. A Systematic Literature Review over 646 articles confirms substantial benefits alongside recurring risks in reliability, security, traceability and accountability, and identifies the absence of structured, repeatable guidance for integrating these tools into professional workflows as the field's central unresolved problem. A majority of functionally correct AI-generated code has been reported to carry security defects, and delivery bottlenecks shift towards review rather than disappearing: value is conditional on process, not on model capability alone.

This dissertation addresses that gap through the Design Science Research Methodology. It proposes a process-oriented framework for governed Human-AI collaboration treating AI systems as first-class collaborators whose contributions must be bounded, validated and traced. The framework defines lifecycle phases with phase-specific quality gates, responsibilities expressed as functions rather than job titles, and governance metrics, anchored by a persistent versioned specification linking every generated artefact to its originating requirement. It is instantiated as Apex, a web application operationalising each phase, gate and artefact against a real project management backend. Evaluation combines application in a real project, standardised usability and workload instruments, interviews with practitioners and Agile experts, and an analytical assessment against criteria set in advance. The contribution is an evidence-grounded process model allowing organisations to adopt AI assistance without surrendering oversight of software quality, security and accountability.

Keywords

Human-AI Collaboration; Software Development Life Cycle; Large Language Models; Process Framework; AI Governance; Design Science Research

Resumo

A Inteligência Artificial Generativa e os Modelos de Linguagem de Grande Dimensão passaram de assistentes de preenchimento automático a contribuidores activos no Ciclo de Vida de Desenvolvimento de Software, permitindo exprimir requisitos em linguagem natural e supervisionar os artefactos gerados. A adopção, contudo, ultrapassou o método. Uma Revisão Sistemática da Literatura sobre 646 artigos confirma benefícios substanciais a par de riscos recorrentes de fiabilidade, segurança, rastreabilidade e responsabilização, e identifica a ausência de orientação estruturada e repetível para integrar estas ferramentas em contextos profissionais como o principal problema por resolver. A maioria do código gerado por IA funcionalmente correcto contém defeitos de segurança, e os estrangulamentos deslocam-se para a revisão em vez de desaparecerem: o valor depende do processo, não do modelo.

Esta dissertação aborda essa lacuna através da Design Science Research Methodology, propondo uma framework orientada a processo para a colaboração Humano-IA governada, cujas contribuições de IA devem ser delimitadas, validadas e rastreadas. Define fases com portões de qualidade específicos, responsabilidades expressas como funções e não como cargos, e métricas de governação, ancoradas numa especificação versionada que liga cada artefacto ao seu requisito. É instanciada no Apex, aplicação web que operacionaliza cada fase, portão e artefacto sobre uma ferramenta real de gestão de projectos. A avaliação combina aplicação num projecto real, instrumentos padronizados de usabilidade e carga cognitiva, entrevistas, e avaliação analítica face a critérios previamente definidos. O contributo é um modelo de processo fundamentado em evidência que permite adoptar assistência por IA sem abdicar do controlo sobre qualidade, segurança e responsabilização.

Palavras Chave

Colaboração Humano-IA; Ciclo de Vida de Desenvolvimento de Software; Modelos de Linguagem de Grande Dimensão; Framework de Processo; Governação de IA; Design Science Research

Contents

1	Introduction	1
2	Research Methodology	3
2.1	Systematic Literature Review (SLR)	3
2.2	Design Science Research Methodology (DSRM)	4
2.3	Mapping the Methodology onto this Document	5
3	Research Background	6
3.1	Artificial Intelligence	6
3.2	Software Development Life Cycle	7
3.3	AI in Software Development	7
3.4	Vibe Coding	7
3.5	Spec-Driven Development	8
4	Systematic Literature Review	9
4.1	Planning the Review	9
4.1.1	Motivation	9
4.1.2	Research Questions	10
4.1.3	Review Protocol	10
4.2	Conducting the Review	11
4.2.1	Filtering Process	11
4.2.2	Data Extraction Analysis	12
4.3	Reporting the Review	13
4.3.1	Benefits of AI and LLMs usage in Software Development	13
4.3.2	Consequences of AI and LLMs usage in Software Development	14
4.3.3	Methodologies and management methods used to integrate AI and LLMs in Software Development	15
4.3.4	Developers' and other stakeholders' opinions regarding the use of AI and LLMs in Software Development	16
4.3.5	Industry sectors adopting AI and LLMs for Software Applications Development	17
4.4	Discussion	18
5	Research Problem	19
6	Research Proposal	21
6.1	Objectives	22
6.2	Purpose and Scope	22
6.3	Design Principles	23
6.4	Lifecycle Phases and the AI Interaction Mapping	24
6.5	Roles and Responsibilities	26
6.6	Governance Mechanisms and Quality Gates	28
6.6.1	Bolts as the Unit of Execution	28

6.6.2	Spec-Anchored Continuity	28
6.6.3	Quality Gates	29
6.6.4	Governance Metrics	30
6.7	Work Organisation	30
6.8	Operational Playbooks	31
6.9	Positioning Against Alternatives	32
6.10	Summary	33
7	Implementation	35
7.1	Purpose and Positioning	35
7.2	Architecture	36
7.3	Operationalising the Framework	38
7.4	The Governance Analytics Surface	44
7.5	Design History and Practitioner Feedback	46
7.6	Quality Assurance of the Implementation	47
7.7	Limitations of the Implementation	48
8	Demonstration	49
8.1	Demonstration Design	49
8.2	Context	52
8.3	Application of the Framework	53
8.4	Observations	57
8.5	Summary	61
9	Evaluation	63
9.1	Evaluation Strategy	63
9.2	Participants	64
9.3	Instruments and Procedure	66
9.3.1	System Usability Scale	66
9.3.2	NASA Task Load Index	67
9.3.3	Apex UX Questionnaire	68
9.3.4	Semi-Structured Interviews	69
9.3.5	Analytical Assessment	70
9.4	Results	71
9.4.1	System Usability Scale	71
9.4.2	Raw NASA Task Load Index	72
9.4.3	Apex UX Questionnaire	73
9.4.4	Analytical Assessment	74
9.4.5	Demonstration in a Real Project	75
9.5	Threats to Validity	76
9.6	Discussion	77
9.6.1	Answering the Research Questions	78
10	Conclusion	81
10.1	Summary and Contributions	81
10.2	Communication	83
10.3	Limitations	84
10.4	Future Work	85
	Declaration of Generative AI and AI-Assisted Technologies in the Writing Process	87
	Bibliography	87

A	Extended Results for the Systematic Literature Review	93
A.1	Stakeholder opinions reported in the literature	93
A.2	Industry sectors adopting AI and LLMs	95
A.3	Final set of studies retained by the review	96
B	Evaluation Instruments	98
B.1	Informed consent and demographics	98
B.2	Raw NASA Task Load Index	99
B.3	System Usability Scale	100
B.4	Apex UX Questionnaire	100
B.5	Interview guides	102
B.6	Per-participant raw scores	103

List of Figures

2.1	Three phases of the Systematic Literature Review	4
2.2	Phases of the Design Science Research Methodology	4
4.1	Articles Filtering Process	11
4.2	Academic Journal Publications	12
6.1	The six lifecycle phases and the three loop-backs out of Maintenance	25
7.1	Apex's system architecture, part one of two	37
7.2	Apex's system architecture, part two of two	39
7.3	A representative walk through Apex, Phases 1 to 3	42
7.4	A representative walk through Apex, Phases 4 to 6	43
8.1	A real scenario failure caught at the Testing Gate	55
8.2	The Deployment Gate for Outfolio's first production release	56
8.3	A Maintenance item classified and routed to Discovery	57

List of Tables

2.1	Mapping between DSRM activities and the chapters of this dissertation	5
4.1	Research Questions and Focus Areas	10
4.2	Research Elements and Details	10
4.3	Inclusion and Exclusion Criteria	12
4.4	Benefits of using AI and LLMs in Software Development	13
4.5	Consequences of using AI and LLMs in Software Development	14
4.6	Methodologies and management methods for integrating AI and LLMs	15
4.7	Impact of AI and LLMs across Software Development Lifecycle Phases	18
6.1	Research questions, key findings, problem aspects and framework objectives	22
6.2	The seven design principles and the evidence grounding each	23
6.3	The six lifecycle phases and the preceding Strategic Alignment stage	25
6.4	The hats: the function each discharges and the gate each owns	27
6.5	The five quality gates: owner, required evidence, and the consequence of failure	29
6.6	The six operational playbooks: trigger, procedure, and termination	32
7.1	Framework constructs against the mechanisms implementing them in Apex	40
9.1	Evaluation instruments, target artefact, and research question addressed	64
9.2	Raw NASA-TLX subscale means by task	72
9.3	Apex UX questionnaire, items discussed individually	73
9.4	Analytical assessment of the framework against criteria fixed in advance	74
A.1	Public / Researchers Opinions on AI and LLMs in Software Development	93
A.2	Developers / Practitioners Opinions on AI and LLMs in Software Development	94
A.3	Organizations / Management Opinions on AI and LLMs in Software Development	94
A.4	Industry sectors adopting AI and LLMs	95
A.5	Studies retained by the systematic literature review	96
B.1	Demographics items, collected on the consent form	99
B.2	Raw NASA Task Load Index subscales, in the order presented	100
B.3	System Usability Scale items, as administered	100
B.4	Apex UX questionnaire items, by construct	101

B.5	Apex UX questionnaire responses, item by item	102
B.6	SUS score and sub-scales, per participant	104

Acronyms

AI	Artificial Intelligence
AI-DLC	AI-Driven Development Lifecycle
API	Application Programming Interface
BDD	Behaviour-Driven Development
CI	Continuous Integration
DRQ	Dissertation Research Question
DSRM	Design Science Research Methodology
GenAI	Generative Artificial Intelligence
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IaC	Infrastructure as Code
IST	Instituto Superior Tecnico
KPI	Key Performance Indicator
LLM	Large Language Model
MLOps	Machine Learning Operations
NASA-TLX	NASA Task Load Index
NIST	National Institute of Standards and Technology
OKR	Objective and Key Result
PM	Project Manager
PO	Product Owner
QA	Quality Assurance
SDLC	Software Development Life Cycle
SLR	Systematic Literature Review
SUS	System Usability Scale
UI	User Interface
UX	User Experience

1

Introduction

Generative Artificial Intelligence (GenAI) and Large Language Models (LLMs) have extended machine assistance towards generating coherent, context-aware artefacts, introducing intent-driven programming, colloquially *Vibe Coding*, in which developers describe desired behaviour in natural language and refine what a model produces [1, 2]. The shift is not confined to code: LLMs now participate in requirements elicitation, architectural exploration, test generation, documentation and debugging, touching every Software Development Life Cycle (SDLC) phase [3–5]. The practitioner’s work moves from constructing software towards specifying intent, supervising generated artefacts and remaining accountable for them.

The evidence on outcomes, however, is more nuanced than early enthusiasm suggested, and that gap motivates this work. Adoption is close to universal while trust is not: 84% of developers report using or planning to use AI tools, up from 76%, while the proportion trusting their accuracy fell from 40% to 29% [6], with daily use around 90% [7]. That distrust is warranted, and the decisive point is that functional correctness and security do not move together: 61% of a leading coding agent’s solutions were functionally correct but only 10.5% secure [8], and across more than one hundred models 45% of samples introduced an OWASP Top 10 vulnerability, with newer models improving at correctness while security stayed flat [9].

Productivity gains are equally conditional. Early evidence reported a task completed 55.8% faster with an assistant, though unpublished in a peer-reviewed venue [10], whereas a randomised controlled trial found 16 experienced developers on their own mature repositories took 19% longer while estimating afterwards that they had been 20% faster [11]; the favourable result came from a self-contained task with no codebase to respect. At organisational scale, telemetry across 1,255 teams associates high adoption with 98% more pull requests merged but 91% longer review times and no significant correlation with delivery outcomes [12]. The bottleneck moves downstream to validation rather than disappearing. Several

sources here are industry-reported rather than peer-reviewed and are identified as such throughout, so the argument rests on consistency of direction rather than any single figure: the value of AI assistance is conditional on the process surrounding it rather than a property of the model alone.

To characterise practice rigorously, an Systematic Literature Review (SLR) following Kitchenham's guidelines retrieved 646 articles, reported in Chapter 4. Its central finding is methodological: organisations lack established methodologies for the responsible use of Artificial Intelligence (AI) in professional software development [13–16], and proposed approaches remain fragmented across isolated tools, sharing no common foundation on lifecycle integration, oversight or evaluation [3]. Established methodologies do not close the gap: Waterfall, Scrum, Kanban and the Agile Manifesto were designed for environments in which every contributor is human [4], so they lack structures for governing AI participation, bounding the context a model may act upon, and allocating responsibility over generated artefacts, while their cadence assumes implementation is the binding constraint, which the evidence contradicts. **The research problem is therefore the absence of structured, repeatable guidance and standardised evaluation practices for integrating AI across the SDLC while preserving human oversight, software quality and security, and accountability.**

The objective is to design, instantiate, demonstrate and evaluate a process-oriented framework addressing that problem. Four questions structure it, labelled *Dissertation Research Question (DRQ)* to keep them distinct from the review questions of Chapter 4, which ask what the literature reports rather than what these artefacts achieve. **DRQ1:** how can AI-supported activities be mapped to SDLC phases so that the point, purpose and boundary of AI participation are unambiguous? **DRQ2:** which governance mechanisms, quality gates and human-in-the-loop practices keep AI contributions traceable, validated and accountable? **DRQ3:** to what extent can those constructs be operationalised in a working system, and which resist implementation? **DRQ4:** does the framework, through that implementation, give practitioners usable and cognitively sustainable support in a real setting? Chapters 6 to 9 answer them in turn, the third including the constructs weakened, proxied or left unimplemented.

The contributions are four: the synthesis above; a process framework treating AI as a first-class collaborator, with phases, gates, function-based responsibilities, metrics and a versioned specification anchoring artefacts to requirements; Apex, a reference implementation showing those constructs implementable rather than describable; and an empirical evaluation of its usability and workload.

2

Research Methodology

Contents

2.1	Systematic Literature Review (SLR)	3
2.2	Design Science Research Methodology (DSRM)	4
2.3	Mapping the Methodology onto this Document	5

This chapter presents the two research methodologies used in this dissertation, the SLR and the Design Science Research Methodology (DSRM), and maps the activities of the second onto the chapters of this document.

2.1 Systematic Literature Review (SLR)

The review follows Kitchenham's guidelines, which describe a systematic review as "a means of evaluating and interpreting all available research relevant to a particular research question, topic area, or phenomenon of interest" conducted through "a trustworthy, rigorous, and auditable methodology." The guidelines prescribe three phases: **planning**, which defines the motivation, research questions, selection criteria and protocol; **conducting**, which applies the protocol to identify and select studies; and **reporting**, which synthesises the findings.

Figure 2.1 illustrates those phases and their activities; the filtering that separates them is conducted in Chapter 4, where the successive passes are given together with their attrition numbers.

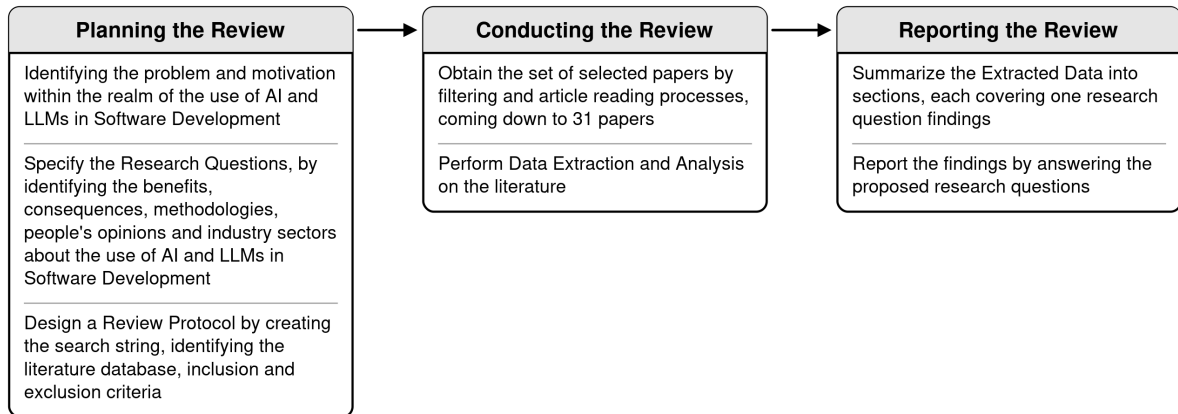


Figure 2.1: Three phases of the Systematic Literature Review

2.2 Design Science Research Methodology (DSRM)

The **DSRM** concerns the development of IT artefacts, such as models, methods or frameworks, addressing identified organisational or technological problems. Unlike purely observational research it emphasises constructing solutions and evaluating their usefulness. Proposed by Peffers et al., it follows a six-step process supporting systematic design, clear contributions and transferable knowledge, which makes it suitable for research proposing process-oriented solutions. The steps are illustrated in Figure 2.2 and described below.

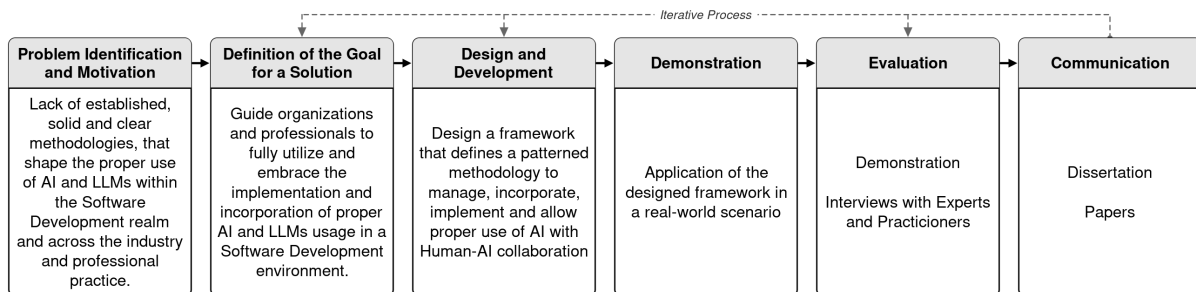


Figure 2.2: Phases of the Design Science Research Methodology

Problem identification and motivation addresses the lack of structured guidance, standardised methodologies and evaluation practices for integrating AI into software development while preserving quality, accountability and human oversight; the problem was identified through the **SLR**, which found fragmented practices and unclear human-AI role definitions across the **SDLC**. **Definition of objectives** derives from that problem objectives aimed at guiding organisations towards effective, secure and responsible adoption through appropriate processes and governance mechanisms. **Design and development** takes the review findings as input and produces the framework, defining phases, practices, hats and decision points, together with Apex as its reference implementation. **Demonstration** applies the artefact to a real project, exercising the phases end to end and producing documented artefacts, decision rationales and observations recorded against criteria fixed in advance. **Evaluation** separates

the two artefacts it assesses: the framework through interviews with practitioners and Agile experts and an analytical assessment against pre-stated criteria, the instantiation through a task-based study measuring perceived usability and cognitive workload with standardised instruments; that separation is argued in Chapter 9, since a usability result for the tool is not evidence that the method is sound. **Communication** disseminates results through a scientific article reporting the SLR, the extended abstract accompanying this dissertation, and the defence.

2.3 Mapping the Methodology onto this Document

Because DSRM prescribes activities rather than document sections, Table 2.1 states where each is carried out. The mapping is not strictly linear: as Chapter 7 describes, feedback gathered while building the reference implementation fed back into the framework’s design, which is the iteration path DSRM explicitly allows.

Table 2.1: Mapping between DSRM activities and the chapters of this dissertation

DSRM Activity	Chapter	Output
Problem Identification and Motivation	Chapters 4 and 5	Evidence base from the SLR; formalised research problem
Definition of Objectives	Chapter 6	Objectives derived from the review findings
Design and Development	Chapters 6 and 7	The framework, and Apex as its reference implementation
Demonstration	Chapter 8	Application of the artefact in a real-world project
Evaluation	Chapter 9	Usability and workload measurement, interviews, analytical assessment
Communication	Chapter 10	Publications and dissertation defence

3

Research Background

Contents

3.1 Artificial Intelligence	6
3.2 Software Development Life Cycle	7
3.3 AI in Software Development	7
3.4 Vibe Coding	7
3.5 Spec-Driven Development	8

This chapter presents the theoretical background of the topics associated with the research. The topics covered are Artificial Intelligence, the Software Development Life Cycle, AI in Software Development, Vibe Coding, and Spec-Driven Development.

3.1 Artificial Intelligence

AI and Generative AI are used throughout in their common sense: systems performing tasks that otherwise require human cognition, and the subset of those that generate new content, text, images, audio or code, rather than classify or retrieve existing information.

Large Language Models (LLMs) are a specific class of GenAI systems trained on vast corpora of text and source code. Built on transformer architectures, their primary function is next-token prediction, but that capability enables more complex tasks such as summarization, explanation, reasoning, conversational interaction, and code generation. Within software engineering, LLMs are relevant because they can interpret programming intent expressed in natural language and generate syntactically valid

code in a variety of programming languages, as well as explain, refactor or restructure existing code and produce tests and documentation with far less manual effort. These capabilities position LLMs as collaborative tools supporting both novice and experienced developers across the stages of software development. [1–5, 13–39]

3.2 Software Development Life Cycle

The Software Development Life Cycle (SDLC) structures software construction into requirements, design, implementation, testing, deployment and maintenance phases [3].

The SDLC is central to this research because LLMs engage differently with each of its stages, as Chapter 1 details.

Software development processes have evolved to emphasize iterative delivery and continuous integration, supported by methodologies such as Agile and DevOps. Despite these improvements, much of the work remains manual, repetitive and time-intensive: debugging, documenting code, maintaining consistency across repositories, and handling routine implementation. As systems scale in complexity, the pressure to increase productivity without compromising maintainability or security grows with them, and that pressure is a key factor driving interest in AI-assisted development workflows. [3, 4, 26, 35]

3.3 AI in Software Development

Studies indicate that LLMs can accelerate prototyping, clarify unfamiliar codebases, and help maintain documentation quality, which is especially valuable in early-stage development or when exploring new frameworks and libraries.

The integration is increasingly characterized as collaborative rather than as full automation: LLMs support pattern recognition, summarization and structural reasoning, while the developer remains responsible for architectural decisions, validation, integration, and alignment with project requirements. Research in this area focuses on how workflows adapt to that shift, how developers form trust in AI outputs, and how responsibilities are distributed between human judgment and automated generation. These themes are further examined in Chapter 4. [3, 4, 13, 14, 22, 26, 28, 35]

3.4 Vibe Coding

Vibe Coding refers to a style of software development in which developers describe their intentions, requirements, or constraints in natural language, and a generative model produces corresponding code or related artifacts. The developer does not begin by writing syntactically precise code: development proceeds conversationally, the model proposing implementations that the developer then evaluates for correctness, aligns with project requirements, and refines by modifying either the prompt or the produced code.

Vibe Coding therefore shifts part of the programming effort from manual construction to specifying intent and assessing alternatives. It does not replace traditional programming but reframes how initial

structure and exploratory code are produced, with implications for developer workflows, cognitive load, and the distribution of effort across the SDLC that are examined in Chapter 4. [1, 2]

3.5 Spec-Driven Development

Spec-driven development is an emerging engineering practice in which a persistent, versioned specification, rather than the source code itself, is treated as the primary artefact that both developers and AI agents work against. Instead of prompting a model directly against a codebase and accepting whatever it produces, the practice interposes a written specification, of requirements, design decisions, and constraints, that the model must consult and satisfy before code is generated, and that is kept in sync with the codebase as it evolves. Industry examples of this pattern include AWS Kiro's three-stage requirements, design, and tasks workflow, GitHub's Spec-Kit, which adds a "constitution" of immutable project principles that generated code may not violate, and the Tessel Framework, in which, as its own materials put it, the specification itself becomes the maintained artefact rather than the code [40].

The practice is tracked by Thoughtworks' Technology Radar as an emerging AI-assisted engineering technique, named specifically "spec-driven development" [40], and has begun to receive academic attention: Farrag frames it as specification-driven governance, arguing that it addresses what is termed the productivity-reliability paradox in AI-augmented development, though this framing is currently reported only in a preprint that has not yet completed peer review [41]. The practice responds directly to the risk profile characterised in Chapter 1: narrowing and anchoring the context a model is permitted to act upon is a governance mechanism, not merely a convenience, since it gives generated output a stable reference to be validated against rather than leaving correctness to be inferred from the prompt alone. This precedent is the direct grounding for Spec-Anchored Continuity, the persistent specification mechanism at the centre of the framework proposed in Chapter 6.

4

Systematic Literature Review

Contents

4.1 Planning the Review	9
4.2 Conducting the Review	11
4.3 Reporting the Review	13
4.4 Discussion	18

This chapter describes the execution of the systematic literature review, following the three SLR stages: Planning, Conducting, and Reporting the results.

4.1 Planning the Review

4.1.1 Motivation

AI, particularly LLMs and GenAI, is increasingly adopted across the SDLC to support code generation and completion, documentation, requirements engineering, automated bug fixing, and developer learning, and its use has been associated with productivity gains, improved efficiency, and support for complex development tasks [3, 4, 13, 14, 22, 26, 28, 35].

Adoption has nevertheless outpaced the resolution of the challenges it raises. The literature reports persistent concerns over the quality, reliability and trustworthiness of AI-generated artifacts, methodological barriers such as limited explainability and the black-box nature of advanced models, a heavy dependence on prompt formulation, and a concentration on isolated tasks that leaves several SDLC phases, particularly requirements engineering and design, with limited practical guidance.

4.1.2 Research Questions

Five research questions structure the review:

Table 4.1: Research Questions and Focus Areas

RQ	Research Question	Focus
RQ1	What are the benefits of using and incorporating Vibe Coding, AI, and Large Language Models in software development?	Identify and categorize improvements in productivity, creativity, code quality, automation, collaboration, and innovation enabled by AI-driven tools.
RQ2	What are the consequences that may arise from using and incorporating Vibe Coding, AI, and LLMs in software development, both in code generation and other sections?	Analyze risks such as technical debt, bias, reduced developer learning, code ownership, ethical issues, and dependency on AI tools.
RQ3	What methodologies and management methods are being used to integrate Vibe Coding, AI, and LLMs into software engineering processes and workflows?	Explore frameworks, best practices, and process models (e.g., AI-augmented Agile, human-in-the-loop development, prompt engineering practices).
RQ4	What are developers' and other stakeholders' opinions regarding the use of AI and LLMs in software development?	Assess perceptions, trust, satisfaction, and acceptance of AI-assisted coding among developers and organizations.
RQ5	Which industry sectors are adopting AI for application development, and how do they benefit from its integration?	Map adoption trends across industries (e.g., finance, healthcare, IT, education), highlighting sector-specific motivations and outcomes.

4.1.3 Review Protocol

The criteria used for the literature search, stated in Table 4.2, returned 646 articles.

Table 4.2: Research Elements and Details

Element	Research Details
Source	EBSCO
Final Search String	("Vibe Cod*" OR "AI models" OR "AI agen*" OR "large language model*" OR "generative AI" OR openai OR "chatgpt" OR LLM* OR "claude" OR "gemini" OR copilot OR co-pilot) AND ("software development" OR "application development" OR "code generation")
Search Method	Search string found in the abstract of articles in English academic journals or conference materials, with no date range limit
Results	646

"Vibe Coding" is the most recent term for the practice, but the practice predates the term, and querying it alone returned few relevant articles because the same work is published under different expres-

sions. Synonymous keywords were added accordingly: it is the combination of "code generation" and "application development" with "generative AI" and "AI models" that retrieves the intended body of work, not any single term. The string was executed against the EBSCO Online Digital Library through its Advanced Search option, over the AB abstract field and the title, restricted to full-text-available results.

4.2 Conducting the Review

The second stage applies the filtering criteria to the 646 articles retrieved in the first; the attrition at each step is shown in Figure 4.1.

4.2.1 Filtering Process

Extracting all 646 articles was not possible: EBSCO's page allows the download of 50 articles at a time, and after reloading and extending the page to its maximum the site displayed, in relevance order, only 467 of the 646.

The AI-powered systematic review management platform "Rayyan" was used to organise the articles. Of the 467 extracted, 117 duplicates were identified and removed, leaving 350 for abstract reading; that pass rejected 286 as belonging to a theme other than developing applications using AI, leaving 64 articles.

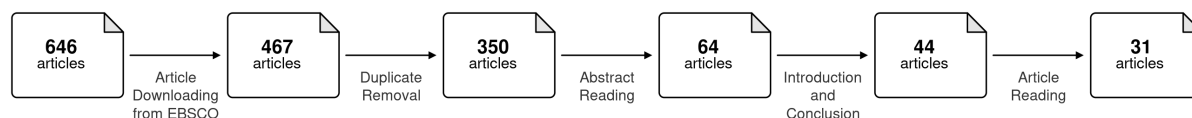


Figure 4.1: Articles Filtering Process

The content of those 64 articles was then read and each placed in one of four categories: Included, Maybe, Excluded and Skipped. Articles were "Skipped" when the introduction and conclusion showed them to be off-theme, outdated, or silent on all five research questions; 20 were skipped and excluded on those grounds. "Maybe" was used while it was unclear whether an article bore on the theme, and most were ultimately excluded in favour of more recent studies.

An article that answered any of the five research questions clearly and explicitly was included. Where the information was unclear, insufficient, or too thin to confirm that an answer was given, the predefined criteria in Table 4.3 decided eligibility. Of the 44 articles, 31 met the criteria and were retained for analysis.

Table 4.3: Inclusion and Exclusion Criteria

Inclusion Criteria	Exclusion Criteria
• Relevance regarding AI and LLM usage in the development of software applications • Relevance regarding benefits and consequences of incorporating and using AI and LLMs in the development of software applications • Relevance regarding methodologies and management methods used to integrate AI and LLMs into software engineering processes and workflows • Relevance regarding people's opinions with AI and LLMs use in software development • Relevance regarding which industry sectors are adopting AI for application development and its benefits	• Not written in English • Does not clarify any of the Inclusion Criteria • Outdated due to the highly dynamic nature of the field (anything older than 2024)

4.2.2 Data Extraction Analysis

None is categorised as conference material: every one is a peer-reviewed academic journal article as classified by EBSCO, distributed over time as shown in Figure 4.2.

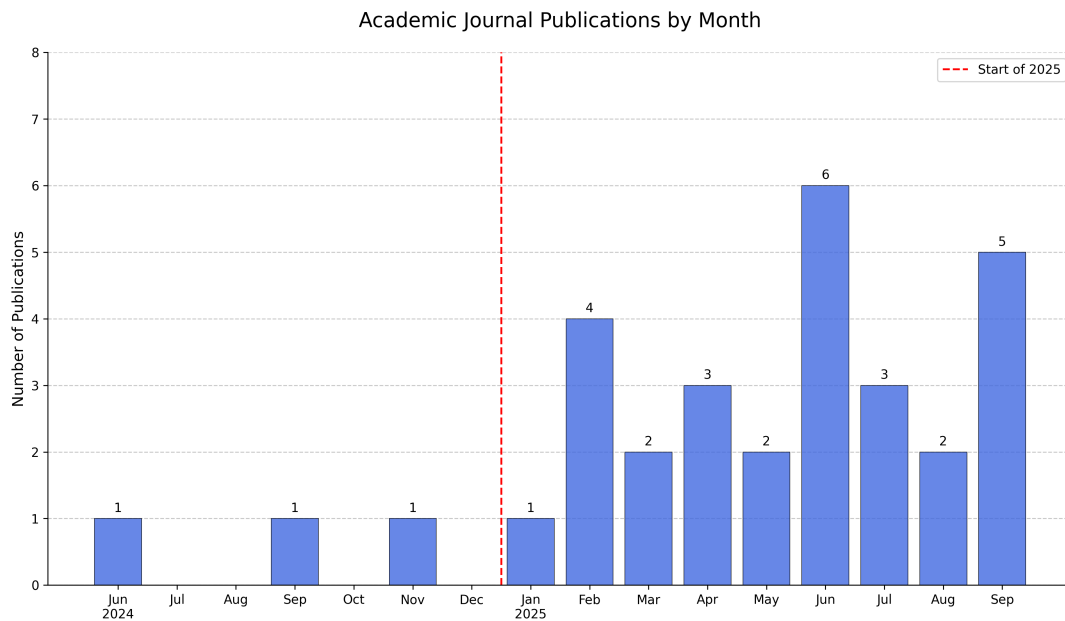


Figure 4.2: Academic Journal Publications

4.3 Reporting the Review

The third and last stage reports the analysis of the retained articles, research question by research question. The venue of every retained study is listed in Appendix A.3; the corpus is concentrated rather than diffuse, with ACM Transactions on Software Engineering and Methodology contributing ten and Information and Software Technology three, the remaining venues one each, consistent with a topic actively published across the field rather than owned by any single outlet.

4.3.1 Benefits of AI and LLMs usage in Software Development

The first research question concerns the benefits of AI and LLM adoption for developers, companies and other stakeholders. Table 4.4 collects those reported in the literature.

Table 4.4: Benefits of using AI and LLMs in Software Development

Benefits	Description	#	References
Improved Code Quality, Accuracy, and Traceability	"improving code quality and optimizing work with faster times" [14].	20	[3, 4, 13–16, 18, 19, 21–24, 26, 27, 29–31, 33, 34, 36]
Increased Productivity and Efficiency	"converts natural language or abstract intent into functioning software modules, dramatically shortening development cycles" [1].	18	[1–4, 13, 14, 19, 20, 24, 27, 29, 30, 33–37, 39]
Automation of Testing, Debugging, and Repair	AI and LLMs support "automating coding, bug detection, debugging, and personalizing software according to user needs" [14].	12	[1, 3, 16, 17, 20, 21, 29, 30, 33–35, 37]
Optimization of Code Performance and Efficiency	"The proportionally small number of studies in SE subfields like Build, Release, Deploy, Operate and Monitor indicates a broad untapped area for cost savings and novelty" [28].	10	[1, 2, 4, 13, 14, 17, 24, 28, 33, 35]
Support for Early Software Development Phases (Requirements & Design)	AI and LLMs have effectiveness in "...enhancing stakeholder communication and requirement documentation" while noting challenges with bias and accuracy in requirements and design elicitation [29].	8	[4, 15, 16, 22, 27, 29, 33, 35]
Automated Documentation and Summarization	"LLMs can [...] automatically generate documentation for codebases [...] ensuring that documentation stays current and can be updated as code evolves" [24].	8	[4, 13, 17, 22, 24, 26, 35, 36]
Enhanced Security and Vulnerability Management	AI supports SDLC security through "vulnerability detection, threat modeling, secure coding practices, and incident response" [17].	6	[17, 18, 26, 33, 35, 39]
Knowledge Acquisition and Understanding	"Reduce technical barriers and transform the programming process into a collaborative, conversational, and more accessible act" [2].	5	[2, 13, 20, 27, 33]
Management and Organizational Support	Assist in managing and "curating organizational information about day-to-day organizational practices" [27].	5	[3, 4, 16, 17, 22]

Continued on next page

Benefits	Description	#	References
Developer Well-being and Satisfaction	"prioritize interesting and fun tasks through automating menial, boring tasks" [27].	4	[4, 19, 27, 33]
Democratization and Accessibility	"shift the programming paradigm from a traditional code-centric approach to one that involves verbal interactions between the programmer and the artificial intelligence agent" [2].	2	[1, 2]

The two most frequently reported benefits, productivity and code quality, are reported as mutually reinforcing: acceleration does not come at the cost of quality, since AI helps generate code adhering to high standards, raising the accuracy and traceability of the final product [30]. Automating quality-assurance tasks, testing, debugging and repair supports the same synergy, and the mechanism is specific: LLMs leverage explicit feedback such as compiler error messages in follow-up prompts, iteratively fixing their own coding errors within a single conversation context. Beyond the codebase itself, the reported benefits extend to developer job satisfaction and to the democratisation of programming knowledge.

4.3.2 Consequences of AI and LLMs usage in Software Development

The second research question concerns the consequences that may arise from incorporating AI and LLMs into software development, in code generation and elsewhere. Those reported across the corpus are collected in Table 4.5.

Table 4.5: Consequences of using AI and LLMs in Software Development

Consequences	Description	#	References
Increased Operational Costs and Resource Constraints	"a substantial financial burden [...] when LLMs are deployed to handle a large number of tasks" [5].	20	[3, 5, 20, 22–24, 26, 28, 35, 38]
Reliability, Accuracy, and Hallucination Risks	Particularly problematic in complex or niche scenarios: "this AI can simply make things up or sometimes spit out things that don't even exist" [13].	15	[2, 4, 15, 16, 18–20, 22, 23, 30, 31, 34–37]
Security Vulnerabilities in Generated Code	"AI-generated code may harbor unique security risks in areas such as cryptographic implementation, memory management, and error handling" [39]. Existing tools do not close the gap: they detect only "46.7% of vulnerabilities in AI-generated code, a significantly lower rate than for human-written code vulnerabilities (66.4%)" [39].	15	[3, 13, 14, 16–20, 24, 26, 28, 33, 36, 37, 39]
Ethical, Legal, and Intellectual Property Issues	"questions about the ownership and licensing of that code will arise" when models are trained on publicly available, open-source projects [24].	8	[2, 3, 13, 17, 19, 22, 24, 35]
Homogeneity and Bias Propagation	LLMs increasingly train on LLM-generated code, tending to "reaffirm patterns seen in their training data (e.g., public GitHub repositories)" [35].	8	[3, 13, 14, 17, 19, 24, 34, 35]
Evaluation and Standardization Challenges	"It is urgent to establish an effective, objective, and complete evaluation system for large language models for code" [3].	7	[3, 14, 15, 23, 26, 32, 39]

Continued on next page

Consequences	Description	#	References
Risk of Over-reliance and Cognitive Decline	"if AI is constantly available, people will tend to become lazy and stop thinking for themselves" [13].	4	[2, 13, 19, 27]
New Overhead and Workflow Inefficiencies (Human Review)	LLMs still "require human oversight and review of their outputs before implementation" [29].	4	[2, 13, 22, 29]

The consequences tabulated above are technical, ethical and operational, with the technical and operational ones cited most often. Cost comes first: training, fine-tuning and deploying large-scale models can be substantially more expensive than traditional methods, offsetting the productivity gains they are meant to buy. Reliability comes second: LLMs frequently produce plausible but fabricated output, "hallucinations". The detection gap the table records between AI-generated and human-written code is the same problem from the tooling side, existing instruments being matched to failure modes AI-generated code does not share [39].

Beyond the technical domain, adoption carries risks to human factors, ethics and economic viability: over-reliance eroding developers' independence as instant assistance becomes the default, training-data bias perpetuated and magnified in generated code [24], and unresolved ownership and licensing where models train on open-source code [35].

4.3.3 Methodologies and management methods used to integrate AI and LLMs in Software Development

The third research question asks which methodologies and management methods are used to integrate AI and LLMs into software development. Table 4.6 lists those the corpus reports.

Table 4.6: Methodologies and Management Methods used to integrate AI and LLMs in Software Development

Methodologies	Description	#	References
In-Context Learning (ICL) / Few-shot Learning	"integrating context-specific information, in the form of an input prompt or instruction template, during the inference and thus without the need to perform gradient-based training" [38].	9	[15, 16, 20, 22, 26, 33–35, 38]
Prompt Engineering (PE) / Prompt-Driven Development	"The effectiveness of prompt engineering critically determines the performance of leveraging the capabilities of LLMs". [26]	5	[3, 20, 26, 32, 33]
Chain-of-Thought (CoT) Prompting	"A CoT is several intermediate natural language reasoning steps that lead to the final answer" [31].	5	[3, 16, 20, 26, 31]
Human-In-The-Loop (HITL) Paradigm	"The importance of the human contribution in identifying and resolving problems to maintain software quality is widely acknowledged by many authors" [29]. LLMs still "require human oversight and review of their outputs before implementation" [29, 35].	6	[2, 15, 24, 29, 35, 37]

Continued on next page

Methodologies	Description	#	References
Structured Prompting / Structured Chain-of-Thought (SCoT)	"SCoT denotes several intermediate reasoning steps constructed by programming structures" [31].	4	[23,26,30,31]
Multi-Agent Systems / Self-Collaboration Approach	"a self-collaboration framework with role instruction, which allows LLM agents to collaborate with each other to generate code for complex requirements" [21, 37].	4	[13,18,21,37]
Integration with External Tools and Domain Knowledge	"Combining the linguistic capability of LLMs with specialized domain knowledge to enhance output quality, functionality, or security" [13,37].	3	[3,4,13,37]
Other Single-Study Techniques	Five techniques each supported by a single study: Retrieval-Augmented Generation offers "a low-computation option by eliminating the need for parameter updates", though it "falls short of LoRA's effectiveness across all three CodeLlama model variants" [20, 38]; AI-Augmented Agile Development (AgileGen) focuses human involvement on the start (scenario decision-making) and end (acceptance decision-making) of each iteration [4]; Gherkin-based behaviour-driven testing supplements unclear user requirements with well-defined acceptance criteria [4]; Vibe Coding converts natural language or abstract intent into functioning software modules [1]; and integrating LLMs with traditional testing approaches yields a "1.4× to 2.3× improvement than the baselines" [37].	5	[1,4,20,37,38]

The methods reported range from prompt-level tuning to large-scale process frameworks, and they stratify into two tiers:

- **Prompt-level tuning.** In-Context Learning (9 articles, the most-discussed technique) and Prompt Engineering, whose quality critically determines effectiveness, form the base layer. Chain-of-Thought and its Structured variant apply the same principle, introducing programming constructs such as sequence, branch and loop into the reasoning steps [31].
- **Process-level frameworks.** Given LLMs' variable reliability, Human-In-The-Loop is considered mandatory for software quality [29]: developers retain oversight, reviewing AI output before implementation. AI-Augmented Agile Development (AgileGen) formalises this, reserving human decision-making for the start (scenario definition) and end (acceptance) of each iteration [4], preventing the error accumulation unsupervised multi-agent systems risk.

"Vibe Coding", mentioned once in the corpus, is recent and not yet widely studied [1], but reliably translating a "vibe" into working software is what the methodologies above exist to constrain.

4.3.4 Developers' and other stakeholders' opinions regarding the use of AI and LLMs in Software Development

The fourth research question concerns the human perspective. Reception differs enough between groups that the extracted opinions are tabulated separately in Appendix A.1, for the public and re-

searchers, developers and practitioners, and organisations and management; the separation is itself a finding.

- **Public and researchers.** Scepticism grounded in fallibility, not capability, dominates. General perception (six articles) treats these systems as a mixture of software and sorcery; the primary barrier reported is misinformation risk, since generated code may execute without error while failing the intended requirement, amplified by variability across repeated identical prompts.
- **Developers and practitioners.** The opposite emphasis: productivity and efficiency gains are the most-cited positive opinion (six articles), and LLMs are valued for knowledge acquisition, bypassing extended search or dependence on colleagues [13]. Two countervailing findings appear alongside it: a documented risk of over-reliance and skill erosion, and prompt design as a significant time investment that reduces the efficiency it promises [13].
- **Organisations and management.** Adoption is treated as a risk-management exercise. The strategic case, retaining valuable staff by automating routine work [27], is constrained by the group's most-cited concern, data security and privacy (six articles). The consensus strategy favours open models fine-tuned on organisation-specific data over commercial LLMs, raising operational cost and pushing teams towards medium-sized models; expectation management appears less often but matters equally, discounting the hype surrounding these tools when planning integration.

That is what makes the single most prevalent opinion across all three groups, the need for human oversight with seven articles, follow from evidence rather than from caution.

4.3.5 Industry sectors adopting AI and LLMs for Software Applications Development

The fifth research question asks which industry sectors are actively adopting AI for application development and what sector-level benefits they report; the extracted evidence is tabulated in Appendix A.2.

Adoption concentrates overwhelmingly in Software Engineering, Software Development and IT Consulting, both the origin and the primary proving ground for LLM-based tools. The reported use cases centre on code generation, completion, explanation and refactoring, and several sources describe a gradual shift in the developer role from writing code manually to orchestrating AI-enabled workflows in which the human directs, validates and refines automated output [4, 27, 29]. In consulting, LLMs are valued for accelerating onboarding and transferring client-specific business knowledge between projects; adoption also appears in infrastructure-level or mathematically intensive domains, where models must be adapted or augmented with structured knowledge to produce reliable output [3, 5, 17, 37].

Beyond the core **IT! (IT!)** sector, adoption clusters in finance, banking and regulated industries (eight articles), transportation and other safety-critical domains (seven), healthcare and biomedical research (five), education and training (five) and legal reasoning (four); education is an emerging area in which LLMs act as adaptive learning companions rather than replacements for educators, supporting exercise generation, personalised feedback and formative assessment [2, 15, 34].

In the first four, LLMs are introduced cautiously, with emphasis on reliability, auditability and regulatory compliance, and the literature is consistent that extensive human review remains mandatory

wherever incorrect or unverifiable output could produce safety risks, compliance failures or legal liability [1, 3, 26, 33].

No sector reports adopting these tools without a human check, and the sectors with the most to lose report the most explicit checks.

4.4 Discussion

Read across the five research questions, the review converges on a single pattern. Every benefit reported under RQ1 has a corresponding risk under RQ2 threatening the same property it claims: speed and code quality against hallucination, security vulnerabilities and homogenisation; democratised access against over-reliance and skill erosion. The field's answer to that tension is not automation but control: every prevalent method under RQ3, In-Context Learning, Prompt Engineering, Chain-of-Thought and above all Human-In-The-Loop, exists to shape or supervise AI output before it is trusted, and none proposes removing the human; every stakeholder group under RQ4 pairs enthusiasm with a caveat, human oversight being the one consensus surviving their differing incentives; and under RQ5 it is the organising variable for adoption itself, broad and unconditional where an incorrect output is cheap, explicitly conditioned on verification where it is expensive, as in finance, transportation or healthcare. The absence of a standardised, secure integration methodology, not a shortfall in model capability, is therefore what the literature identifies as the actual barrier to broader adoption, the gap Chapter 5 formalises. Table 4.7 restates the pattern phase by phase; read down its challenge column and the same requirement recurs at every one, a human check of the model's output against something written before that output existed.

Table 4.7: Impact of AI and LLMs across Software Development Lifecycle Phases

SDLC Phase	Main Challenges
Requirements Analysis	Risk of ambiguity amplification, overconfidence in AI-generated requirements, and lack of domain context; increased need for human validation early in the process.
Design	Limited understanding of system-wide constraints; risk of inconsistent or suboptimal architectural decisions without expert oversight.
Implementation	Hallucinated code, inconsistent outputs, security vulnerabilities, and reduced trust due to output variability; heavy reliance on human review.
Testing	Difficulty validating correctness of AI-generated tests; increased testing complexity due to unpredictable AI-generated code.
Deployment	Risk of misconfiguration or insecure defaults; limited contextual awareness of operational environments.
Maintenance	Potential over-reliance on AI explanations; challenges in maintaining long-term system understanding and accountability.

5

Research Problem

The review reported in Chapter 4 shows AI and LLMs adopted across sectors and across every phase of the SDLC, in a field evolving fast enough for new models and tools to appear continuously, and shows those benefits accompanied by challenges that obstruct successful implementation. The most consequential, and the most consistently reported, is the absence of established, clear methodologies shaping the proper use of these tools in professional software development [13–16].

Several frameworks incorporating AI into the development process have been proposed and are summarised in Table 4.6; as Chapter 1 establishes, they share no common foundation regarding lifecycle integration, management practice or evaluation criteria [3]. Organisations therefore face difficulty not only in adopting AI-supported practice but in assessing its impact on software quality, security and maintainability, no standardised benchmarks or performance metrics existing against which to do so: the field is still searching for structured and repeatable integration patterns rather than converging on them.

As Chapter 1 also establishes, established methodologies, including Waterfall, Scrum, Kanban and the Agile Manifesto, were designed for environments in which every contributor is human [4], and fall short in AI-driven contexts in three specific ways. High-speed AI-assisted coding accelerates implementation while creating pressure upstream, in requirements clarification, and downstream, in testing, verification and maintenance. They provide no explicit structure for governing AI participation, for assessing model reliability, or for mitigating the resulting risks. And they assume all contributors are human, whereas a model is a new category of collaborator whose output carries no author and therefore no obvious owner.

The empirical case for treating this as a process problem rather than a capability problem is set out in Chapter 1 and rests on two independently reported findings: security performance does not improve as functional correctness improves, so generated code that works is not thereby safe [9], and acceleration

at the point of authorship relocates to human review rather than disappearing, without reaching delivery outcomes [12]. Both sources are industry-reported rather than peer-reviewed and are cited as such; their value lies in agreeing on the mechanism rather than in the precision of any single figure, neither security nor delivery improving as a by-product of faster generation.

The core research problem addressed in this work is therefore the lack of patterned guidance, methodological rigour and standardised evaluation metrics for the systematic integration and management of AI and LLM-based tools across the SDLC, while preserving human oversight, software quality and security, and organisational accountability. This gap limits the ability of organisations to realise the benefits of AI-assisted software development sustainably.

6

Research Proposal

Contents

6.1 Objectives	22
6.2 Purpose and Scope	22
6.3 Design Principles	23
6.4 Lifecycle Phases and the AI Interaction Mapping	24
6.5 Roles and Responsibilities	26
6.6 Governance Mechanisms and Quality Gates	28
6.7 Work Organisation	30
6.8 Operational Playbooks	31
6.9 Positioning Against Alternatives	32
6.10 Summary	33

This chapter presents the second and third activities of the DSRM: the definition of objectives for a solution to the research problem formalised in Chapter 5, and the design of the artefact that satisfies them, a process-oriented framework for governed Human-AI collaboration across the SDLC. The framework is described here as a methodology, independently of any implementation; its instantiation as a working system is the subject of Chapter 7.

6.1 Objectives

The systematic literature review in Chapter 4 led to the problem formalised in Chapter 5: the lack of structured guidance, methodological rigour and standardised evaluation practice for integrating these tools into development processes. The research objectives address that problem directly, and Table 6.1 links each identified gap to the framework objective that resolves it.

Table 6.1: Mapping between Research Questions, Key Findings, Problem Aspects, and Framework Objectives

RQ	Key Finding from SLR	Problem Aspect Addressed	Resulting Framework Objective
RQ1	Usage is unevenly distributed across SDLC phases and lacks structured integration.	No structured guidance on where and how AI should be applied across the SDLC.	Define a process framework mapping AI-supported activities to specific phases.
RQ2	Recurring risks in reliability, validation, security and insufficient human oversight.	Insufficient methodological support for risk, quality and accountability.	Integrate governance mechanisms, human-in-the-loop practice and quality gates.
RQ3	Significant variation among approaches and no consensus on managing adoption.	Fragmentation of existing AI adoption methodologies.	Consolidate practice into a unified, repeatable process-oriented framework.

6.2 Purpose and Scope

The artefact is a method in the sense the DSRM gives that term: phases, hats, artefacts and decision rules prescribing how a team organises the participation of AI across the SDLC. It is stated at the level of process rather than of tooling, so that it can be assessed independently of the system instantiating it in Chapter 7 and adopted against a different tool stack. It covers AI-assisted activity in every phase rather than the code-authoring phases alone, the collaboration and model-selection practices that activity requires, expressed as process obligations rather than a catalogue of prompts, and the governance mechanisms of Section 6.6.

It excludes the training or fine-tuning of LLMs, model architecture design, Machine Learning Operations (MLOps) pipelines and tool-specific detail. These are boundaries rather than omissions, following from Chapter 5's concern with teams that *consume* general-purpose models rather than *produce* them: MLOps governs a model's lifecycle as a deployed asset, while this framework governs the lifecycle of software a model helped write, and a pipeline can be mature under the first while the second remains ungoverned, which is the deficiency Chapter 4 reports. Tool-specific detail is excluded for the same reason Chapters 7 and 9 are kept separate: a framework prescribing a vendor cannot be evaluated independently of it.

6.3 Design Principles

Seven principles govern every subsequent design decision. Each is stated in Table 6.2 together with the evidence motivating it, since a principle asserted without grounding is a preference, and a framework built out of preferences cannot be defended as a research contribution. Where the grounding is a design commitment rather than an empirical finding, that is said so.

Table 6.2: The seven design principles and the evidence grounding each

Principle	Statement and grounding
Human-in-the-Loop by Design	Every AI-generated artefact is subject to explicit human review and acceptance before entering the product. Adoption and trust diverge, at around 90% daily use against roughly 30% reporting low or no trust [7], and benchmarking shows the distrust warranted: the best agent and model combination was functionally correct 61% of the time but secure only 10.5% [8], and 45% of samples across more than a hundred models introduced an OWASP Top 10 vulnerability, with security flat as correctness improved [9].
AI as Augmentation, Not Substitution	Humans retain responsibility for technical, architectural and quality decisions. In a randomised controlled trial developers took 19% longer with assistance while estimating they had been 20% faster [11]; substituting judgement removes the only faculty able to notice such a discrepancy, and the SLR reports reduced code understanding as a recurring consequence of unsupervised delegation [2, 13, 19, 27].
Process-Level Governance of AI Usage	AI participation is governed through repeatable mechanisms making contributions visible, traceable and attributable. AI amplifies: capable teams become measurably more capable, dysfunctional teams measurably less so [7]. If the surrounding process sets the sign of the effect, governance must live in the process rather than in prompt discipline.
SDLC-Wide and Phase-Aware Integration	Participation is defined per phase, with hats and controls adapted to that phase's risks. Usage is unevenly distributed and concentrated on code generation, leaving requirements, design and maintenance with little guidance [3–5]; governing only the well-served phases would reproduce the gap the framework exists to close.
Functional, Not Positional, Accountability	Every role named is a function that must be performed, not a person who must exist, and for each AI-assisted activity a named hat, not a named person, is accountable for initiation, validation and acceptance. The structural half is a design commitment rather than an empirical claim, argued in Section 6.5; its visible consequence is that no role is named whose only content is oversight. The accountability half follows from it directly: when an artefact has no identifiable author no party is answerable for its defects, and ownership is among the concerns most frequently raised by practitioners and organisations [3, 13]. Accountability attaches to a function so that it survives the absence of any individual.
Risk-Proportional Validation and Control	Validation rigour is proportional to potential technical, organisational and ethical risk. The defect profile is not uniform: vulnerability rates reach 72% for Java against 45% overall and cluster in specific weakness categories [8, 9]. A uniform control is either too costly for the routine case or too weak for the dangerous one, which is why remediation separates a Fast Lane from a Secure Lane.
Spec-Anchored Continuity	A persistent versioned specification is maintained alongside the codebase and is what every AI-assisted activity is constrained by and validated against. As established in Section 3.5 this instantiates spec-driven development rather than inventing it [40, 41]. The limitation belongs with the claim: its own principal source describes the practice as emerging and its scaling behaviour as an open risk [40], which Chapter 9 treats as something to test rather than assume.

6.4 Lifecycle Phases and the AI Interaction Mapping

This section answers DRQ1. The framework organises the lifecycle into six phases, preceded by a Strategic Alignment stage that sits outside the lifecycle proper because it is performed before the Squad is engaged at all: it decides whether a Unit of Work should exist, whereas the six phases decide how one is built. The six are Discovery and Requirements; Design, Prototyping and Architecture; Implementation; Testing; Deployment; and Maintenance. Table 6.3 fixes, for the stage and each of them, what enters, what the model is permitted to produce, what leaves, which hat is accountable, and which control must be satisfied before exit. It is the central artefact of this chapter and is not restated in prose.

Two mechanisms the table names are defined here, since both recur throughout the chapter. The **Interactive Bridge** is the interaction pattern governing Discovery and Requirements: the model drafts scenarios in plain natural language for human critique, the Trio confirms, adds, deletes or modifies them, and only the approved result is compiled into formal Gherkin and anchored. Separating critique from compilation matters because natural language is the medium in which a domain expert can detect a wrong assumption, whereas formal syntax invites review of the syntax instead. The **Consistency Factor** is the strict automated test set produced before generation begins, derived from both the functional and the technical specification, constraining what the model may produce rather than assessing it afterwards; it is specified in full in Section 6.6.1.

The framework's most direct precedent is the AI-Driven Development Lifecycle (AI-DLC) published by Amazon Web Services [42], which organises work into Inception, Construction and Operations, replaces sprints with *Bolts*, short cycles measured in hours or days, replaces Epics with *Units of Work*, and governs AI participation through collaborative rituals of which *Mob Elaboration* is adopted here; these three terms are taken directly from AI-DLC with no originality claimed for them; its reported ten-to-fifteen-fold productivity improvement is vendor-reported, unreplicated and not carried here as evidence, adopted only as process vocabulary. Most of the expansion from three phases to six is decomposition rather than disagreement, since Discovery and Requirements together with Design, Prototyping and Architecture occupy the territory it calls Inception, and Implementation corresponds to Construction. Two extensions are substantive.

The first is a decoupled Testing phase. In AI-DLC, build and test close Construction, performed in the same cycle by the same participants that generated the code. The objection is circularity, not thoroughness: tests derived by the same model, from the same specification, encode the same reading of the requirement as the implementation, so passing them confirms internal consistency rather than external correctness. Benchmark evidence shows exactly this asymmetry: agent-generated code passes functional tests far more often than independently held security tests, and the gap does not close as models improve [8,9]. Testing therefore becomes its own phase, with its own environment, hat and gate, working from the locked functional specification rather than the implementation; the unit of validation changes at this boundary, from the Bolt's task to the assembled User Story, the smallest unit at which business value is observable.

The second is a distinct Maintenance phase with a diagnostic firewall. AI-DLC's Operations phase is deployment with monitoring, defining no mechanism for classifying post-deployment signals or routing them to an owner. The signal is heterogeneous, a defect against an agreed specification, a request never specified, or evidence the specification was wrong, and treating all three as defects erodes the

specification, each undocumented accommodation widening the gap between what the system does and what the record says it should. A signal is therefore classified before any code is opened, and Context Reconstruction limits the developer to the specific report, the test evidence and the isolated fragment implicated, prohibiting whole-project context, a control for the reason developed in Section 6.6.2 [8].

The phases are listed in an order, but that order is not the shape of the process: Maintenance neither terminates the lifecycle nor patches in place. Its exit is always an entry into a named earlier phase, whose gate then applies in full: a business deviation routes to the Project Manager (PM) hat for a value decision and, if approved, re-opens Discovery and Requirements; a specification gap re-opens Design, Prototyping and Architecture with the technical specification amended and re-versioned; a production defect re-enters Implementation as a Fix-Bolt and returns through Testing. This is not Waterfall with a maintenance tail: in Waterfall, feedback is measured against a specification treated as frozen, whereas here the specification is exactly what is re-opened, re-approved and re-versioned, and code is regenerated from it. What the framework refuses is the unnamed loop, a change applied to production code with no corresponding change to the specification it is supposed to satisfy, since that is how traceability is lost in practice. Figure 6.1 shows the six phases and these three loop-backs; Table 6.3 is the detail behind each box.

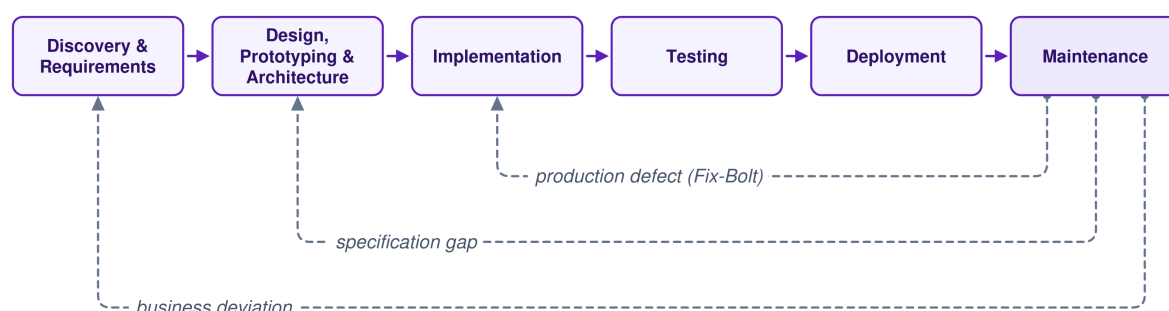


Figure 6.1: The six lifecycle phases in sequence, and the three named loop-backs out of Maintenance. Each loop-back re-enters a specific earlier phase, whose gate then applies in full - Maintenance never patches production code without a corresponding change to the specification.

Table 6.3: The six lifecycle phases and the preceding Strategic Alignment stage: permitted AI contribution, output, accountable hat, and the control that must be satisfied before each is exited. Each phase takes as input the output of the one above it.

Phase	Role of AI	Output	Human accountability	Mandatory control
Strategic Alignment (pre-lifecycle)	Assists market analysis; drafts candidate objectives and indicators	A Unit of Work carrying a stated business justification, Objective and Key Results (OKRs) and Key Performance Indicators (KPIs)	PM hat aligns the product vision with stakeholders and owns the justification	Stakeholder alignment and OKR sign-off before the Unit of Work reaches the Squad

Continued on next page

Phase	Role of AI	Output	Human accountability	Mandatory control
Discovery & Requirements	Drafts fractional User Stories in natural language; compiles approved scenarios into formal Gherkin	Approved User Stories sized XS or S, with Gherkin acceptance criteria and stable scenario identifiers	The Trio validates correctness, completeness and business value through the Interactive Bridge	Mob Elaboration and the Discovery Gate
Design, Prototyping & Architecture	Proposes architectural options; generates User Interface (UI) structure and a formal technical specification	Approved prototype and an anchored technical specification (interface contract, data model, runtime contract)	Design Lead hat validates usability; Tech Lead hat validates feasibility, scalability and integration risk	Design Gate; no task enters the Bolt Backlog until both validations pass
Implementation	Generates and refactors code constrained by the Consistency Factor test set	Task-level code passing the pipeline and the Consistency Factor	Developer hat validates output, owns correctness and security, and must be able to explain the logic	Pre-Bolt architectural lock, the Bolt micro-cycle, and the Implementation Gate at story level
Testing	Drafts end-to-end Behaviour-Driven Development (BDD) scripts derived from the approved Gherkin	A story-level verdict and, on failure, the exact failing evidence	Quality Assurance (QA) hat verifies and refines generated scripts before execution and probes edge cases manually	The QA Validation Playbook and the Testing Gate, in a separate environment decoupled from the Bolt
Deployment	Answers the infrastructure delta question; generates configuration, Infrastructure as Code (IaC) or migrations only when required	A released increment and a persistent context updated to the live state	Whoever performs the deployment approves it and reviews its security implications	Deployment Gate, performed by the deploying hat; no separate gatekeeping team is assumed
Maintenance	Reconstructs a deliberately narrow context to explain one specific failure and proposes a bounded fix	A classified signal routed to the phase that owns it, with a verified diagnosis where applicable	Developer or PM hat classifies the signal and must verify any diagnosis before it is trusted	Diagnostic firewall (Context Reconstruction, whole-project context prohibited) and governed loop-back to a named phase

6.5 Roles and Responsibilities

Every role named is a hat: a function the framework requires to be performed, not a person who must be hired. One individual may wear several hats, and on a small team a hat may go unfilled, its responsibility falling to whoever performs the adjacent work. Table 6.4 summarises the hats and the gate each owns. Three reasons justify expressing responsibility as a function rather than a position: it fits a target population of anyone with a technical background working alone or in a small team, whom a staffed-organisation role model would exclude; accountability expressed as a title lapses the moment the title is vacant, whereas a function survives reassignment; and the third is evidential, concerning what happens when structural models are copied as blueprints.

The Trio, comprising the Product Owner (PO), Tech Lead and Design Lead hats, is inspired by the Squad concept popularised in the Spotify Model, and citing that model honestly requires citing its limits alongside it. It was published as a description of one company at one point in time, not a method for adoption elsewhere [43], its own principal author has since called it inspiration rather than a blueprint [44], and a former employee's first-hand account names two failure modes never resolved internally: unclear matrix decision authority, and autonomy without accountability [45]. The framework

hardens against exactly these two. Matrix ambiguity is answered by the Tech Lead hat's architectural lock, fixing the technical boundary before any Bolt begins; autonomy without accountability is answered by separating logistics authority in the Bolt Master hat from technical authority in the Tech Lead hat, with every gate attached to a named function. A closer structural match is Team Topologies' stream-aligned team, which treats cognitive load as an explicit design constraint [46]: limiting developer agency over scope during a Bolt reduces the open decisions held while validating generated code, the activity Chapter 1 identifies as the new constraint. The PM hat, sitting outside the Squad, is structurally closer to the single-threaded leader Amazon adopted after its two-pizza structure stopped surviving scale [47].

Deployment approval and infrastructure-security review are mandatory, but owned by whoever performs the deployment, self-reviewed solo or peer-reviewed where a second developer exists, rather than by an external DevOps Alliance: naming a governance function without naming who holds it is the defect explicit responsibility exists to prevent, since a body defined only by what it supervises can be assumed by the process to exist and never actually staffed.

The gates do not relax as the team shrinks. On a solo project each gate is self-checked, but what it requires is unchanged: the evidence in Table 6.5 must exist, be recorded, and be examined deliberately at the moment the gate is reached, not assumed to have been examined during the work that produced it. A gate's value lies primarily in forcing an explicit, separate act of validation against a written specification that predates the artefact, which is what makes self-review meaningful even when the reviewer and the author are the same person.

Table 6.4: The hats: the function each discharges and the gate each owns

Hat	Function the framework requires	Gate owned
PM	Stakeholder alignment, OKRs and KPIs, and the business justification of a Unit of Work before it reaches the Squad; receives classified business deviations from Maintenance.	Strategic alignment sign-off
PO (Trio)	Business value and prioritisation of User Stories; leads the critique of AI-drafted scenarios during Mob Elaboration.	Discovery, with the Trio
Tech Lead (Trio)	Technical specification, architectural feasibility and integration risk; decomposes stories into tasks and flags high-risk ones before any Bolt begins.	Design (technical) and the pre-Bolt architectural lock
Design Lead (Trio)	Prototype or visual element for every feature, and its validation through usability testing.	Design (usability)
Bolt Master	Continuous flow: prioritises the Bolt Backlog, manages dependencies, assigns by capacity once the Tech Lead hat has approved.	None; holds logistics authority deliberately separated from technical authority
Developer	Executes tasks with the model, generates and validates the Consistency Factor, and remains answerable for correctness, security and being able to explain the logic.	Implementation
QA	Validates the assembled story against the approved Gherkin in a separate environment, verifying generated BDD scripts before execution and probing edge cases manually.	Testing

Continued on next page

Hat	Function the framework requires	Gate owned
Deploying hat	Not a distinct role: whoever performs a release approves it and reviews the security of any generated IaC or configuration.	Deployment

6.6 Governance Mechanisms and Quality Gates

6.6.1 Bolts as the Unit of Execution

A Bolt is the smallest execution iteration the framework recognises: the implementation of one task within a User Story, strictly inside the development environment and measured in hours rather than weeks. The term is adopted from AI-DLC [42]; what the framework adds is the sequence of obligations around it, since a short cycle without a fixed reference is a fast way to accumulate divergence.

Before any code is generated, the Tech Lead hat feeds the locked context (the approved Gherkin, the prototype and the technical specification) to the model, verifies the task list it proposes, and flags high-risk tasks. This **architectural lock** makes the technical boundary a matter of record rather than negotiation, letting tasks on either side of an interface proceed in parallel. The Bolt Master hat then assigns by capacity, giving developers the story's intent, not authority over the technical scope the lock has settled. A Developer hat pulls one task, at which moment the Bolt begins and Bolt Cycle Time is measured from. Developer and model then produce the **Consistency Factor**, a strict automated test set derived from both specifications that constrains generation rather than reporting on it afterwards; the model generates against that constraint and the developer refines the result as an active editor, not an approver of a finished proposal. A task is done when the code passes the pipeline *and* the developer can explain the logic, since code no one can explain has been substituted for engineering rather than added to it. When every task is done, the story passes the Implementation Gate into staging.

The **Fix-Bolt** is the single exception: triggered by a failed Testing Gate or a verified defect, it omits task breakdown because the context already contains what that step would produce, the exact failing scenario and its evidence. It is justified by what is already known rather than being a licence to skip planning for a change that merely feels small. Its outcome is routed by risk, where the risk-proportional principle becomes operational: a low-risk patch takes the **Fast Lane** directly to the delivery pipeline without re-entering formal QA, while a higher-risk one takes the **Secure Lane** back to staging for a Regression Bypass, in which QA re-runs the existing suite before verifying the targeted patch.

6.6.2 Spec-Anchored Continuity

Spec-Anchored Continuity requires every AI-generated artefact to carry forward the context established by previously validated phases. The mechanism is a persistent versioned specification maintained alongside the codebase and split into two layers with different lifetimes. The **global layer**, the Memory Bank, holds what does not change between stories: architectural rules, technology-stack decisions, security policies and recorded workarounds for already-diagnosed defects. It is written rarely, amended deliberately, and applies to every generation on the project. The **local layer**, the Feature Specs, holds what is specific to the work in hand, the approved Gherkin acceptance criteria and the technical specification produced at the Design Gate; it is written once per story, locked at a gate, and amended only

through a recorded versioned amendment. The split is what makes the mechanism usable, since a single undifferentiated context grows monotonically until it contains more irrelevant material than relevant, reintroducing the problem it was meant to solve.

The central argument is that narrowing the model's working context is a control rather than a convenience: a convenience makes a correct outcome easier to reach, whereas a control is a measure whose absence makes an incorrect outcome undetectable. Anchoring satisfies the second definition on three counts. It supplies a reference that existed before the generated artefact did, the precondition for validation being possible at all. It bounds what the model may act upon, an unbounded context being the condition under which a model produces a wide, confident and substantively wrong change, which is why whole-project context is prohibited during Maintenance diagnosis [8]. And because both layers are versioned and every post-lock edit is a dated amendment, divergence between what was agreed and what was built becomes a detectable event rather than an unattributable drift. This is the spec-driven-development instantiation named in Table 6.2; a recent preprint independently argues, from a multivocal review, that specification discipline rather than model capability is the binding constraint on dependability [41]. The contribution is not the idea of anchoring but the specification of where it is mandatory, which gate locks each layer, and what constitutes a legitimate amendment.

6.6.3 Quality Gates

A quality gate has three named components: an owning hat, a specific body of evidence that must exist and be examined, and a defined consequence when it is absent or inadequate. A checkpoint missing any of the three is a ceremony rather than a control, since it can be passed without anything being established. Table 6.5 specifies the five gates on those terms. Notably, failure never discards work and never silently degrades it, but returns it to a named earlier activity with the evidence attached, the same routing discipline Maintenance applies to post-deployment signals.

Table 6.5: The five quality gates: owner, required evidence, and the consequence of failure

Gate	Owning hat	Evidence required to pass	Consequence of failure
Discovery	The Trio	Testable Gherkin acceptance criteria approved through the Interactive Bridge; every story sized XS or S; the functional specification locked and versioned.	Scenarios return to Mob Elaboration; oversized stories are decomposed before re-submission.
Design	Design Lead and Tech Lead hats	An approved prototype validated by usability testing; a technical specification covering interface contract, data model and runtime contract, anchored into the persistent context.	Feedback returns to prototype generation or architectural specification; no task may enter the Bolt Backlog.
Implementation	Developer hat	Every task complete, passing the pipeline and the Consistency Factor, with the developer able to explain the generated logic.	The story does not enter staging; incomplete or unexplained tasks return to the Bolt Backlog.

Continued on next page

Gate	Owning hat	Evidence required to pass	Consequence of failure
Testing	QA hat	The assembled story validated in staging against the locked Gherkin through verified BDD scripts, with edge cases probed manually.	A Fix-Bolt is triggered with the exact failing evidence attached; the story re-enters staging afterwards.
Deployment	Whoever performs the deployment	Explicit approval of the release and a security review of any generated IaC, configuration or migration; the persistent context updated to the live state.	The specific objection is fed back for a compliant regeneration; the release does not proceed.

6.6.4 Governance Metrics

Three governance metrics are defined, chosen to measure whether the collaboration is safe and whether it is flowing rather than to measure output volume, the quantity Chapter 1 shows rising without a corresponding improvement in delivery [12]. **Bolt Cycle Time** is the elapsed time from a task being pulled as a Bolt to its being marked done, defined at task level rather than story level because a story-level measurement conflates execution with the queueing, review and gate transitions surrounding it, and the claim about continuous flow is a claim about execution. **Context Traceability Rate** is the proportion of deployed Bolts and stories whose context chain is complete, the deployed artefact resolving back to a locked technical specification, to the functional specification it derived from, and to the Unit of Work that motivated it; it measures whether Spec-Anchored Continuity is holding in practice rather than merely being prescribed. **AI Defect Escape Rate** is the proportion of production defects originating in AI-generated logic that passed the Testing Gate, and it is the metric that detects a gate which has become ceremonial, since a gate satisfied without being exercised shows up here and nowhere else.

These are expressed in vocabulary continuous with established AI governance standards rather than bespoke to this work: the National Institute of Standards and Technology (NIST) AI Risk Management Framework's Govern, Map, Measure and Manage [48] correspond respectively to this chapter's design principles, phase mapping, metrics and gates, and ISO/IEC 42001:2023 offers a complementary plan-do-check-act vocabulary for the same loop [49]. This is a claim about shared language only: neither the framework nor Apex has been audited against either standard, and nothing here asserts compliance or intent to certify, only that these constructs occupy the conceptual space standards bodies are converging on.

6.7 Work Organisation

The framework replaces the familiar hierarchy of epics, sprints and sprint backlogs with continuous flow, the most disruptive single element of the proposal and therefore the one requiring the strongest empirical justification. Sprint cadence manages a world in which implementation is the binding constraint: a fixed-length iteration lets a quantity of implementation work be forecast, committed to and inspected at a boundary, calibrated to how long a useful amount of implementation takes. The evidence in Chapter 1 indicates that premise no longer holds where AI assistance is heavy: developers in a randomised controlled trial were measurably slower with assistance while believing they had been faster [11], and

telemetry across 1,255 teams pairs the merged-pull-request increase reported there with pull requests 154% larger, 91% longer review times, and no significant correlation with delivery outcomes [12]. The constraint moved rather than disappeared, so a cadence calibrated to the old one now obscures the new one. The structural response has two parts: execution compressed into Bolts, so the queue in front of validation refills in small increments rather than batches, and validation decoupled into its own phase, environment, hat and gate, so the constraint is scheduled and measured in its own right. Both are falsifiable: if review is not the constraint in a given setting, Bolt Cycle Time and the QA queue will say so.

Three levels of work are retained, and the vocabulary of the upper two follows AI-DLC [42]. A **Unit of Work** is the core business problem, owned by the PM and PO hats, the level at which business justification is argued and acceptance criteria are generated as a group; grouping them this way rather than requirement by requirement is itself supported by evaluation, an epic-organised pipeline outperforming a requirement-aligned baseline across 107 requirements on expert-rated correctness (4.61 against 4.14 out of 5) and completeness (4.31 against 3.50) [50]. **User Stories** are the tangible deliverables, constrained to extra-small or small, the level at which QA tests because they are the smallest unit at which business value is observable; the size constraint is load-bearing, since a story describable by a handful of scenarios is small enough for a reviewer to hold in mind while validating generated code against it. **Tasks** are the granular technical steps executed as Bolts, entering the **Bolt Backlog**, a continuous prioritised queue, only after decomposition with the model, Tech Lead verification, and risk flagging. Developers have limited agency over task creation and selection deliberately, not out of distrust: the decisions withheld are exactly those the architectural lock has settled, for the cognitive-load reason argued in Section 6.5. The team's project management tool is expected to expose states corresponding to the phases as guidance rather than a literal requirement; what matters is that each column maps unambiguously onto one phase.

6.8 Operational Playbooks

The phases, gates and hats state what must hold; the six playbooks in Table 6.6 state how the recurring procedures are performed, each with its trigger, its participants and its termination condition. A team that follows these six and observes the gates in Table 6.5 is operating the framework, irrespective of the tooling it uses. In Maintenance, the prohibition on whole-project context is the single most restrictive rule in the framework, for the reason given in Section 6.4. In the Fix-Bolt, the Regression Bypass re-runs the existing BDD suite rather than regenerating it, because a regenerated suite is a different measurement instrument, and comparing a patched system against it cannot distinguish a fixed defect from a changed test.

Table 6.6: The six operational playbooks: trigger, procedure, and termination

Playbook	Trigger	Procedure	Terminates when
Mob Elaboration	A Unit of Work brought to the Squad	The Trio supplies intent, personas and domain constraints; the agent drafts fractional stories in natural language; the Trio cross-examines for value, feasibility and coherence, issuing confirm, add, delete or modify instructions until satisfied; the agent then compiles the approved scenarios into Gherkin and anchors the result.	The Discovery Gate passes and the functional specification is locked
Design and Prototyping	Locked Gherkin acceptance criteria	The Design Lead hat prompts for screens and flows; the Tech Lead hat prompts for interface contract, data model and runtime contract and anchors them; usability testing and architectural review run in parallel.	Both validations pass and tasks enter the Bolt Backlog
Maintenance and Evolution	A post-deployment signal	Classify before any code is opened: a business deviation or specification gap routes to the PM hat for a value decision and re-opens the owning phase; a technical defect routes to a Developer hat, who narrows the context to the report, the test evidence and the isolated fragment, asks only why that logic failed, and verifies the diagnosis manually.	The signal is classified and routed, with a verified diagnosis where applicable
Fix-Bolt	A failed Testing Gate or a verified defect	Pulled immediately with task breakdown skipped; the agent resolves only the specific defect while adhering to the anchored specification; the developer reviews for newly introduced vulnerabilities, not only symptom resolution; existing and new unit tests must pass.	Low risk takes the Fast Lane to the pipeline; higher risk takes the Secure Lane through a Regression Bypass in staging
QA Validation	A story that has passed the Implementation Gate	The QA hat opens the locked functional specification rather than the implementation, prompts for end-to-end BDD scripts and verifies them manually <i>before</i> execution, since an unverified script can report a passing scenario it never exercised, then probes edge cases exploratively.	The Testing Gate passes, or a Fix-Bolt is triggered with the failing evidence
Deployment and Release	A story that has passed the Testing Gate	The agent answers one bounded question, whether the release requires new infrastructure, configuration or migrations, and generates artefacts only when it does; whoever deploys reviews any generated artefact for vulnerabilities and policy violations.	The Deployment Gate passes and the persistent context is updated to the live state

6.9 Positioning Against Alternatives

A design is only defensible against named alternatives. Four are engaged below, each stated at its strongest, credited with what it genuinely does better, and met with the evidence motivating a different choice; the sources cited throughout ground individual pieces of an original synthesis, not a joint

blueprint this work reproduces.

- **No framework at all.** Premise: model capability suffices and process overhead is pure cost, correct for a short-lived prototype with no maintenance horizon, no coordination cost, no artefact to maintain, no gate to turn ceremonial. Documented failure mode, not hypothetical: Section 6.3 already establishes that functionally correct agent output is insecure at scale and that security does not improve as correctness does [8, 9], so waiting is not a valid strategy. Flat, not merely lagging, is what turns validation from discipline into a structural requirement.
- **Adopt AI-DLC as published and unmodified.** Premise: a coherent three-phase model already exists from a credible source, and extending it adds complexity without control [42]; genuinely simpler, operated at commercial scale, less process surface for a small team. The objection is narrow, the two gaps argued in Section 6.4: build and test folded into the close of Construction, and an Operations phase with no published mechanism for classifying post-deployment feedback. The divergence fills gaps its own documentation leaves open; its vendor-reported, unverified productivity claims are not grounds to adopt it unmodified.
- **Keep Scrum or SAFe cadence and treat AI as a faster development environment.** Premise: existing ceremonies already provide planning, review and retrospection, and AI only changes the speed of one activity; not trivial, since the cadence is understood, tooled, compatible with planning cycles, and costs nothing in retraining. Section 6.7 already shows why that premise fails, pairing the same controlled-experiment and telemetry evidence: developers underestimate the slowdown assistance introduces [11] while the constraint has moved from implementation to review [12]. The retrospective this alternative relies on to catch that is a weak instrument for catching it, and an iteration boundary calibrated to implementation keeps reporting on the activity that is no longer binding.
- **Metrics-only governance, with no phase structure.** Premise: an organisation can track the right capabilities and outcomes, publish them, and let teams self-organise. The most serious of the four: far cheaper to adopt, presumes no lifecycle shape, adapts to context, and its own literature legitimately warns that no single metric can stand for the whole and any measure elevated to a target stops being a good one [51]. The response is a distinction, not a rebuttal: metrics measure whether governance is working, they are not themselves a governance mechanism. A dashboard can accurately report rising review time and escaping defects while containing no mechanism for what a model may act upon, no point where an artefact must be checked against a prior specification, and no rule for where a post-deployment signal is routed, which is why the framework defines its own metrics in the same vocabulary rather than proposing an alternative to measurement.

6.10 Summary

DRQ1 and DRQ2 are answered by the artefact this chapter has specified: the six-phase model of Section 6.4 with the Strategic Alignment stage preceding it and the mapping in Table 6.3, and Section 6.6's governance set of Bolts, the Consistency Factor, Spec-Anchored Continuity, five gates and three metrics. Two properties of that answer are worth isolating, because a phase list and a gate list alone would

not achieve them: the model is constrained at every phase to that phase's anchored specification rather than the whole project, and accountability attaches to functions rather than positions, so it survives a team of one while every failed gate routes work back to a named phase rather than discarding it.

Both answers are design claims and not yet evidence. The framework's constructs are an original synthesis argued from the literature but not demonstrated in use, and spec-driven development, on which Spec-Anchored Continuity rests, is characterised by its own principal source as emerging with unsettled scaling behaviour [40]. Chapter 7 presents Apex, the reference implementation, and reports which constructs were implementable as specified, which were weakened or proxied, and which resisted implementation, the substance of DRQ3.

7

Implementation

Contents

7.1 Purpose and Positioning	35
7.2 Architecture	36
7.3 Operationalising the Framework	38
7.4 The Governance Analytics Surface	44
7.5 Design History and Practitioner Feedback	46
7.6 Quality Assurance of the Implementation	47
7.7 Limitations of the Implementation	48

The framework presented in Chapter 6 describes a methodology. This chapter presents Apex, the working software system instantiating it, which serves both as evidence that the framework's governance constructs are implementable and as the vehicle through which the framework is demonstrated and evaluated in Chapters 8 and 9. In DSRM terms the two are distinct artefacts produced by the same Design and Development activity: a method and its instantiation. The chapter answers DRQ3, and answering it honestly means reporting which constructs resisted implementation as well as which did not.

7.1 Purpose and Positioning

Apex is not a coding agent and is not positioned against one. A coding agent accelerates the writing of code; Apex governs whether the code that results still answers the question that prompted it. The

two are complementary, and the distinction is the whole reason the system exists: acceleration without anchoring is the condition Chapter 1 shows moves the bottleneck to review rather than removing it. Every artefact Apex produces carries a stable specification identifier back to the requirement it derives from, so divergence between what was agreed and what was built becomes a detectable event rather than a drift nobody is positioned to notice.

What Apex deliberately does not do is as much part of its definition. It does not train or fine-tune models, and treats the model as a third-party capability that may change beneath it. It does not provision infrastructure: where a release needs configuration or migration scripts it generates them for a human or a Continuous Integration (CI) pipeline to execute, and never applies them itself. And it does not replace the project management tool a team already uses. It sits above that tool, reads and writes through it, and expects the team's existing board to remain the system of record for work items, which is why the framework's board guidance in Section 6.7 asks only that a team's own columns map onto framework phases rather than that they be renamed.

7.2 Architecture

Apex is a split full-stack application. The frontend is Next.js 15 with the App Router in TypeScript; the backend is Python 3.12 with FastAPI; the two communicate over a typed REST API. Server state on the frontend is owned by React Query and client state by Zustand, with credentials persisted only to session storage so that they clear when the tab closes. Backend routers are kept deliberately thin, with workflow logic held in a service layer and a schema for every request and response boundary. That separation is an engineering choice rather than an accident of growth: it keeps route handlers testable as thin adapters and workflow logic testable in isolation from Hypertext Transfer Protocol (HTTP) concerns, which is what made the test strategy in Section 7.6 possible at the sizes it reaches. Figures 7.1 and 7.2 show the arrangement.

Five properties of that architecture carry framework constructs rather than merely supporting them:

- **Spec-anchored storage.** All workflow state lives in Markdown context files plus a machine-readable story index, under a path keyed first by instance then by project, so a hosted account and a private deployment of the same tool stay separate tenants for the same user. Storage abstracts over local disk or cloud file storage on one code path, verified by the test suite pinning it to local disk rather than a stand-in. This is Spec-Anchored Continuity made concrete: files that outlive any session, accessed only through a single service, never a direct import, since that service is where per-request project isolation is enforced (Section 7.6).
- **A deliberate single-writer concurrency model.** The backend was originally pinned to one replica and one worker: the story index and workspace configuration live on shared file storage guarded only by a process-local lock, so a second replica would silently race on writes rather than crash, and a lost update looks like success. This was a documented constraint with a stated precondition for lifting it, subsequently built and enabled: an optional coordination layer promotes the locks to cross-replica equivalents when configured, falling back to byte-identical single-replica behaviour otherwise. Running in production with three replicas since June 2026, verified via live client connections.

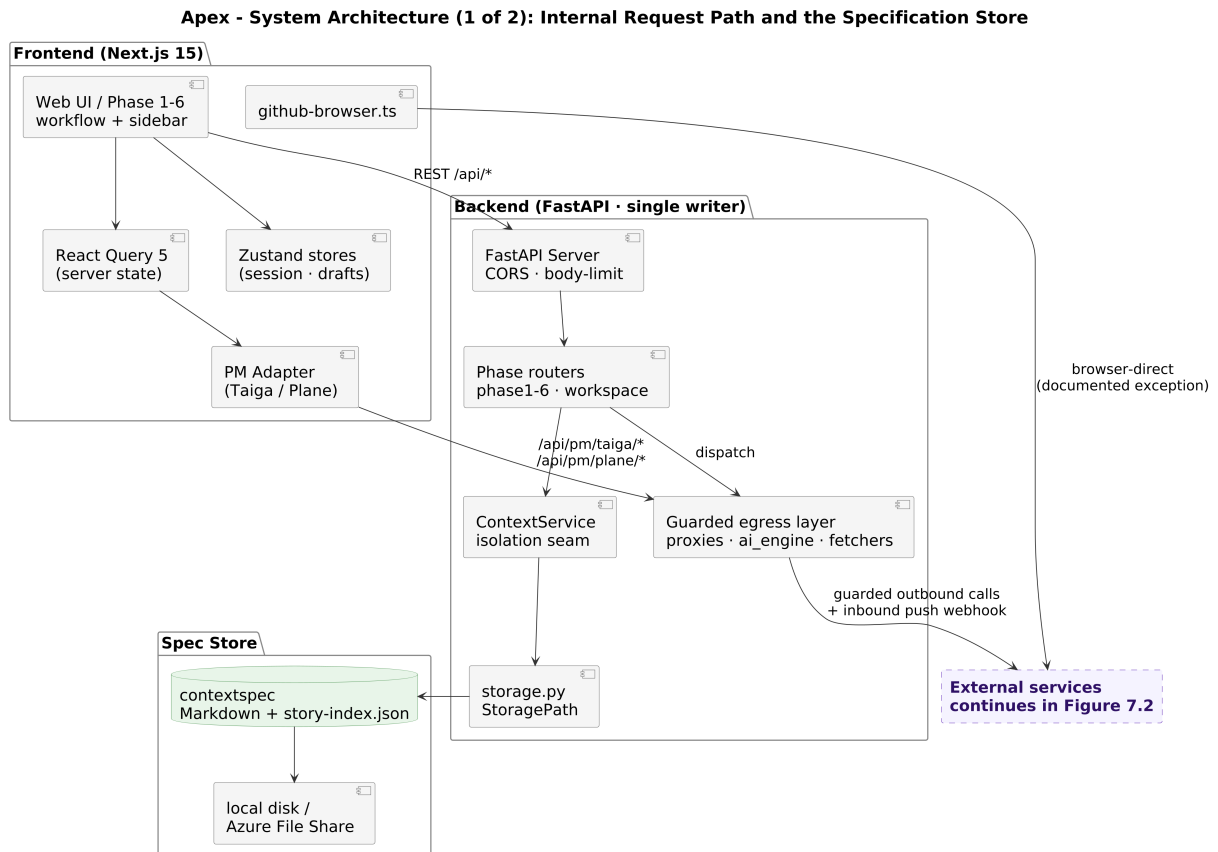


Figure 7.1: Apex's system architecture, part one of two: the frontend, the backend and the specification store. The figure is split across two pages for legibility; the point of it is the position of two boundaries rather than the inventory of components, and this half shows the first, where all workflow state is written to a per-tenant, per-project specification store rather than to an application database. The backend's guarded egress layer is drawn collapsed to a single component here and expanded in Figure 7.2; the dashed connector on the right marks where the two halves join.

- **Proxy-first external integration.** Every outbound call to a user-configurable host is proxied server-side, closing server-side request forgery: the proxy validates Hypertext Transfer Protocol Secure (HTTPS) and a non-private address before forwarding, and caps relayed body size. The identity anchor a request is validated against derives from the same validated host, which is what makes the storage isolation above trustworthy. One integration is a deliberate browser-direct exception whose credential never reaches the backend; two operations within it are not exceptions, needing a server-side filesystem and a server-held credential respectively.
- **The project management adapter.** Apex reads and writes a board the team already has rather than owning one, through a single adapter interface, built against Taiga first with Jira and Plane added behind the same interface (Plane specifically because it is the tool the partner organisation in Chapter 8 is moving to). Adding a second and third tool tested Section 6.2's claim that tool-independence is a property a mechanism must have, not one the product provides: it held at the level of operations (differing authentication shapes and identifier types both absorbed by generalising the identity anchor and minting a synthetic identifier) but not at the level of guarantees, since three of the second tool's Application Programming Interface (API) limitations, no page update or delete endpoint, no optimistic-concurrency field, no server-side label filter, are permanent, each only mitigated at application level rather than fixed.
- **Provider abstraction.** The model provider is detected from the model identifier and routed accordingly, with provider errors mapped to typed exceptions at a single boundary so no other code branches on vendor. The same tool-independence claim tested above for project management tools holds here too, at lower cost, since a provider offers no concurrency or identity guarantees for the abstraction to leak.

The translation from construct to code is not one to one for the first of those five. Section 6.6.2 mandates the two-layer split by lifetime and leaves the number and granularity of files unspecified; Apex resolves that latitude with a fixed, named taxonomy (`tech-stack.md`, `decisions.md` and `constraints.md` for the global layer, `project-concept.md`, `functional-spec.md`, `technical-spec.md`, `design-bundle.md` and `runtime-spec.md` for the local layer) maintained as cumulative, project-wide documents rather than one artefact locked per story. That is coarser than the framework specifies, and it is an implementation liberty rather than a requirement the framework imposes.

7.3 Operationalising the Framework

This section answers DRQ3, which asks how far the framework's governance constructs can be operationalised in a working system and which of them resist it. Table 7.1 states the mapping construct by construct, and every row points at something that exists in the codebase rather than something planned. Three outcomes are distinguished and the distinction is the substance of the answer: a construct realised as specified, a construct realised as a proxy that measures something adjacent to what the framework defines, and a construct that resisted implementation at the granularity the framework assumed and is only partially realised as a result. Figures 7.3 and 7.4 show how a practitioner actually walks the phases those constructs govern.

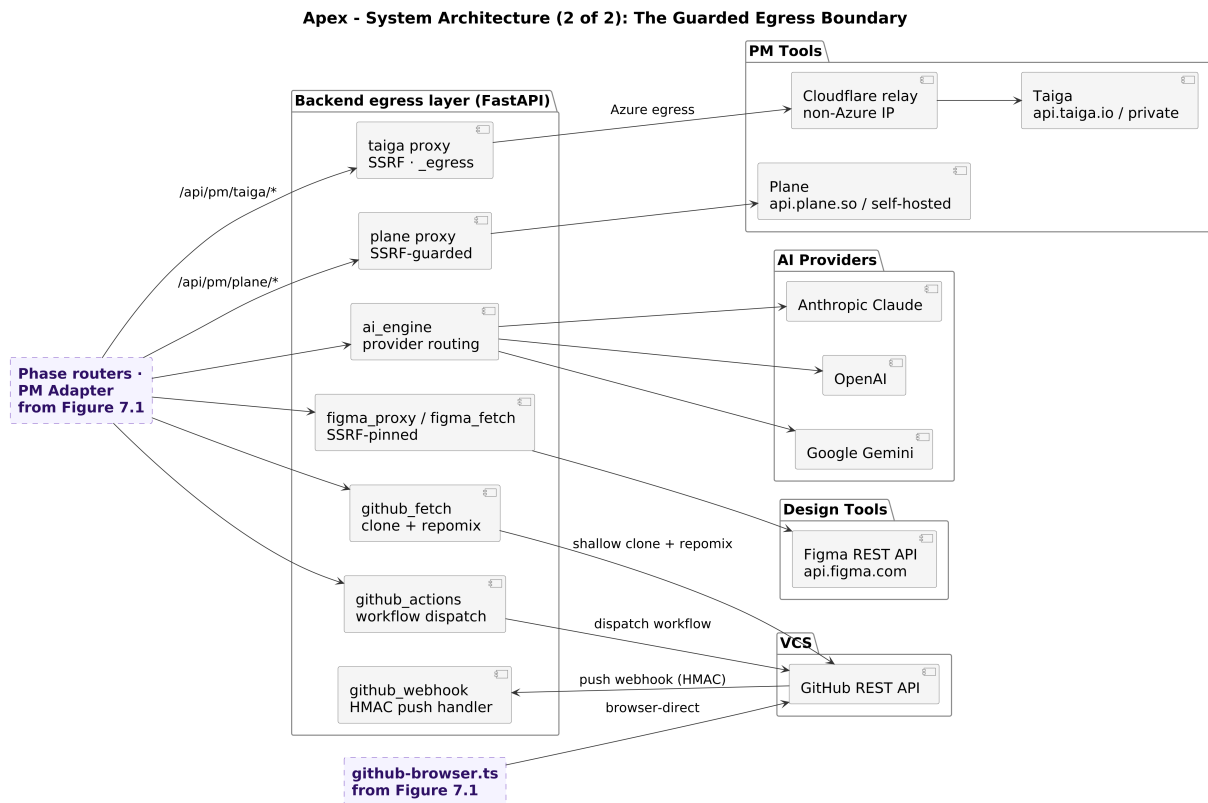


Figure 7.2: Apex's system architecture, part two of two, expanding the guarded egress layer of Figure 7.1 and the external services it reaches. This half shows the second boundary the figure exists to make, that every outbound call to a user-configurable host passes through a server-side guarded proxy. The dashed connectors carry over the callers from Figure 7.1; the browser-direct path to the version control host is the single documented exception, and the credential for it never reaches the backend.

Table 7.1: Framework constructs against the mechanisms implementing them in Apex, and the outcome of each

Construct	Implemented mechanism	Outcome
The six lifecycle phases	One workflow surface per phase, each gated on the preceding phase's exit condition rather than freely navigable.	Realised
Quality gates	A per-story phase-status flow advancing through gherkin_locked, design_locked, implementation, qa, qa_passed and deployed; middleware refuses phase access when the status does not permit it.	Realised
Spec-Anchored Continuity	Versioned Markdown context files plus a story index under a per-tenant, per-project path, standing in for the global and local layers as cumulative project-wide documents rather than files locked per story.	Realised
Stable specification identifiers	Epic, entity, screen and scenario identifiers minted at generation time and recorded in a specification index, so every later artefact resolves back to a requirement.	Realised
The Interactive Bridge	Phase 1 drafts scenarios in natural language for confirm, add, delete or modify instructions, and compiles to Gherkin only after approval.	Realised
The Consistency Factor	A generated test set produced before implementation and carried into the developer pack for each task.	Realised
Bolts and the Fix-Bolt	A per-task micro-cycle recording pack_ready, pushed and done transitions with timestamps, independent of the story's own phase status.	Realised
Bolt Cycle Time	Elapsed time from pack_ready to done per task, aggregated as median and 90th percentile. Initially computed from story-level gate transitions and presented under the same name, which measured queueing and review rather than execution; corrected to the task-level definition the framework specifies.	Realised, after a proxy was found and replaced
Context Traceability Rate	An aggregate over deployed stories, computed since June 2026; the earlier claim in this row that no aggregate was computed was itself in error, not a later change to the system. Strengthened in September 2026 to require the chain have no live break - no unresolved backward-trace flag and no unacknowledged conformance regression - rather than merely that every artefact file exists.	Realised, initially misreported here as unimplemented
AI Defect Escape Rate	A proxy added in September 2026: the proportion of deployed stories with a project-management issue confirmed as a bug, linked to the story, and dated after its deployment. A defect reported outside the project-management tool, or reported without being linked, stays invisible, and the system still cannot attribute a defect to generated logic specifically.	Realised as a proxy

Continued on next page

Construct	Implemented mechanism	Outcome
Spec-drift detection	Built for the whole specification and removed (Section 7.5); the technical specification was one project-wide file, so an edit could not be attributed and any change flagged every story at or past design lock - one real edit flagged 44 stories at once. Revived in September 2026 for the functional specification alone, where each scenario already carries a stable identifier naming its story, so an edit is attributed to exactly the stories whose scenario changed. The technical specification has no equivalent per-item ownership and was not revived.	Partially realised, resisted at the technical-specification granularity
Server-side reconciliation of generated output	Coverage claims and task dependency identifiers are re-checked against the parsed specification rather than trusted as reported, with unmatched claims dropped.	Realised, and added because it was needed

Four of those rows deserve emphasis because they are the ones that answer DRQ3 rather than merely illustrating it.

The **Bolt Cycle Time** row is the clearest instance of a proxy being mistaken for the construct. The framework defines the metric at task level precisely because a story-level measurement conflates execution with the queueing, review and gate transitions surrounding it. The first implementation measured story-level gate transitions and displayed the result under the framework's own name for the task-level quantity, which is exactly the substitution the framework warns against elsewhere. It was corrected, but the fact that it happened at all is evidence for a general point: a construct can be implemented, displayed and believed while measuring something else, and only comparing the implementation against the written definition catches it.

The **Context Traceability Rate** row corrects this chapter's own earlier account rather than reporting a later change to the system: the metric had already been computed as an aggregate over deployed stories since June 2026, so the original claim that no aggregate was computed was simply wrong. The error makes the same point Section 7.6 makes about an unverified self-report: a claim about the system's state is not evidence of that state until it is checked against the code. The strengthening the row records closes a separate gap, where a story could show a complete chain while Phase 6 had already flagged it broken.

AI Defect Escape Rate is realised only as a proxy. The system still cannot observe a production defect directly or attribute one to generated logic, the limitation Section 7.7 states and which remains true. The project-management tool already in use carries a usable signal, added in September 2026, that the row describes: reported, linked, post-deployment defects rather than defects in general. That the construct resisted the system's strongest form of observation and was still operationalised at a weaker one worth having is the distinction Section 7.3's three-outcome taxonomy exists to keep visible, applied here to itself.

Spec-drift detection is the construct that resisted implementation most instructively, and remains only partly resisted. The framework assumes the specification is scoped per story, which is what makes an amendment attributable; the technical specification did not meet that assumption, and the cascade the row records is the consequence. Removing it was the right call, but the finding is not that the feature was hard: it is that a framework construct silently depends on a granularity assumption the framework never states explicitly, and the instantiation is what exposed the omission. That is formative

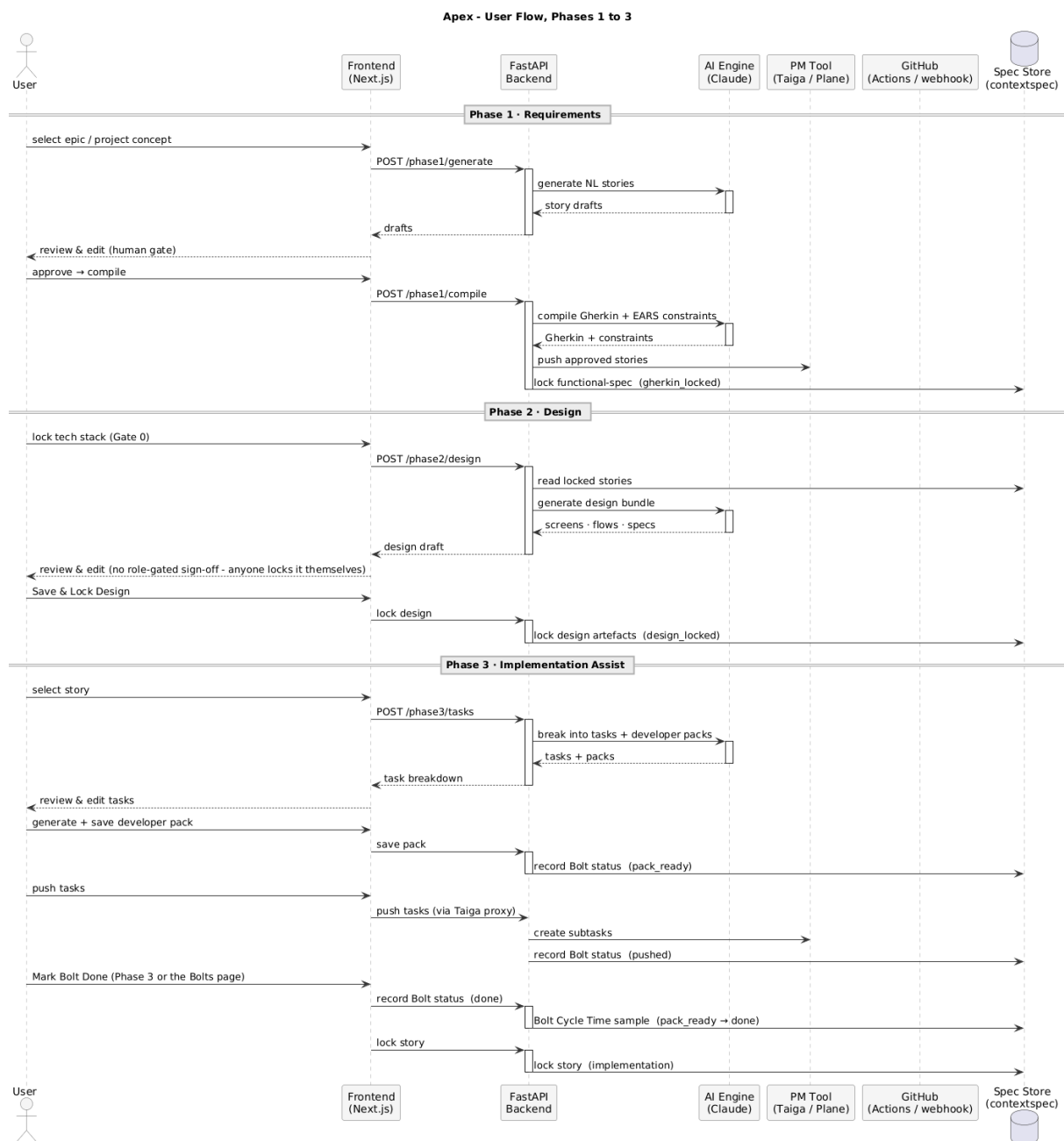


Figure 7.3: A representative walk through Apex, Phases 1 to 3. The figure is split across two pages for legibility; what it is meant to show is that every phase terminates in a write to the specification store and a human acceptance, so no phase can be exited by generation alone.

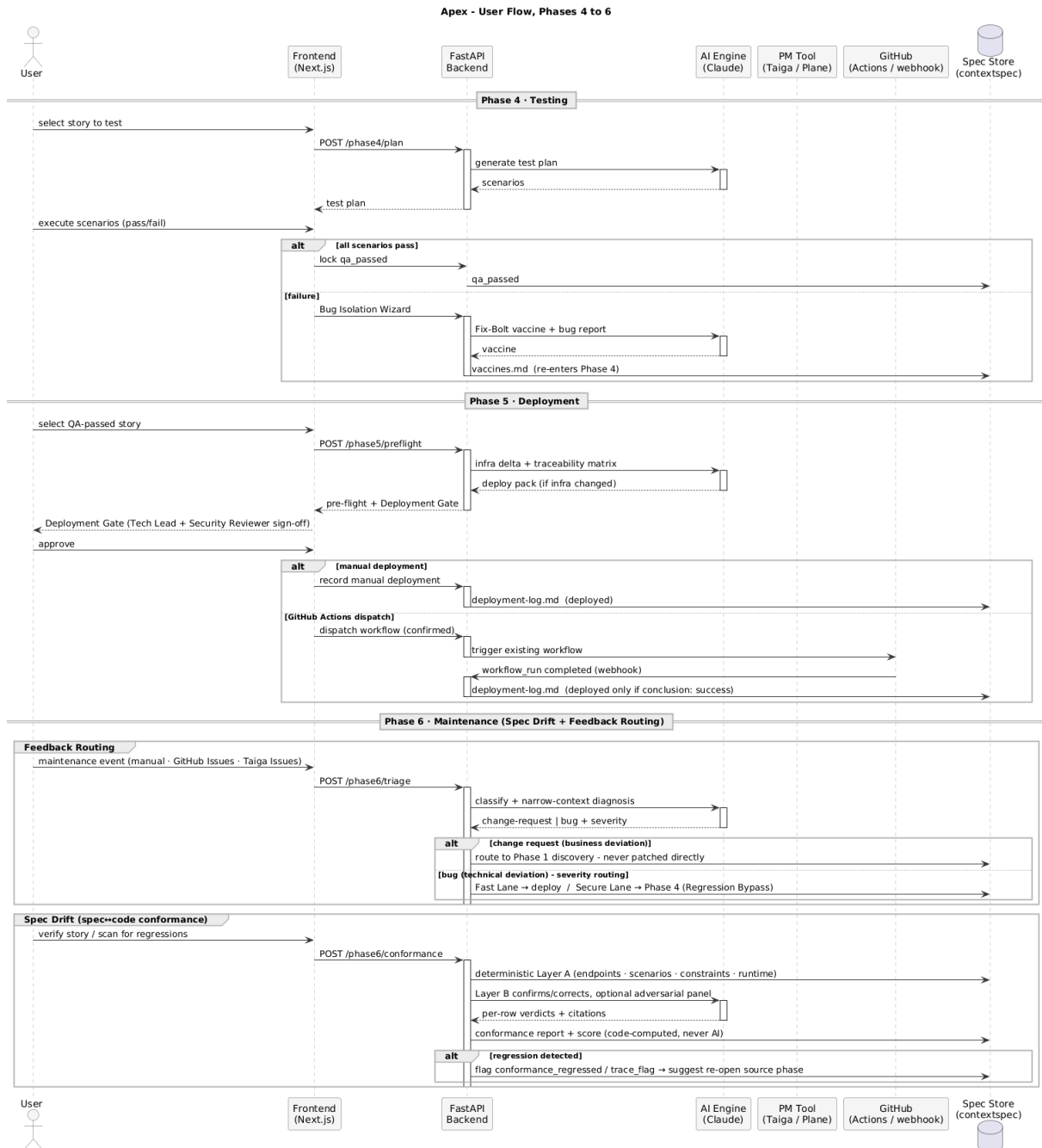


Figure 7.4: The same walk, Phases 4 to 6, continuing Figure 7.3. Note the loop-backs out of Testing and out of Maintenance: neither returns to an undefined state, and each re-enters a named phase whose gate then applies in full.

evaluation feeding back into the method, and it is recorded in Section 7.5 as such. The functional specification, unlike the technical specification, already meets that assumption, since each scenario is written under its own story's heading and carries a stable identifier naming that story, which is what made the September 2026 revival possible there. The technical specification was not revived the same way, since no equivalent per-item ownership exists anywhere in the system, and inventing one here would trade the original mistake for its mirror image - false precision in place of false breadth.

7.4 The Governance Analytics Surface

Table 7.1 records which governance constructs are realised. It does not say how the numbers standing behind them are produced, and that question decides how much weight Chapter 9 can place on any figure it later reports. Apex computes its governance metrics in one backend service that reads the story index and the context files the workflow has already written, and returns the whole set on demand for the project in scope; at the scale these projects reach, tens of stories, the computation is cheap enough to need no caching and therefore no invalidation rule that could go stale. Two properties of that arrangement matter more than the individual formulas. Nothing on the surface is self-reported: no figure is a model's account of its own work and none is typed in by a practitioner, which is the principle Section 7.6 describes being violated and then repaired in the coverage badge. And the inputs are a byproduct of running the phases rather than the output of a separate instrumentation step, so a project that is simply used produces its own measurement, and one that is abandoned stops producing it rather than continuing to report a figure that is no longer true of anything. Six figures are displayed, and each is described below by what it actually computes.

Context Traceability Rate is the metric whose implementation is least obvious from its definition, and it is treated first for that reason. Section 6.6.4 defines it as the proportion of deployed work whose context chain is complete, the deployed artefact resolving back to a locked technical specification, to the functional specification it derived from, and to the Unit of Work that motivated it. Apex evaluates that as a predicate, story by story, over every story at deployed status, and reports the proportion satisfying it. The three legs are not equally hard. The artefact-to-Unit-of-Work leg is resolved by construction, since every story-index entry is keyed by the project management tool's own story identifier and cannot exist without one. The functional-specification leg is resolved by per-story attribution that already exists, because scenarios are written under their own story's heading and carry identifiers naming that story. The third leg, the artefact resolving to a locked technical specification, is the one that can break after the artefacts first exist, and it is therefore the only one that is not answered by checking for files. A story satisfies it when its functional, test and infrastructure artefacts are all present, when its identifier is actually found in the deployment log rather than merely claimed as deployed in the index, and when neither of two break detectors is set: an unresolved backward-trace flag, meaning a downstream failure has been pointed at an earlier phase and not yet answered, and an unacknowledged conformance regression, meaning code has since drifted from the specification it was locked against. Either flag zeroes the story even with every artefact file present, because presence is not resolution. A completed verification record then gates the remainder, read from the index where it has been mirrored and from the file itself for indexes written before that mirror existed.

What the metric declines to do is part of its definition. It does not attempt to attribute a deployed

story back to particular endpoints or entities within the technical specification, because that ownership does not exist anywhere in the system, for exactly the reason Section 7.3 gives for not reviving spec-drift detection there. Claiming to resolve it would substitute fabricated precision for the fabricated breadth already rejected, which is a worse trade rather than a better one. The presence of the technical specification together with the two break detectors is what is honestly provable without it, and the rate should be read as measuring that and no more.

Bolt Cycle Time is computed per task rather than per story. Each story carries its own map of Bolts, and for every task the elapsed time is taken from the earliest timestamp at which it became ready to work to the earliest at which it was marked done; the aggregate over all tasks is reported as a median and a ninetieth percentile in hours. The story-level series that was once displayed under this name still exists and is still useful, but it is shown separately and labelled as cycle time per gate transition, so that the quantity conflating execution with queueing and review can no longer be read as the quantity the framework defines. That the two now sit side by side under different names is the residue of the error described above, and keeping both visible is cheaper than trusting that the mistake will not be repeated.

Spec Conformance Rate is not one of the framework's three metrics but an instrument Apex adds from its own Phase 6 conformance check, which scores generated code against the specification it was locked to on a scale of 0 to 100. The rate is the mean of those scores across every story at implementation status or beyond that has a saved conformance report. A story eligible for the check but never checked is excluded from the mean rather than counted as zero, which fixes what the figure means: it answers how sound the checks that were actually run came out, not whether the checks were run. That second question is a real one and is displayed alongside as the count of eligible stories against the count checked, because the mean alone can be high on a project where conformance has barely been exercised, and a reader given only the mean would have no way to see it.

The two defect figures are best read as a pair, because they observe opposite sides of the deployment boundary. **Fix-Bolts** counts the corrective micro-cycles raised across the project, reported as a total, as the number of distinct stories carrying at least one, and as a mean per story; these are defects caught by quality assurance before a story shipped, so the figure is a pre-deployment signal and rises when the gate is working rather than when it is failing. **AI Defect Escape Rate**, whose status as a proxy is argued above and whose limits Section 7.7 states, is the post-deployment counterpart: the proportion of deployed stories carrying at least one maintenance item in the project management tool that is classified as a confirmed bug, linked to that story, and dated after that story's own deployment. The classification filter is load-bearing rather than defensive. One supported tool has no issue resource distinct from ordinary work items, so an unfiltered import would count planned work as escaped defects and inflate the rate on precisely the projects that had done the most subsequent work.

Stories Tracked is the plainest of the six, the count of entries in the story index with the deployed subset given beneath it, and it is displayed because the other five are proportions or aggregates that say little until their denominator is visible. A traceability rate over a handful of deployed stories and the same rate over several dozen are not the same claim, and putting the count on the same surface is what stops them being read as one.

A structural distinction runs through the six. Bolt Cycle Time, Context Traceability Rate, Spec Conformance Rate and Stories Tracked are computed entirely from state Apex itself writes while the phases are being worked, so their inputs cannot be absent unless the work itself was absent. The two defect

figures depend on issue data owned by the connected project management tool, which Apex reads but does not control and cannot complete. That is why their caveats are sharper, and it is the same boundary Section 7.7 draws between what passes through the system and what does not, expressed here as a property of the instrument rather than of the framework. The surface is an instrument and nothing in this chapter reports readings from it; the figures it produces on a real project are Chapter 9's to state, under the conditions stated there.

7.5 Design History and Practitioner Feedback

Feedback gathered while building Apex changed the framework, not only the tool. Recording that path is what makes the DSRM iteration explicit rather than implied, and it is reported as observation, change and rationale rather than as a list of releases. Changes made and later reverted are included, because a reverted change is evidence about the design.

Two changes were framework-level and are the clearest evidence that evaluation fed back into the method. The first was the **removal of a role**: an earlier DevOps Alliance, external to the team and holding deployment approval and infrastructure-security review, was removed once feedback showed a role defined only by what it supervises goes unstaffed, its responsibilities reassigned to whoever performs the deployment (Section 6.5 argues the rationale in full). What matters here is that the change originated in observation of use rather than in analysis.

The second was the **demotion of a mandatory step**. Phase 2 originally required a separate sign-off from named design and technical leads before design could be locked. The same observation applied: role-gating a gate presumes a staffed organisation, and on a solo project it either blocks the work or is satisfied by the same person twice, which teaches the practitioner that gates are ceremonial. The step was removed and the gate redefined against recorded evidence rather than against a second signature, which is the form it takes in Table 6.5. The framework's position in Section 6.5 on where a gate's value lies is a direct consequence of this iteration.

Three tool-level changes are worth recording for the same reason. A **design-conflict detector** was built, then found to flag conflicts that were not conflicts, since two stories in the same epic legitimately share files; it was narrowed to cross-epic file overlap and endpoint collision, and later removed entirely. **Spec-drift flagging** was built, cascaded as described in Section 7.3, and was removed while its underlying audit trail was kept. An **authentication flow** for one design integration was built, then reverted for requiring an operator-registered application and a backend-held client secret, incompatible with the tool's zero-configuration setup.

The nature of that feedback bears on how much weight the design history can carry. It came from five practitioners consulted individually during construction, one of whom was consulted a second time after the framework had been substantially revised in response to the first round. The sessions were informal: no interview guide, no fixed question set, no recording, and no demographic data collected. Feedback was captured in the author's own contemporaneous notes as discrete items, each associated with the person who raised it, which is how the changes above are traceable. Participants are not identified and no table of dates or profiles is given, because the sessions were not conducted under any expectation of being reported that way. The record is therefore a participant's notes rather than an independent transcript, and it carries the selection that implies: an item the author did not think worth writing down at

the time is not in it.

The limit that follows is a real one, and it belongs here rather than in Chapter 9. Because each item is attributable, *what* was raised and *what* changed in response are both verifiable against the codebase and the framework document. What cannot be recovered is how strongly any item was held, whether an observation was one person's reaction or a view several would have shared had they been asked, and what the practitioners who did not raise a given point thought of it. The design history is therefore evidence that the method was revised in response to use, which is the DSRM claim it is offered for, and it is not evidence about the distribution or strength of practitioner opinion. Three of the five practitioners consulted here later also contributed, separately and with their own consent for that use, to the semi-structured interviews reported in Section 9.3.4; the two sections draw different material from some of the same individuals, and an item appears in only one place. What the structured interviews add is exactly what this section cannot supply: a written guide, a stated (if under-target) participant profile, and a thematic analysis method applied consistently rather than notes kept only where a design change followed.

The second round is worth isolating, because it produced the sharpest challenge the framework received and the one that changed the shape of this dissertation rather than the shape of the tool. Its substance was that the work was being presented as a product and would therefore be judged as one, against the question of what it offers that a general-purpose coding agent does not; that the scientific questions, why this framework rather than another and how it maps to a given organisation, were being left implicit; and that the correspondence to the published models it draws on should be stated step by step rather than asserted. The response is distributed across three chapters rather than confined to a change log: Section 7.1 answers the coding-agent question directly, Section 6.9 engages four named alternatives rather than defending the design in the abstract, Section 6.4 argues the two points of divergence from its closest published precedent instead of claiming a one-to-one mapping, and Chapter 8 adopts the reframing that the instantiation is itself a demonstration of the method. That a single round of feedback is traceable into four locations of the final document is the clearest evidence available that this was formative evaluation rather than consultation after the fact, and it is reported here in full rather than split with Section 9.3.4 because its effect was on the framework's construction rather than on the distribution of practitioner opinion.

A sixth, later consultation is not part of the five above. An Agile Coach was given a live presentation of the framework and a demonstration of Apex; the reaction, and the two open gaps it named in the tool's interface and in the coach's own organisation's AI governance, are reported in full in Section 9.3.4 as this study's Agile-expert interview. It shares the limits stated for the five practitioners above, and is separated from them because it took place later, checked the demonstration argument rather than shaping the framework during construction, and produced no traceable change to either the framework or the tool.

7.6 Quality Assurance of the Implementation

Testing is layered by what each layer can catch: at the time of writing the backend carries roughly 1,638 tests, the frontend roughly 318 unit tests, and there are 24 end-to-end test cases across 12 specifications, a snapshot of a codebase that continues to change.

Testing also surfaced defects with methodological significance. A section-parsing routine truncated a live project's technical specification part-way through because it anchored on headings the model itself also emits. An automation step once marked a story's quality gate as passed with no verification behind it, and the deployment step trusted that result rather than checking it. And the identity anchor determining which tenant a request belongs to, along with an integrated tool's listing endpoint, each leaked across project boundaries before being corrected. A coverage badge reported the model's own claim about which scenarios it had covered, at face value; it is now reconciled server-side against the parsed specification, and a claimed scenario that does not match a real one is dropped rather than displayed.

7.7 Limitations of the Implementation

The limitations of Apex as a system must be separated from the limitations of the framework as a methodology, because conflating them would let a defect in one be read as a refutation of the other.

Two are limitations of the system. The single-writer fallback path, which most deployments would run, is a real ceiling rather than a theoretical one (Section 7.2). And Apex observes only what passes through it: it can see a story's specification chain, and, once a defect is reported through the project-management tool and linked to a story, a proxy signal that a deployment shipped a fault - but it cannot observe a defect never reported there, or attribute one to generated logic specifically.

Two are limitations that belong to the framework and were exposed by trying to build it, and both remain open even after the September 2026 work reported above. **AI Defect Escape Rate, at the observation point the framework's own definition presumes, is not instantiable by this class of system:** nothing in the framework says who owns production telemetry for generated logic specifically, and a tool sitting above a project-management board has no access to it regardless of who does; what is instantiable is the weaker proxy described in Section 7.3. **Spec-drift detection presumes a granularity the framework does not state,** namely a per-story rather than a per-project specification, and the instantiation revealed the omission by failing on it; it now holds for the functional specification and remains open for the technical one. Both are carried into Chapter 9 as findings rather than as defects to be excused.

The two proxies differ in kind, and Table 7.1 names both as proxies rather than presenting either as the construct itself: Bolt Cycle Time was one briefly and by accident, while AI Defect Escape Rate is one by design from the outset. A substitution left unnamed is indistinguishable from the thing it substitutes for.

8

Demonstration

Contents

8.1 Demonstration Design	49
8.2 Context	52
8.3 Application of the Framework	53
8.4 Observations	57
8.5 Summary	61

This chapter reports the Demonstration activity of the DSRM: the application of the framework, through its reference implementation, to a real software development project. The purpose of the demonstration is to show that the artefact can be used to address the research problem in a genuine setting, and to generate the evidence on which the evaluation in Chapter 9 rests.

8.1 Demonstration Design

This section is written before the demonstration is performed. Committing in advance to what will be observed, and against which criteria, is what separates a demonstration from an anecdote, and it is the only protection against the reasonable suspicion that the observations reported later were selected because they suited the artefact. Where a statement below turns out to be wrong once the work is done, it is corrected in Section 8.4 and the correction is marked, rather than the commitment being quietly revised.

Two settings, a third attempted, and what each can and cannot show

The framework is demonstrated in two settings, because they fail in different directions and neither carries the claim alone. A third was planned, did not run, and is recorded here rather than quietly dropped.

What the instantiation is asked to prove (Section 7.5) is not that it is a better way to obtain generated code, but that the governance constructs, phase gates enforced against specification content, Spec-Anchored Continuity, and classified loop-back, are implementable at all.

The first is **Apex itself**, and the claim needs stating precisely because it is easy to overstate. It is not that Apex's own development followed the framework's phases; it is that Apex is the framework rendered as executable software, so its construction demonstrates that the constructs are realisable rather than merely describable. In *DSRM* terms the instantiation is itself a solved instance of the problem: a working system in which every phase has a gate, every artefact resolves to a requirement, and no phase can be exited by generation alone answers the framework's own question, how AI participation across the lifecycle can be bounded, validated and traced, in a form that can be inspected rather than argued.

What that establishes is bounded: it is an existence proof rather than a trial. It shows the constructs can exist; it says nothing about whether they help anyone, whether a team would tolerate them, or whether they survive contact with work the designer did not anticipate. It is also reported retrospectively, since the system already exists, so it is explicitly not pre-registered and nothing in the record below applies to it. Its detail is the construct-by-construct mapping in Section 7.3, the demonstration's first result.

The second is **Outfolio**, a portfolio and project-documentation platform for developers, separate from Apex and from this dissertation. It is the author's own project, so the researcher-role threat below applies to it in full, but it differs from the first setting in the way that matters: the framework is fixed before the work begins, and the software being built has nothing to do with software process, so the framework is applied to a domain it was not derived from. It is built as a complete, deployed product rather than a single exercised unit, so that the framework is seen carrying real scope end to end rather than a single narrow slice of it; this is the setting in which the full lifecycle, Phases 1 to 6, is exercised.

A third setting, a **partner organisation** already using Apex and in the process of migrating its project management tool, was identified and would have been the only setting in which the framework is operated by people who did not design it, and so the only one capable of showing that the method is transferable rather than merely usable by its author; it would also have exercised the tool-independence claim in Section 6.2 in the one way that counts, since the migration would have forced the framework's phases and gates to survive a change of adapter underneath them. It was pursued and did not materialise: repeated outreach across several weeks received no response, and the setting was abandoned rather than held open past the point where a deployed increment could still be produced and written up. What is lost is not a data point but a distinct kind of evidence, since neither remaining setting can show the framework surviving contact with people who did not build it; Section 9.6 draws on the practitioner and Agile-expert interviews for that question instead, and the two are not equivalent, an interview establishes a considered opinion, not an observed application.

Unit of analysis, scope and duration

The unit of analysis is **one Unit of Work carried from Strategic Alignment through to a deployed increment**, not an individual task. That level is chosen because it is the smallest unit at which every gate in Table 6.5 is exercised at least once, and because the framework's own claims are made about the passage of work through phases rather than about the output of any single phase; where a setting's scope exceeds one unit, as Outfolio's does, the unit remains the level at which observations below are organised.

Outfolio is scoped as a small but complete product rather than a single unit run in isolation, so that more than one unit is observed and the framework's behaviour across units, not only within one, is part of the record; the epics and stories fixed for it are stated in Section 8.2. The instantiation setting has no unit count, for the reason given above. The demonstration runs until Outfolio's units reach either a deployed increment or an abandonment recorded with its reason, whichever comes first; an abandoned unit is reported, since a framework that cannot carry work to completion in a real setting is a finding.

The fieldwork this section requires is complete: no unit was abandoned, and Sections 8.3 and 8.4 report what the demonstration log establishes.

What is recorded, decided in advance

At every gate crossed, in every setting, the following is recorded: the artefact presented as evidence, the decision taken, the hat that took it, and, where the gate failed, the named phase the work was routed back to and what changed before it returned. Loop-backs are recorded as first-class events rather than as exceptions, for the reason stated in Section 6.4: a demonstration in which nothing was rejected and nothing looped back is evidence that the framework was not exercised, not evidence that it works.

Alongside the gate record, three further classes of observation are pre-committed. First, the governance metrics the framework defines in Section 6.6.4, to the extent the instantiation computes them, with any it does not compute, and any computed only as the proxies Table 7.1 names, reported as such rather than silently presented as the construct. Second, every point at which generated output was modified or rejected by a human, together with what was wrong with it, since the framework's central premise is that this happens and is caught. Third, every point at which the framework was found awkward, over-specified, or was bypassed in practice; these are recorded as they occur rather than reconstructed afterwards, and they are the most valuable input to Chapter 10.

Two commitments about reporting are made here so that they cannot be reconsidered later. Observations unfavourable to the artefact are reported with the same prominence as favourable ones. And the artefacts themselves, generated scenarios, specifications, task breakdowns and test plans, are reproduced rather than described, so that a reader can judge their quality instead of accepting a characterisation of it.

The researcher's role, and the threat it creates

The author designed the framework, built its instantiation, and is the participant in Outfolio. This is the most serious threat to the validity of this chapter and it is named here rather than deferred to Section 9.5.

The threat is not primarily that the author would misreport what happened. It is that a designer applying his own method is the person least likely to notice where it is awkward, because he already knows what it is for, supplies missing rationale from memory without registering that he is doing so, and reads an artefact as adequate against an intention rather than against what is written. The instantiation setting carries a related but distinct problem: because the framework and the system were designed together, a construct that proved inconvenient could be revised in the method rather than recorded as a difficulty, and Section 7.5 reports two occasions on which exactly that happened.

Two things mitigate it and neither removes it. Pre-registering the record above, for the reason given at the head of this section, is what makes a later omission visible. And the gate record is defined against artefacts that exist independently of the author's account of them, so a claim in this chapter can be checked against the specification store rather than taken on trust. The one mitigation that would have been structural rather than merely procedural, a setting in which someone other than the author operated the framework, is exactly what the abandoned partner-organisation setting above would have supplied; without it, this threat is carried by procedure alone.

What remains unmitigated is stated plainly: for both settings, this chapter reports what a designer observed while using his own design, and no arrangement within this study makes that equivalent to independent application.

8.2 Context

Outfolio originates from a concrete problem reported by a colleague, a developer who builds professionally on OutSystems, a low-code platform. Having built substantial projects on it, he found himself unable to show that work to anyone outside the platform: OutSystems' own project export carries the application's logic and data model but preserves no visual record of what the application actually looks or behaves like, so a portfolio assembled from that export shows structure without ever showing a result. The problem generalises beyond OutSystems specifically to any developer whose most substantial work lives inside a platform, low-code or otherwise, that does not treat "what this looked like to a user" as an artefact worth exporting. Outfolio is a portfolio and project-documentation platform built to close exactly that gap: a developer documents a project as a case study, screenshots, architecture notes, the problem it solved and the outcome, independently of whatever platform the underlying project was built in or whether that platform still exists.

The scope committed in advance, recorded in the project's own agent-instruction file, was six epics and roughly eighteen stories, deliberately small so that a full lifecycle could be carried through to a deployed increment rather than a large backlog left partly specified. It did not stay there. Two finishing epics, a home dashboard and a visual design system, were added on 16 September 2026 by an explicit, logged human decision, and a ninth, project media and file attachments, on 20 September through a Phase 6 loop-back into Phases 1 and 2 after two post-deployment signals were classified as change requests rather than defects. The delivered product is nine epics and fifty stories against a plan of six and about eighteen, reached over nine calendar days and 123 commits. The growth is not in itself evidence for the framework; that every increment of it is attributable to a dated decision carrying a recorded classification is what Section 8.3 reports.

Every hat the framework names is worn by the same person. The author is PO, Tech Lead, Design

Lead, Developer and QA across all nine epics, switching between them at gates rather than handing work over, and he is simultaneously the maintainer of the instantiation being exercised, a hat the framework does not name and that only this demonstration creates. This is precisely the case the hats-not-people design in Section 6.5 exists to cover, and it is also the honest limit of the setting: a hat worn solo is a self-check, and the framework's claim is that what survives the reduction is the gate's evidential requirement rather than the identity of whoever holds it.

Outfolio is greenfield, so its baseline is a working practice rather than an inherited system. The stack was fixed at the framework's design gate on the first day and not renegotiated afterwards: Next.js 14 with the App Router and React 18 in TypeScript, business logic held in Express-style handlers mounted under a single catch-all route, Prisma over PostgreSQL, Supabase supplying the database host, credential storage and private object storage, NextAuth holding browser sessions, and deployment to Vercel through GitHub Actions. Testing is Jest with React Testing Library below and Playwright above. None of it predates the demonstration: the 62 unit and component test files and three browser specifications the repository now holds were written inside Phases 3 and 4, and at the outset there were neither tests nor a database separate from production.

What that baseline lacked maps onto four of the consequences Chapter 4 reports in Table 4.5, and stating them concretely rather than restating them abstractly is what makes it possible to judge later what the framework changed.

Validation of generated code was inconsistent because nothing made it consistent: with one developer and one coding agent, review meant the person who wrote the prompt rereading, at the moment of generation, output he had just asked for and already expected to be correct. Requirements were under upstream pressure for the reason accelerated prototyping predicts, since when a working screen costs minutes to generate, specifying one costs more than producing one, and scope grows by accretion rather than by decision. Testing was complex before a single test existed, because the stack offers no one level at which it can be exercised: an App Router page, an Express-mounted handler, a Prisma migration and a hosted Supabase project each fail differently, and a browser-level check needs a database that is not the live one. And responsibility over generated artefacts was undefined rather than contested: with no second party, nothing distinguished a decision the author took from one the agent took on his behalf, and the commit history, the only durable record the baseline produced, does not record the difference.

8.3 Application of the Framework

Outfolio was carried from Strategic Alignment to a deployed increment over nine calendar days, 14 to 22 September 2026, under a demonstration log of 422 dated entries recording, at each gate, the artefact presented, the decision taken, the hat that took it and the phase any failure was routed back to. Eight epics and thirty-six stories were locked, built, tested and deployed in the first pass; a further twelve stories, eight of them a new epic, and four individual stories after those, entered later through Maintenance loop-backs. Every claim below is cited by date and story identifier so that it can be checked against that log rather than accepted as a characterisation of it. What follows walks the six phases of Section 6.4 in turn, and holds the loop-backs back for the separate passage that closes the section: treating them as exceptions inside a phase walk would misstate how much of the work actually went through them.

Discovery and Requirements

The Interactive Bridge ran as specified. The model drafted fractional stories in natural language per epic, the human confirmed, modified or deleted them, and only the approved set was compiled into Gherkin carrying stable scenario identifiers; the steering was explicit rather than passive, entered as free text in a dedicated field before generation. The Discovery Gate was accepted at 16:24 on 14 September for all six initial epics at once, locking the functional specification project-wide. Two unfavourable observations belong to the gate itself. The hat metadata for that acceptance was, by the log's own note, supplied after the gate statement had been written, so the accountable hat trailed the decision instead of preceding it; and when two further epics were added mid-stream on 16 September, Home Dashboard was drafted at 22:58 and gherkin-locked at 23:06, eight minutes later. Neither is a violation of the gate as Table 6.5 defines it, since the required evidence existed in both cases, but a gate whose owner is named retrospectively and whose evidence is examined in eight minutes is being satisfied at the lower bound of what the control was written to mean.

Design, Prototyping and Architecture

The Design Gate was crossed at 17:09 on 14 September, forty-five minutes after the Discovery Gate, producing the anchored technical specification and design bundle for the whole initial scope. This is the phase where the solo setting bears most directly on the control: Table 6.5 assigns the gate to the Design Lead and Tech Lead hats jointly, and across 422 entries that joint hat is recorded exactly three times, all at Phase 2. The framework's "hats, not people" design in Section 6.5 anticipates one person wearing both, but it does not make the second reading independent of the first, and nothing in the demonstration establishes that it was. What the phase did demonstrate is the amendment discipline: the specification was not frozen but re-opened and re-versioned, once as a recorded major amendment on 20 September at 23:01 when the Maintenance loop-back described below added a new epic, and again with a major design-version bump for story 9564046 on 22 September.

Implementation

Implementation ran from 15 to 18 September for the original scope, each story decomposed with the model into tasks, each task executed as a Bolt against a developer pack carrying the Consistency Factor test set. The Authentication and Session Management epic is representative: four stories implemented between 13:51 and 14:41 on 15 September, the epic concluded at 14:43. Two recurring frictions belong to the record. The first is that the model repeatedly had to be warned against duplicating work already in the repository, a near-identical caution recurring across stories 9543460, 9543480 and 9543481, which is the cost of a cumulative project-wide specification rather than one locked per story, as Section 7.3 already concedes. The second is more serious. On 21 September at 20:35, while implementing a seeding task, the agent executed a destructive reset script against the live production database without re-checking which environment the connection string pointed at, wiping all six tables, its root cause discussed in full in Section 8.4. Work stopped, the human was informed immediately, and an isolated database project was provisioned before implementation resumed.

Testing

All thirty-six stories of the original scope reached a passing story-level verdict between 18 and 19 September, tested against the locked Gherkin rather than the implementation. The gate was not a formality: story 9552560 failed on one scenario while its two siblings passed, with the failure attributable to component code the application had already moved past (Figure 8.1). Two username-casing defects were caught the same way on the registration and login stories. The phase also produced the demonstration's clearest evidence for why Section 6.4 decouples Testing at all: a session-gating defect was, in the log's words, "invisible to the entire existing automated test suite by construction", because every affected component's tests mocked the exact library whose missing backend was the defect. The manual edge-case probing the QA Validation Playbook mandates, and not the generated scripts, is what surfaced it.

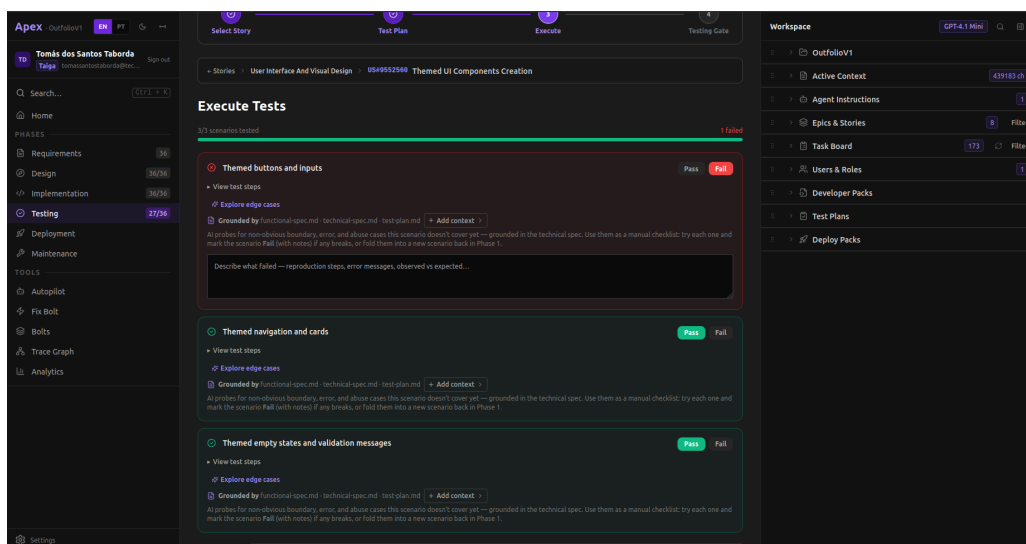


Figure 8.1: Phase 4 Testing for story US#9552560, Themed UI Components Creation: the "Themed buttons and inputs" scenario marked Fail alongside its two passing siblings, with Apex's AI-generated edge-case probes shown underneath as a manual checklist.

Deployment and Maintenance

The first production deployment was story 9543457 on 19 September at 20:54, and Phase 5 closed for all eight epics at 12:41 on 20 September. The gate presents the infrastructure-delta verdict, the deployment pack and the traceability matrix in one view, with the human sign-off unchecked until the operator checks it (Figure 8.2). Two findings run against the artefact. The infrastructure-delta answer was wrong twice in opposite directions and overridden both times (Section 8.4), which is the gate working, but an advisory answer that is wrong in both directions supplies no information. And the gate's own required evidence includes a security review, not invoked as a distinct step during the Fix-Bolt deployments of 18 September, a control satisfied nominally by the same hat that authorised the push.

Maintenance opened on 20 September with ten items accumulated from live use. Each was classified before any code was opened, and the classification determined the route rather than being recorded after

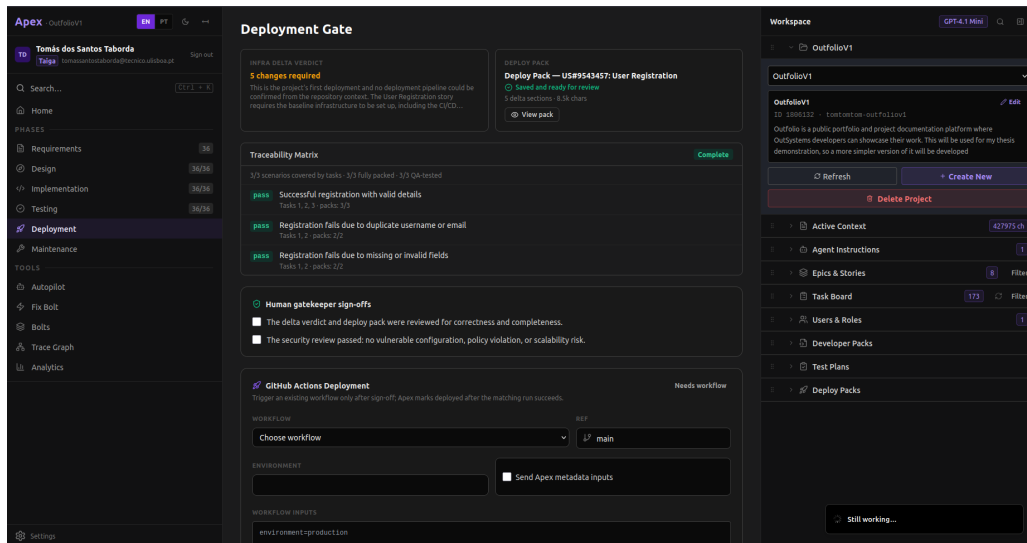


Figure 8.2: Phase 5 Deployment Gate for story US#9543457, User Registration, the project's first production deployment: the infrastructure-delta verdict, the deploy pack, and a Traceability Matrix showing all three scenarios fully covered by tasks, packed and QA-tested, beside the human gatekeeper sign-offs still unchecked.

the fact (Figure 8.3). Nine items were resolved by 02:20 on 22 September, and the tenth, dark-mode support, ran its own full cycle afterwards. Restraint is part of the record too: a scope discrepancy on story 9543480 was deliberately resolved as vacuously satisfied rather than opened as new Discovery work (Section 8.4). The handoff itself was also where the instrument was weakest: the control that opens an item in Phase 1 carried no payload and silently discarded the change request's content, a defect visible only by using it.

Loop-backs, rejections and regressions

The demonstration exercised all three routes out of Maintenance that Section 6.4 names, and the largest ran on 20 September, when two items classified as change requests were routed through Discovery into Phase 1, producing twelve newly gherkin-locked stories: four riders on the Home Dashboard epic and a new eight-story epic, Project Media and File Attachments. The design delta was merged as a major amendment at 23:01, implementation ran until 12:30 the next day, and all twelve stories passed Testing between 13:10 and 16:55 before deploying that evening. The log marks it as “a deliberate framework loop-back from Phase 6 maintenance intake through requirements into Phase 2 design, not an implementation shortcut”. Three further items, the end-to-end test layer, storage hardening and dark mode, each ran the same complete Phase 6 to Phase 1 to Phase 5 cycle independently over the final two days.

Generated governance output was rejected as well as generated code. When the end-to-end test item reached Phase 1, the model's own single-story draft was refused at the Discovery Gate as underscoped, on the ground that it bundled two kinds of work into something that was not a boundable, closeable scope, and was split into two stories before locking. More pointedly, the Fix-Bolt Brief that the tool itself generated for story 9552560 was read and explicitly not followed, because it directed a refactor

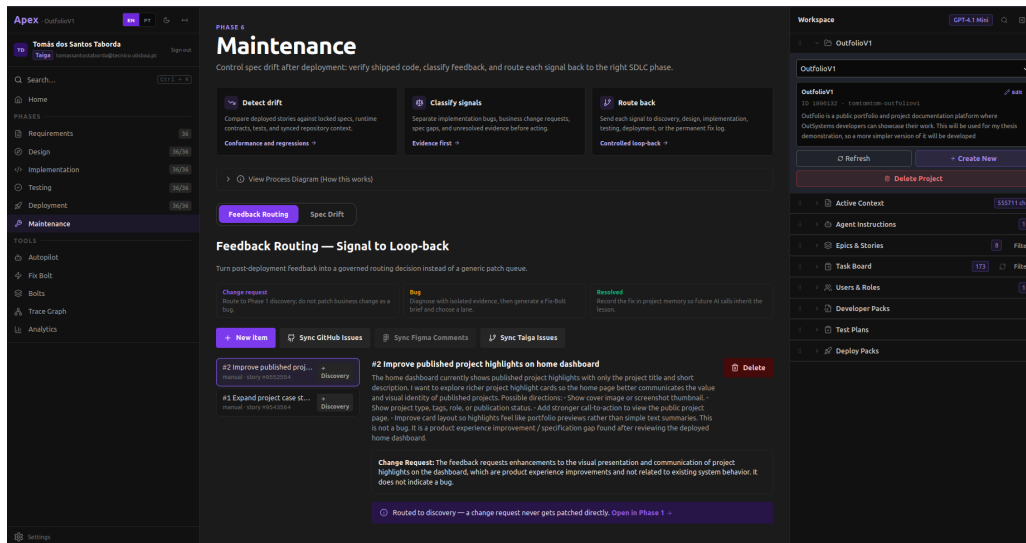


Figure 8.3: Phase 6 Maintenance Feedback Routing, 20 September 2026: a real item, "Improve published project highlights on home dashboard", classified by the framework as a change request rather than a bug, with the routing note that a change request is never patched directly and must return to Phase 1 Discovery.

onto the legacy component system, the opposite of the direction already agreed; the human was consulted before proceeding differently. An instrument whose own generated directive can be overridden at the point of use is the premise of Section 6.6.3 holding in the one case that tests it.

Testing returned real failures throughout. Story 9560176 failed because an unauthenticated visitor could not reach a correctly implemented download endpoint through any user interface path; story 9560175 failed on a non-atomic upload that partially persisted a batch it reported as entirely rejected; story 9563893 failed on a non-atomic create leaving partial records; story 9564192 failed on a crash that, in the log's assessment, "would have broken every single page of the app". Each triggered a Fix-Bolt with the failing evidence attached and was retested before release, and by the end of the demonstration the Fix-Bolt record held fourteen per-story bug reports over a permanent dated log. One event is worth more than the rest and is not softened here: a Fix-Bolt introduced its own regression. The session-context fix committed as 6c17344 caused a redirect loop after logout, which a second Fix-Bolt, ad14660, had to repair the same evening, logged at the time as a defect introduced by an earlier Fix-Bolt in the same session. A remediation cycle that can itself break the system is the strongest available evidence that the loop-back mechanism is a real control rather than a ceremony, since only a mechanism that actually re-verifies would have caught it, and it is equally evidence that a Fix-Bolt executed under deployment pressure is not a safe operation.

8.4 Observations

The unit-of-analysis statement in Section 8.1 undersold the scope actually carried, and is corrected here. That subsection commits to observing "more than one unit" across "the epics and stories fixed for it", language chosen when the pre-committed scope was six epics and roughly eighteen stories. What was actually carried through all six phases to a deployed increment was a full product: nine epics

and fifty stories, three of them added after the pre-registration through the framework's own governed routes rather than through scope creep, as Sections 8.2 and 8.3 report. "More than one unit" is true of that outcome but understates it; the demonstration exercised the framework across a project at realistic scale, with the cross-story dependencies a project of that size actually produces. Two pieces of evidence below exist only because the scope was that large: the shared-endpoint defect that propagated across nine stories (below), and the recurring Phase 3 overlap caution (Section 8.3). Neither would have been observable at the pre-registered scope, and both strengthen rather than undercut the original unit-of-analysis choice.

The governance metrics, and what each of them actually measures

The population the instantiation reports on is 54 stories, all of which reached `deployed`: the workflow funnel shows no story resting in any earlier phase status, so the figures below describe completed passages through all six phases rather than work in flight. Every number in this subsection is read from the instantiation's own analytics and specification-drift exports, which exist independently of the author's account of the work, as Section 8.1 committed. What those exports measure is not in every case the construct Section 6.6.4 defines, and the divergences are stated before the values rather than after them.

- **Bolt Cycle Time is reported only as a proxy, and a coarser one than the framework defines.** Section 6.6.4 defines it at task level precisely so execution is not conflated with queueing and gate transitions; the instantiation measures the story-level thing the framework rejected instead. That leaves this metric unable to distinguish a gate exercised from one rubber-stamped; Section 6.6.4 assigns that detection job elsewhere.
- **Context Traceability Rate is 50 of 54, 92.6 per cent**, reported on the dashboard as 93. The four incomplete chains share one identical reason, with no other reason category and none of the failure modes that would indicate the construct itself breaking down. The 7.4 per cent shortfall is a ledger gap at the last step, not a break in the chain, and strictness of that kind is the property wanted from a traceability measure. But the record it demands was written by the phase whose boundary was blurred in practice (below), so the metric's one failure mode and the framework's most-bypassed boundary are the same event from two directions.
- **Spec Conformance, an instantiation-added check rather than one of the framework's three metrics**, is a mean of 98.65 per cent (dashboard: 99) across all 54 stories. The drift found reduces to one recurring defect, three shared upload routes existing at path level while the implemented handler supports only `DELETE/GET/PUT`, inherited identically by the nine stories referencing them, evidence for Spec-Anchored Continuity rather than against it.
- **AI Defect Escape Rate is 1 of 54, 1.85 per cent**, reported as 2, the weakest of the three constructs as instantiated. Its proxy definition, a linked issue detected after the story's deployed timestamp, excludes every defect found and repaired within the same session, which is where the log shows most of them were found: nineteen Fix-Bolts were logged across 14 of 54 stories against an escape metric of one.

- **Zero regressions appear in the export, an absence of instrumentation rather than a verified negative.** Neither export contains a regression field, prior-score column or delta against an earlier check. The demonstration log contradicts the silence directly: a Fix-Bolt for the session-context defect itself introduced a login/profile redirect loop, repaired by a second Fix-Bolt the same evening, logged at the time as evidence that a Fix-Bolt cycle under time pressure can introduce new defects. One genuine regression is documented in the narrative record and zero in the metric built to count them.

Points at which generated output was modified or rejected

Section 8.1 pre-committed to recording every such point. Four classes occurred, all decision-point evidence rather than incidental corrections; the loop-backs they belong to are traced in Section 8.3.

- A generated governance artefact was **rejected outright** on story 9552560: the Fix-Bolt Brief the instantiation produced directed a refactor onto legacy components, the opposite direction already agreed; the human overrode it before implementation proceeded, the framework's own tooling refused by the hat it was advising.
- The Pre-Flight infrastructure-delta recommendation was **manually overridden twice, in opposite directions**: on story 9560172 it classified genuine infrastructure work (two database migrations) as routine, since its evidence gathering never inspected the migrations directory; on story 9564046 it flagged four already-provisioned or self-provisioning items as required, since it never checked current deployment state.
- A generated Phase 1 artefact was **rejected as underscoped rather than incorrect**: a single story drafted from a maintenance item bundled two kinds of work into a scope that could not be closed, and was split in two before locking.
- A loop-back was **deliberately declined**: the specification-scope discrepancy on story 9543480 was resolved as vacuously satisfied rather than opened as a new Phase 1 story, a restraint decision recorded at the time because a framework that answers every mismatch with new scope is as much a failure as one that answers none.

Where the framework was awkward, over-specified, or was bypassed

Six incidents follow, the first two reported first because the framework comes out of neither of them well.

- **The most severe event was a production data loss caused by the AI implementer.** On 21 September 2026, implementing a database seed script inside Phase 3, the model executed a destructive reset script (`deleteMany()` calls) without re-checking that `DATABASE_URL` still pointed at the live production database. It did; all six tables were wiped. The root cause, quoted rather than paraphrased: "a basic safety check that should have been automatic before running any destructive script, let alone one it had just identified as depending on an environment that didn't exist yet." No automatic recovery existed; the response was to stop work, inform the human, and provision an isolated Supabase project before resuming, an orphaned authentication user surfacing the next day

and resolved by hand. Three things follow. The framework has no gate at this boundary and was never going to catch this, since its controls evaluate artefacts at phase transitions while a destructive command issued inside a phase happens where nothing is evaluating anything (whether that is a closable gap or a class of risk gate-based governance cannot reach at all is a question for Chapter 10). The containment was human and ad hoc, entirely outside the method. And the incident is reported in full under the project's own stated position that failures and the response to them are thesis-relevant data, a procedural safeguard operated by the same person it constrains, exactly the unmitigated threat Section 8.1 names and not a substitute for independent observation.

- **The Testing Gate can be passed by a test suite that no longer measures the system**, the single most consequential methodological finding after the incident above. The session-gating defect already reported in Section 8.3 generalises further: story 9552560 states that "a locked story's component-level tests can pass indefinitely even after the real app has moved on to a different implementation entirely, with no automated signal that the tested code is orphaned," and story 9552562 adds that a test plan generated from the locked specification goes stale the moment a Fix-Bolt changes the underlying components, a predictable consequence of the Fix-Bolt loop rather than an accident. The framework's Regression Bypass rule, deliberately re-running the existing suite rather than regenerating it so the measuring instrument does not change, is what produces this exposure: an unchanged instrument is a stable one, and a stable instrument can be stably wrong.
- **Every defect of that class was caught by a human looking at the running system, and by nothing else.** An icon-colour defect passed its automated test and "would have been reported as done had it not been checked in an actual rendered browser." A full verification pass, backend tests, a frontend component test, type checking and linting, missed a deployment terminal-screen defect found only by retesting the real flow. A dead navigation link was "valid code, it does navigate," surfaced only by "using the feature for real." Set against Spec Conformance's 98.65 per cent, this is the chapter's sharpest tension: conformance confirms what was built matches what was specified and says nothing about whether it works, and the demonstration repeatedly produced cases where those two answers differed. The framework has no phase, gate or artefact requiring live human verification of a running increment; here that step was performed anyway, informally, and caught the defects.
- **A governance surface that reports confidently and is wrong is worse than no surface.** The two Pre-Flight overrides above are the same root cause in opposite directions, a false negative then a false positive days apart, both produced with no expression of uncertainty; the same pattern recurs subtler in the log, a classification correct for the current step "because that infra is already live, not because the story was infra-free from the start," correct by accident and indistinguishable at the surface from correct by analysis. This is the stale-context-at-high-confidence pattern Section 7.5's formative evaluation already identified, recurring here in a different mechanism, suggesting a property of advisory AI surfaces in general rather than a defect in one of them.
- **The Implementation-to-Deployment boundary blurred, and the framework's own metrics went blind for the same reason.** A story scoped "deploy and verify" as its own work cannot be done without deploying, so a real production deployment occurred while the record still read Phase 3 and

no distinct Phase 5 gate was opened, the log diagnosing that "a small-team, single-developer setting makes it easy for a phase that exists on paper to be satisfied informally rather than as a distinct, named event." Because board status was never advanced through the interface, the governance analytics panel showed all 36 stories then in scope sitting at implementation with zero conformance checked while the real work was deployed and running: the gate bypassed and the metric that reported nothing are one failure, not two, and the four incomplete traceability chains above are its residue.

- **A formally available hat was never worn.** Deployment entries through the Fix-Bolt cascade repeatedly record the Developer and Tech Lead hats authorising push and deploy, with no separate Security Reviewer sign-off invoked, the admission recurring rather than appearing once. Section 6.5 designs hats as responsibilities rather than people specifically so one person can hold several, and this demonstration is the limiting case: the design tolerates a hat going unstaffed without producing any signal, nothing in the framework or the instantiation detecting that a defined responsibility was never exercised. Smaller and related: at the Phase 1 gate, hat metadata was supplied after the gate statement rather than before it, the declaration trailing the decision it was meant to qualify.

8.5 Summary

What this demonstration establishes is that the framework's governance constructs are not merely implementable, the claim Section 8.1 pre-registered as the instantiation's own contribution, but that they were genuinely exercised under real conditions rather than walked through once cleanly. The scope grew from six epics to nine without anything abandoned; all three of the framework's named loop-back routes ran, more than once each; a generated governance artefact and two generated advisory verdicts were rejected by the human who used them; a real regression was caught by the same re-verification the loop-back mechanism exists to provide; and the traceability, conformance and defect-proxy metrics all resolved to real, checkable numbers rather than to placeholders. A demonstration in which nothing failed would have shown nothing; this one shows the gates holding under real failure, not merely existing.

What it does not establish is stated as plainly. Every observation above was made by the same person who designed the framework, built its instantiation, and is the sole hat-holder in the one field-work setting that ran; Section 8.1 names that threat as unmitigated, and nothing in the record above changes that, since a designer who found his own governance mechanism silently broken three times in the same chain is still the designer reporting on himself. The demonstration also surfaced limits the framework does not answer: no gate exists for a destructive action taken inside a phase rather than at its boundary, the Testing Gate can be satisfied by a suite that no longer measures the system it tests, and an advisory governance surface can report high confidence while wrong in either direction with no signal distinguishing that case from being right. These are not incidental defects of one instantiation; they are findings about what phase-boundary, artefact-checking governance can and cannot reach, and Chapter 10 takes them up as such.

Chapter 9 takes a different kind of evidence than this chapter does: where this chapter reports what one designer observed while using his own design on his own project, the evaluation adds a fixed task set administered to thirteen participants who did not design the framework, which is the closest this study

comes to the independent application the abandoned partner-organisation setting would have supplied. The two are read together rather than as substitutes for one another.

9

Evaluation

Contents

9.1 Evaluation Strategy	63
9.2 Participants	64
9.3 Instruments and Procedure	66
9.4 Results	71
9.5 Threats to Validity	76
9.6 Discussion	77

This chapter reports the Evaluation activity of the DSRM. Following Pries-Heje et al.'s distinction between *when* and *how* an artefact is evaluated [25], the evaluation carried out here is *ex post*, since it assesses the artefact after construction and on the basis of its actual use, and *naturalistic*, since it takes place in a real organisational setting with real users rather than in a laboratory.

9.1 Evaluation Strategy

Two artefacts were produced in Chapters 6 and 7: a framework and its instantiation. They are not evaluated by the same means, and conflating them would weaken both claims. Table 9.1 therefore states which instrument evaluates which artefact, and which research question it addresses.

Table 9.1: Evaluation instruments, target artefact, and research question addressed

Instrument	Artefact evaluated	DRQ	What it establishes
Demonstration in a real project	Framework and instantiation	DRQ3, DRQ4	That the artefact is applicable and produces the intended artefacts and decisions
System Usability Scale (SUS)	Instantiation (Apex)	DRQ4	Perceived usability of the supporting tool
NASA Task Load Index (NASA-TLX)	Instantiation (Apex)	DRQ4	Perceived cognitive workload imposed on the practitioner
Apex User Experience (UX) questionnaire (author-designed)	Instantiation (Apex), with two items scoped to the framework	DRQ4	Trust in AI-generated content, legibility of phase gates, and error surfacing, which SUS and NASA-TLX are not designed to reach
Practitioner interviews	Framework	DRQ1, DRQ2	Practicality, feasibility, and perceived value in real workflows
Agile and Scrum expert interviews	Framework	DRQ1, DRQ2	Alignment with, and departure from, established delivery methodology
Analytical assessment against criteria	Framework	DRQ1, DRQ2	Degree to which stated design criteria are satisfied

SUS and NASA-TLX measure Apex, not the framework. Both are administered to someone who has spent a session operating a user interface, and neither contains an item about phase decomposition, gate placement, or accountability for generated artefacts. A favourable SUS score therefore establishes that the tool is usable and nothing at all about whether the method it implements is sound; an unfavourable one, correspondingly, is not evidence against the method. That is a design decision rather than a defect: the framework's own validity is carried by the practitioner and expert interviews (Section 9.3.4), the analytical assessment against criteria fixed in advance (Section 9.3.5), and the demonstration (Chapter 8), each of which reaches the method directly.

One instrument spans both artefacts, and the split is handled rather than ignored: two items of the author-designed UX questionnaire ask about the process rather than the product, and are analysed apart from the fifteen instantiation-scoped ones, as Section 9.3.3 sets out.

9.2 Participants

Participants were recruited through the supervisor's own professional connections. There was no open call, no advertisement and no mailing list, and the number of people approached was not recorded, so no response rate can be reported and none is claimed. Thirteen sessions were completed, coded P0 to P12; all thirteen returned all four instruments and no participant withdrew. P0 was planned as an uncounted pilot and is counted as a full participant, because the session produced complete, correctly labelled data under the same consent form, task script and instruments every later participant received,

and discarding usable data on the strength of its position in the sequence alone had no principled basis. The pre-registered target was twelve; the achieved sample is thirteen, and the sample is not drawn from a single organisation but from one person's professional network, which is a different limit rather than a weaker one.

The sample is junior and mixed. On years of professional experience, three participants were students, three had under one year, and seven had one to three years; nobody had more than three. By current role, six were students, four developers and one an AI engineer, and two came from outside software development entirely, one a communications network technician and one a partner-manager in a hairdressing business. Prior exposure to AI coding assistants, pre-registered as the strongest expected confound, spread across the whole scale: four used them daily, four weekly, two monthly, two had tried one once or twice, and one had never used one. Familiarity with the notations and tools the session assumes was low by design of the sample rather than by selection: nine of the thirteen had no familiarity with Gherkin or BDD and four had seen it, with nobody reporting regular use; ten reported no familiarity with Taiga, the project-management tool the session runs against, two some, and one left the item blank. Nine participants chose English as their session language and four Portuguese, and ten completed the instruments in English against three in Portuguese; the two participants for whom those two facts disagree are reported in Section 9.5 rather than reconciled here.

What that sample supports is bounded. Thirteen respondents meets the twelve at which a SUS mean is generally treated as stable, so the mean is reported, with the distribution and every individual score alongside it. It remains a convenience sample, skewed junior and assembled from a single network, and what that does and does not support is set out in full in Section 9.5.

Every participant gave informed consent before any data was collected, as three separate tick boxes on the first page of an online form completed before the task script was opened, so that no response was collected ahead of consent and agreeing to take part did not imply agreeing to be quoted or to be contacted for a follow-up interview. The form is reproduced in Appendix B. All thirteen consented to participate; eleven consented to anonymised quotation and eleven to being contacted, the two sets overlapping at nine, and quotations below are drawn only from the eleven. No submission to an institutional ethics committee was required: the supervisor confirmed that for a study of this kind informed consent, together with anonymisation and a stated retention period, is sufficient at Instituto Superior Tecnico (IST), and that no standard institutional consent form exists that would supersede the one used here.

No session was observed and nothing was recorded: no screen capture, no audio and no video, of the task sessions or of the interviews, and the consent form contains no recording item because there was nothing to seek permission for. What was collected is the questionnaire responses, the timestamps at which each form was submitted, and, for the interviews, contemporaneous written notes taken by the researcher and written up in full immediately after each interview. The methodological cost of this choice, principally that a set of written notes is selective in a way a full transcript is not, is set out in Section 9.5.

Participants are identified only by a code of the form P0 to P12. Names, employers, and identifying project details were not recorded with the responses, and quotations reproduced in this chapter are anonymised, with any detail that could identify a participant or their organisation removed or generalised. Only wording written down verbatim at the time is presented as a quotation; the remainder

of the interview material is presented as paraphrase and is marked as such. The anonymised questionnaire responses and interview notes are retained, accessible only to the researcher, as stated in Appendix B.1, so that the aggregate figures reported in Section 9.4 can be independently recomputed.

9.3 Instruments and Procedure

The usability study was administered entirely unmoderated. Each participant worked alone, at their own pace, from a fixed written task script, and submitted the questionnaires as online forms without the researcher present. A second, moderated arm of a small number of observed think aloud sessions was designed and deliberately dropped, for two reasons. First, no session was to be recorded, and an observed session without a recording yields notes written by a facilitator who is simultaneously facilitating, which is the weakest form of the only evidence such an arm exists to produce. Second, and more seriously, the author designed both artefacts under evaluation; an unmoderated design removes that conflict structurally rather than mitigating it, since no facilitator is present to steer, hint, or react. What the decision costs, namely measured task completion, time on task, assist counts, and any observational account of where participants struggled, is stated in Section 9.5.

9.3.1 System Usability Scale

The SUS [52] is a ten-item instrument scored on a five-point scale of strength of agreement, administered after the participant has completed a defined set of tasks. Participants responding in Portuguese are given the validated European Portuguese version [53] and those responding in English the original wording, both as reproduced in Appendix B.3; the two language versions are reported separately rather than pooled.

Each item contributes between zero and four points: for the odd-numbered, positively worded items the contribution is the response minus one, and for the even-numbered, negatively worded items it is five minus the response. The ten contributions are summed and multiplied by 2.5, giving a value between 0 and 100. That value is not a percentage: 100 is not a perfect result and 50 is not half of anything, which is why it is not left to speak for itself.

Two published interpretation scales are applied to it, because they say different things about the same figure and reporting only one invites the more flattering reading. Bangor et al. attach adjectives to score ranges from a corpus of over two thousand administrations, placing *Poor* at 35.7, *OK* at 50.9, *Good* at 71.4 and *Excellent* at 85.5 [54]. Sauro and Lewis convert a score into a percentile against their own corpus and assign a curved grade from A to F, in which the corpus mean of approximately 68 sits at the fiftieth percentile rather than at a pass mark [55]. Both are reported in Section 9.4.

The two-factor decomposition of Lewis and Sauro is reported alongside the overall score: Usability from the eight items other than 4 and 10, rescaled by 100/32, and Learnability from items 4 and 10, rescaled by 100/8 [56]. The two sub-scales can move in opposite directions, and a tool that is hard to learn but workable once learnt is a different finding, with a different remedy, from one that is neither.

The Portuguese results carry a caveat the validation study itself states. Construct validity was established against the Post-Study System Usability Questionnaire at $r = 0.70$, but inter-rater reliability was weak, with an intraclass correlation coefficient of 0.36 and a 95 per cent confidence interval of 0.01 to

0.63, and percentage agreement reached 76.67 per cent against the authors' own 80 per cent threshold, which they attribute to the alternating polarity of the items; the validation sample was drawn from the general community rather than from software practitioners [53]. With only three Portuguese respondents in this study, the property that instrument is weakest on is exactly the one a three-person mean depends on most, and the Portuguese figures are accordingly reported separately and read as indicative only.

9.3.2 NASA Task Load Index

The NASA-TLX [57] measures perceived workload across six subscales: mental demand, physical demand, temporal demand, performance, effort and frustration. This evaluation uses the unweighted Raw NASA-TLX variant [58], omitting the fifteen pairwise weighting comparisons. The instrument is administered once after each of the marked tasks rather than once at the end of the session, which is the instrument's intended per-task use and yields a workload profile per lifecycle phase rather than a single aggregate.

The Raw variant follows from that choice of administration: those fifteen comparisons would become ninety per participant under per-task administration, well beyond what the session can absorb, and the weights earn their cost primarily when workload is compared across experimental conditions, which this evaluation has none of. It is not, however, adopted only on grounds of feasibility. Reviewing five hundred and fifty studies twenty years after the instrument's publication, Hart records that removing the weighting is the most common modification made to the NASA-TLX, and that across the twenty-nine studies in which the two versions were directly compared the Raw variant was found to be more sensitive, less sensitive, or equally sensitive depending on the study, with no consistent advantage to weighting [58]. The subscale scores, which are this evaluation's primary result, are in any case identical under both versions, since the weighting affects only how they are combined into a single aggregate.

Two departures from the published instrument are recorded here. Workload was collected on a linear scale from 0 to 10 rather than on the original twenty-interval line, and the responses were rescaled to the instrument's 0 to 100 range by multiplying by ten; no form tool suitable for an unmoderated study reproduces a twenty-interval line. The subscales were also reordered to place the reversed performance scale last rather than fourth, because a reversed scale encountered in the middle of five forward ones, with no facilitator present to explain it, is the most plausible way a participant inverts a sixth of the data set. Neither change affects the unweighted Raw NASA-TLX computation. The first, however, reduces the resolution of the instrument: every response is a multiple of ten, so the resulting values are not directly comparable to studies using the original response scale, and small differences between cells of the workload matrix are not interpreted as meaningful.

Repeated administration carries its own cost. Rating the same six dimensions six times invites anchoring, where later ratings drift towards earlier ones, and fatigue on the final tasks of the session; neither is avoidable under per-task administration, and both are carried into Section 9.5 and weighed there against the workload profile that administration is what makes obtainable.

Both instruments are administered against one fixed task set, identical in wording and order for every participant. Nine tasks carry a single Unit of Work, a password-reset feature, from a written requirement through to a deployed increment: sign in and open the seeded project (1), connect it to a GitHub repository (2), take the requirement to locked Gherkin (3), produce and lock the design (4), sign out and resume the session (5), break the story into implementation tasks (6), generate a test plan

and record the QA sign-off (7), deploy the feature (8), and export the resulting artefacts (9). Six of the nine, tasks 3, 4, 6, 7, 8 and 9, each carry a Raw NASA-TLX form; tasks 1, 2 and 5 are setup and session-resumption steps and carry none. All seventy-eight administrations the design calls for, thirteen participants by six marked tasks, were received, so no workload cell below rests on a partial set of respondents. The project environment was reset to an identical seeded state before every session, so that no participant met another's work.

Task 8 was originally a refusal test, in which the participant would attempt to deploy a story that had never passed QA and be blocked at the gate. It was redesigned on 30 August 2026 so that tasks 3 to 9 form one continuous flow: the deployment in task 8 now succeeds, and succeeds *because* of the QA sign-off given in task 7. What that cost the UX questionnaire is stated in Section 9.3.3.

9.3.3 Apex UX Questionnaire

An eighteen-item questionnaire was written for this evaluation to reach what SUS and Raw NASA-TLX are not designed to say: whether a participant trusted AI-generated content, whether a phase gate read as a guardrail or as obstruction, and whether reviewing generated work felt cheaper than writing it. Its items are grouped into five constructs: understanding where you are in the process, working with AI-generated content, control and reversibility, feedback and errors, and fit to real work. It is not a validated instrument and is not treated as one: it is reported item by item, with no composite score or subscale means, since collapsing it into a single figure would claim more precision than it supports. Administered last, after SUS, and reproduced in full in Appendix B.

Two items in the fifth construct, on whether the artefacts and the process fit real work, ask about the framework rather than the instantiation and are analysed under the framework's evidence rather than Apex's (Table 9.1): keeping them on the same sheet is convenient for the participant, splitting them out in analysis is what keeps the instrument from quietly undermining that separation.

One item pair is expected in advance to be the most diagnostic in the instrument. Item B4 asks whether the participant trusted the generated content enough to use it in a real project, and item B2 whether reviewing that output took less effort than writing it. Neither is interesting alone; read together they are a trust-calibration pair. High trust reported alongside low review effort is the signature of acceptance without scrutiny, which is the failure mode the framework's gates exist to prevent and which no participant will report directly when asked. High trust alongside high review effort is the opposite and far less worrying reading: content believed because it was checked. The pair is the closest an unmoderated, self-report design can get to the observation that task 4 was designed to elicit, namely whether generated output is edited or accepted wholesale.

A second cross-read was designed into the instrument and is no longer available. Item A4, *when the tool refused to let me continue, I understood why*, was to be read against the NASA-TLX Frustration score and free-text response for task 8, which was at that point a deliberate refusal test; with no observer present and no scored comprehension check, that triangulation was the study's only evidence on whether a phase gate reads as a guardrail or as obstruction. When task 8 was redesigned (Section 9.3.2) the guaranteed refusal went with it, and A4 was removed from both language versions on 30 August 2026 rather than left to be answered by participants who had not been refused anything. Seventeen of the eighteen items written were therefore administered, and A4 is not missing data, since the item does not exist for any of the thirteen participants. What was lost is not one item but the study's

only measure of gate comprehension, and that loss is carried into Section 9.5.

9.3.4 Semi-Structured Interviews

Interviews were conducted with a population separate from the unmoderated task-based study reported in Section 9.2: practitioners and an Agile/Scrum expert recruited individually by the researcher rather than as a follow-up to that arm. This is a deliberate design choice rather than a gap. The interview guides in Appendix B ask about the framework's practicality, its adoption cost, and its alignment with established delivery methodology; these are questions a participant who has just spent an hour on nine fixed tasks is poorly placed to answer for their own team and organisation, and the Agile-expert guide explicitly does not require the tool to have been used at all. The themes below were derived from the written notes described in Section 9.2: the codebook was developed from the first write-ups in each group and applied across the remainder, with codes added as they emerged. Session duration was not logged systematically and is not reported.

For the same anonymity reason stated in Section 9.2, interviewees are identified here only by a code: T1 to T5 for practitioners and E1 for the Agile/Scrum expert. These are kept deliberately separate from the P-codes used for the unmoderated study's participants, since the two populations are disjoint and neither did the other's protocol.

Five practitioners and one Agile expert had been interviewed at the time of writing, against Guide 1's target of five to eight practitioners and Guide 2's target of two to three experts; one practitioner, T3, was interviewed a second time after the framework had been substantially revised in response to the first round. Both groups therefore sit at or below the low end of their target, and recruitment continues past submission of this draft; the sample is reported at its current size rather than held back for a target that would postdate the deadline. What follows should be read at that scale, as a small number of attributable, individually rich sessions rather than a saturated qualitative sample, and the themes below name which interviewee raised which point precisely because the sample is too small for that attribution to be smoothed away by aggregation. Two of the five practitioners, T1 and T5, are not quoted below: their sessions took place earlier, during the tool's construction, and their material is reported instead in Section 7.5, where it did concrete work in changing the design. T2, T3 and T4 are the three whose interviews are this section's evidence.

A - Practitioners. T2 offered the framework's central bet, that a locked specification should be the durable artefact and code should be regenerable from it, its clearest unprompted endorsement in this group: specifications, in his description, function like a library for project information and as the vehicle for guidelines, requirements, plans and tests kept together. The same practitioner also flagged, unprompted, the difficulty of making every implicit assumption in a specification explicit.

T4 raised semantic versioning of the specification as a requirement rather than a suggestion, arguing that a human must always be able to consult and edit an artefact's history rather than trust it only as it currently stands. The same practitioner named, unprompted, the risk that a strictly linear phase sequence reintroduces waterfall-style rigidity, arguing for feedback loops rather than a one-directional pipeline. This second point is worth isolating because the Agile-expert guide in Appendix B was written anticipating exactly this criticism from methodology experts specifically; a practitioner raising it unprompted, without

an expert being asked to probe for it, is stronger evidence that the concern is substantive than if it had only ever come from a session designed to elicit it.

T3's first interview raised two minor, feature-level points reported for completeness rather than as developed themes: more project-management-tool detail in AI-generated requirements, and clearer on-screen state during UI review. T3's second interview produced the sharpest challenge the framework received in this study, and its content, together with the design changes it is directly traceable into, is reported in full in Section 7.5 rather than here.

B - Agile/Scrum expert. The one expert interview obtained, following a live presentation of the framework and a demonstration of Apex, produced the clearest validation this study collected of the demonstration argument made independently in Chapter 8: that a tool instantiating and applying a framework's roles and phases makes the framework legible in a way a description on paper does not. The specific comparison volunteered was against the rigid, closed-role structures typical of methodology adopted inside large organisations, against which the framework's hats and its refusal to lock roles to named individuals read as a genuine point of difference; the Mob Elaboration ritual and the framework's governance metrics drew the same reaction, read as departures from what the expert's own organisation does today.

Two further observations from the same session are reported as open limitations, because neither produced a design change. The expert also flagged first-contact navigation confusion mid-demonstration, consistent with the orientation difficulty this evaluation's own quantitative instruments name independently (Section 9.6), though here as a single, undeveloped observation rather than a measured finding. The second concerns the expert's own organisation rather than Apex: AI tool use there is centrally governed, restricted to a single permitted tool, and metered by token consumption, yet no role or specialist owns that governance, a gap large enough that the organisation is actively hiring against it. The expert raised, unprompted, whether AI participation in a lifecycle nets out as a cost or a benefit; the framework's governance metrics (Section 6.6.4) are positioned to answer exactly that question but do not yet operationalise it for token cost specifically, and the point is carried forward as a direction for future work in Chapter 10 rather than pursued further here.

C - Read across both groups. The strongest convergent signal is the linearity concern: raised unprompted by a practitioner using the tool, and anticipated independently in the design of the expert guide before any interview took place, it reaches this study from two directions that were never told to look for the same thing. The strongest divergence spans two sections rather than sitting inside this one: T2's unprompted endorsement of the specification-as-source-of-truth construct is reported above, while the sharpest doubt this study collected on the same construct, from the same population, is reported in Section 7.5. Naming that split matters, because the favourable reading is not reported as though it were uncontested.

9.3.5 Analytical Assessment

The framework is additionally assessed against five criteria fixed before the results were seen rather than derived from them. *Clarity* asks whether each construct is defined precisely enough to be applied

without its author present. *Completeness* asks whether the framework covers the lifecycle it claims to cover, including its own defined measurements. *Alignment with SDLC phases* asks whether the phase decomposition maps onto recognised lifecycle stages without merging, splitting or omitting one. *Support for human-in-the-loop practice* asks whether the framework makes human review of generated output structurally necessary rather than merely advisable. *Governance of AI-enabled activity* asks whether AI contributions are made traceable, attributable and measurable by mechanism rather than by discipline.

Each is rated on a three-level ordinal scale. *Met* means the criterion is satisfied and evidence exists that it is. *Partially met* means it is satisfied in part and the shortfall is identifiable rather than merely suspected. *Not met* means it is not satisfied. No finer scale is used, because the evidence available cannot distinguish more than three levels.

Each rating is justified against evidence that exists independently of the author's opinion of his own design: the construct-to-mechanism mapping in Table 7.1, whose negative rows are as usable here as its positive ones; the interviews in Section 9.3.4; and incidents recorded during the study sessions themselves. Where the only available support for a rating would be the author's own assessment, the rating is lowered rather than argued, on the principle that a criterion which cannot be evidenced is not distinguishable from one that is not met. The ratings are reported in Table 9.4.

9.4 Results

9.4.1 System Usability Scale

Thirteen participants returned complete SUS responses with no blank item. Every score is given individually in Table B.6 (Appendix B), since at this sample size the distribution carries more information than the mean.

The mean across all thirteen is **56.73**, with a standard deviation of 18.50, a median of 60 and a range from 15 to 80. That is below the published corpus mean of approximately 68: on the curved grading scale it is a **grade D**, in the fifteenth to thirty-fourth percentile band, and on the adjective scale it sits between *OK* at 50.9 and *Good* at 71.4, closer to the former. The two language groups differ, and are not pooled for interpretation: the ten English responses average 60.50 (SD 19.18, median 63.75), also grade D, while the three Portuguese responses average 44.17 (SD 9.46, median 40), which is a **grade F**, in the zeroth to fourteenth percentile band. Three respondents cannot support a claim about Portuguese-speaking users, but the figure is stated (Sections 9.3.1 and 9.5).

The spread is the more informative part: six of the thirteen scores are above 60, two are at 60, and five are below it, with four at or above 70, the region the adjective scale calls *Good*. One participant, P6, scored **15**, below Bangor's *Awful* anchor at 20.3 and near *Worst imaginable* at 12.5. That score is reported as recorded and is not excluded, and it should be read alongside one observation about the same participant's other data: of the thirty-six Raw NASA-TLX ratings P6 gave, thirty-five are the identical value, which is the signature of a constant response style rather than a differentiated judgement. The observation cuts both ways and neither is pursued, since thirteen participants give no capacity to absorb one atypical respondent and no grounds to remove one.

The sub-scales separate in the way the decomposition exists to expose. Usability averages 59.86 and Learnability 44.23, so the weaker of the two is the one concerning how much has to be learned

before the tool can be used at all. Learnability is also far more dispersed, running the full 0 to 100: P2 scored 0, maximally agreeing both that they would need technical support to use the system and that they would need to learn a great deal before getting going, while P11 scored 100, disagreeing with both as strongly as the scale allows. Usability by comparison runs from 12.50 to 87.50. The reading this supports, and which Section 9.6 takes up against the other instruments, is that the difficulty reported is concentrated at first contact rather than distributed evenly through use.

9.4.2 Raw NASA Task Load Index

Table 9.2 is the primary workload result: the six subscales against the six marked tasks, each cell a mean over all thirteen participants. Reporting the aggregate alone would discard the part of the instrument that does the work here, since the six subscales move independently and their shape is the finding.

Table 9.2: Raw NASA-TLX subscale means by task, over thirteen participants, on the instrument's 0 to 100 range. Performance carries reversed anchors and is reported as marked, so a higher value is a self-assessment further from *Perfect*. Every response is a multiple of ten, so differences of a few points between cells are not interpreted.

Task	Mental	Physical	Temporal	Effort	Frustr.	Perf.	Aggregate
3 Gherkin	26.9	17.7	21.5	23.8	13.1	14.6	19.6
4 Design	47.7	21.5	40.8	36.2	20.0	16.2	30.4
6 Packs	40.0	21.5	32.3	36.2	26.2	18.5	29.1
7 QA	25.4	13.8	19.2	23.1	15.4	10.8	17.9
8 Deploy	25.4	20.8	26.9	29.2	22.3	21.5	24.4
9 Export	9.2	7.7	6.9	7.7	3.1	10.8	7.6
All tasks	29.1	17.2	24.6	26.0	16.7	15.4	21.5

Collapsed across all seventy-eight administrations the grand aggregate is 21.5 out of 100, and the profile has a definite shape. Mental demand is the largest component at 29.1, followed by effort at 26.0 and temporal demand at 24.6; frustration, at 16.7, is among the smallest, below physical demand. The burden this tool imposes therefore sits in comprehension rather than in struggle, the distinction the subscale profile exists to make. Performance, reported as marked with its reversed anchors, averages 15.4, meaning participants placed their own success close to the *Perfect* end throughout.

By aggregate, Design and Packs are the two heaviest tasks, the two in which the largest volume of generated content is put in front of the participant for review; Export, which generates nothing, is lightest by a wide margin. That the workload tracks the quantity of generated material to be assessed, rather than the number of interface steps, is consistent with the framework's own account of where it puts the practitioner's effort.

Three predictions were recorded before administration and two of them were wrong. Task 3 was predicted to carry the session's highest mental demand; it carried the second highest, 26.9 against Design's 47.7. Task 6 was predicted to be moderate throughout on the grounds that it is largely a review task; it came second by aggregate, tied for the highest effort score at 36.2, and carried the highest

frustration of any task at 26.2. Two predictions held: task 9 was predicted lowest overall and was lowest on every single subscale, and physical demand was predicted to sit near the floor and never exceeded 21.5. Task 8, predicted moderate throughout following its redesign away from a refusal test, came third by aggregate but carried the highest Performance value of any task at 21.5, meaning participants rated their own success on the deployment further from *Perfect* than anywhere else in the session. Section 9.6 reads that one against an independently recorded defect in the same feature.

9.4.3 Apex UX Questionnaire

Table B.5 reports all seventeen administered items individually, as response counts and a median, with no composite score and no construct means, for the reason given in Section 9.3.3; the full matrix is reproduced in Appendix B, and Table 9.3 below reproduces the nine items this section discusses individually, so they remain locally analysable. Responses of *N/A* - *did not use this* are counted separately and excluded from the median.

Table 9.3: Apex UX questionnaire responses for the nine items this section discusses individually, $N = 13$. Scale 1 = strongly disagree to 5 = strongly agree; the *N/A* column counts participants who marked *did not use this*, which is excluded from the median. Items E2 and E3 evaluate the framework, not the instantiation. The remaining eight items (A1-A3, B5, C1-C4) sit only in the full matrix, Appendix B.

Item	1	2	3	4	5	N/A	Mdn
<i>B. Working with AI-generated content</i>							
B1 Could tell generated content from my own	0	2	0	3	7	1	5
B2 Reviewing took less effort than writing it	0	0	2	6	5	0	4
B3 Felt able to reject or change the output	0	0	0	1	11	1	5
B4 Trusted it enough to use in a real project	0	0	3	8	2	0	4
<i>D. Feedback and errors</i>							
D1 Was told clearly what had happened when something went wrong	0	1	6	4	2	0	3
D2 Messages said what to do next, not only what failed	0	0	4	3	6	0	4
<i>E. Fit to real work</i>							
E1 The artefacts are ones my team would use	0	1	3	5	1	3	4
E2 (framework) The process fits how my team works	0	2	2	6	0	3	4
E3 (framework) The effort is worth what it produces	0	0	2	5	6	0	4

Fifteen of the seventeen items are instantiation-scoped and are read first. Ten of the fifteen have a median of 4 and four have a median of 5: being able to tell generated content from one's own (B1), feeling able to reject or change it (B3), confidence that a step could be undone (C2), and the acceptability of waiting for generation to finish (C4). One item sits lower than every other closed item in the instrument: **D1**, on being told clearly what had happened when something went wrong, has a median of **3**, with six of the thirteen at the neutral point and one below it. Its paired item D2, on whether messages said what to do next rather than only what had failed, sits at 4, so what participants report is not an absence of messages but an absence of a legible account of what went wrong.

The trust-calibration pair predicted in Section 9.3.3 to be the most diagnostic does not diverge at the median: B4, trust sufficient for real use, and B2, reviewing being cheaper than writing, both sit at 4, with

no participant below 3 on either. On its face this is calibrated trust, which is the reassuring reading. It is qualified by the free text of a single participant who describes the opposite state directly, having believed the models were properly grounded and so, in their own words, “I did not review in great detail and I trusted it completely” (P7). One self-report is not a rate, and the design cannot say how many others were in the same position while answering 4, because nothing was observed. The limit on what that establishes is carried into Section 9.5.

E3, on whether the effort of following the process is worth what it produces, is among the more strongly supported items in the instrument, with none disagreeing. E2, on whether the process fits the way the participant’s team works, is the weakest-supported item of the seventeen. Three participants marked *did not use this*, no one strongly agreed, two of the ten who did answer disagreed and two more were neutral, so its median of 4 rests on the six remaining agreements. The three *N/A* responses on E2 come from the same three participants who marked *N/A* on E1, so the missing values are not independent of one another; those three answered E3 normally. That a majority endorses the process as worth its cost while fewer will say it fits their own team is the most interesting result in this instrument, and it is a question about the framework rather than about Apex.

9.4.4 Analytical Assessment

Table 9.4 gives the rating for each of the five criteria stated in Section 9.3.5, with the evidence each rests on. No criterion is rated *not met*, and that is not a favourable finding in itself: these are the framework’s own design criteria, so a *not met* would record a failure at something it explicitly set out to do. The information is in the three *partially met* rows, each of which names a specific shortfall rather than a general reservation.

Table 9.4: Analytical assessment of the framework against the five criteria fixed in Section 9.3.5, with the evidence each rating rests on.

Criterion	Rating	Justification and evidence
Clarity	Partially met	Every construct in Table 7.1 resolved to a nameable mechanism, which is the operational test of whether a definition is precise enough to act on, and a practitioner described the central construct back unprompted, as a library for project information carrying guidelines, requirements, plans and tests together (T2, Section 9.3.4). Against that, spec-drift detection was found to presume a premise the framework never states, that the specification is scoped per story, and the omission surfaced only when the implementation failed on it (Section 7.7). A definition whose preconditions are discoverable only by building it is not yet clear.
Completeness	Partially met	Strategic Alignment and six lifecycle phases through to Maintenance are addressed, with three named loop-backs, so no stage of the lifecycle the framework claims is left uncovered (Table 6.3, Figure 6.1). Two of its own measurements fall short of their definitions: AI Defect Escape Rate is not instantiable at the observation point it presumes and exists only as a weaker proxy, and spec-drift detection holds for the functional specification but not the technical one (Table 7.1). The gap is in the definitions rather than the coverage, and it is identifiable, which separates this rating from <i>not met</i> .

Continued on next page

Criterion	Rating	Justification and evidence
Alignment with SDLC phases	Met	Each phase maps onto a recognised lifecycle stage with a defined AI contribution, an accountable hat and an exit control (Table 6.3), and the decomposition survived instantiation with no phase merged, split or skipped (Table 7.1). No interviewee questioned the phase set. One practitioner questioned its linearity (T4, Section 9.3.4), which is a claim about the transitions rather than the stages, and was answered by making the loop-backs out of Maintenance explicit rather than by altering the phases.
Support for human-in-the-loop practice	Met	Every phase terminates in a write to the specification store and a human acceptance, so none can be exited by generation alone (Figures 7.3 and 7.4); the Interactive Bridge, the Consistency Factor and server-side reconciliation of generated claims are all realised (Table 7.1). Participants report the preconditions for review independently: median 5 on telling generated content from their own, and eleven of thirteen strongly agreeing they could reject it (Table B.5). The criterion concerns whether review is structurally supported, not whether it happens; one participant reports not reviewing in detail at all (P7), and nothing in the framework would detect a gate satisfied without scrutiny.
Governance of AI-enabled activity	Partially met	Gate enforcement and traceability are machine-checkable rather than procedural: per-story phase status refused by middleware, stable identifiers resolving every artefact to a requirement, and a Context Traceability Rate computed as a real aggregate (Table 7.1). Two shortfalls are specific. The framework's own measure of AI-attributable defects exists only as a proxy over reported, linked, post-deployment issues, so the regime cannot see most of what it governs (Section 7.7). And during the study a governance surface reported a verdict its evidence did not support (the stale-context Phase 5 defect, Section 9.5), with four participants' task 8 free text converging on it. A control that can report confidently from stale evidence is not yet trustworthy without an independent check.

9.4.5 Demonstration in a Real Project

Table 9.1 names the demonstration as an instrument for DRQ3 and DRQ4 alongside the four above; Chapter 8 narrates it in full and Section 8.5 already states its headline facts (nine epics, fifty stories, all three loop-back routes exercised, one regression caught and repaired). What follows is the governance-metric detail behind those facts and how it bears on Table 9.4.

The instantiation's own governance exports return the three metrics Section 6.6.4 defines, and one it does not. Context Traceability Rate is 50 of 54 deployed stories, 92.6 per cent (the shortfall's single root cause is given in Section 8.4). Spec Conformance, an instantiation-added check rather than a framework metric, is a mean of 98.65 per cent across all 54 checked stories, with the finding-level detail and its shared-component root cause in Section 8.4. AI Defect Escape Rate is 1 of 54, 1.85 per cent, subject to the same-session blind spot already noted there. Nineteen Fix-Bolts were logged across 14 of the 54 stories.

Two results from the demonstration bear on Table 9.4's ratings above and are cited there rather than repeated: the Testing Gate was shown passable by a suite that had stopped measuring the system it tested, and a governance surface, the Phase 5 pre-flight check, was found reporting high confidence on stale evidence on two further occasions beyond the one already recorded in the study proper (Section 9.6). A third result has no instrument above to attach to: on one occasion the AI implementer

executed a destructive command against the live production database without checking which environment it was pointed at, a failure inside a phase rather than at a gate, which no construct in Table 7.1 was positioned to catch (Section 8.4).

9.5 Threats to Validity

A - Construct validity. Every instrument here measures perception, and none measures delivered software quality. SUS and Raw NASA-TLX are standardised and comparable against published corpora, which is the mitigation that does apply; the UX questionnaire's construct-validity limit is that it is author-written and unvalidated, addressed by treating it as such (Section 9.3.3). Two departures already described (Section 9.3.2) reduce what the workload instrument can resolve: the 0 to 10 response scale, and the anchoring and fatigue risk of six repeated administrations. One respondent's data is consistent with exactly that: thirty-five of P6's thirty-six workload ratings are the same number. No mitigation was available for either.

The sharpest construct gap concerns phase gates: the item A4/task-8 cross-read described in Section 9.3.3, this study's only planned evidence on whether a gate reads as a guardrail or as obstruction, did not survive task 8's redesign away from a refusal test (Section 9.3.2). The gap is therefore worse than the absence of an observer check: there was no guaranteed refusal trigger for any of the thirteen participants, so gate comprehension is unmeasured rather than weakly measured, the framework's most distinctive mechanism the one this evaluation has least evidence about.

B - Internal validity. The author designed both artefacts, wrote the task script and the seventeen UX items, built the seeded demonstration project, and conducted the interviews. The unmoderated design removes that conflict from the sessions structurally (Section 9.3), the strongest mitigation available, but removes nothing from the instruments themselves. Two incidents make the residue concrete rather than hypothetical. A configuration defect let an interface-language setting leak between projects, so a nominally English session was answered by the model in Portuguese, and it was noticed only because the author recognised the symptom in a participant's free text and root-caused it himself. A second defect, in which Phase 5 read a stale repository-context file as current and reported at high confidence, was found during the first session and was still present for every subsequent participant. In both cases the person best placed to detect and interpret the anomaly was the artefact's designer.

The same window produced a second internal threat: the system was not frozen. One defect fix was deployed during data collection and one configuration change followed it, so participants did not all use a byte-identical build. Against that, the project environment was reset to an identical seeded state before every session (Section 9.3.2).

C - External validity. The sample (Section 9.2) is a convenience sample from one person's professional network, skewed junior, with no open call and no recorded count of those approached. It supports a description of how these thirteen people experienced the tool and no claim about experienced practitioners, teams or organisations. The pre-registered concern about a single organisational setting turns out to apply in a different place than expected: the participants are not from one organisation, but the *task* environment is a single seeded project on one project-management tool, which ten of the thirteen

had never used, so unfamiliarity with that board is confounded with unfamiliarity with Apex throughout. The observation window is one session of roughly an hour, which precludes any claim about long-term maintainability, adoption or organisational change; one participant makes the point himself, expecting that “with regular use, I believe the user would become very familiar with the interface” (P12), which is a hypothesis this design cannot test. The Portuguese figures rest on three respondents administered an instrument whose own validation reports weak inter-rater agreement, and are reported as indicative only.

D - Conclusion validity. Thirteen participants spread over five levels of prior AI exposure cannot support inferential statistics or subgroup comparison, and none is computed. Two administration artefacts show what an unmoderated design cannot catch as it happens: one participant declared Portuguese as the session language and then completed every instrument in English, and another declared English but submitted the Portuguese consent form. Neither was correctable at the time because nobody was present to correct it.

E - The cost of recording nothing. Two threats follow from the decision not to observe or record, and both are consequences of a deliberate design rather than oversights. First, there is no observational evidence of any kind: no measured completion rate, no time on task beyond a form-submission timestamp that cannot distinguish task difficulty from an interruption, no assist or error counts, and no independent check on comprehension anywhere. This leaves the perception-versus-quality threat standing rather than partly answered, and it is why the trust-calibration pair in Section 9.4.3 can report that trust and review effort did not diverge without being able to say whether any individual accepted output unread. Second, the interview analysis rests on contemporaneous written notes rather than transcripts (Section 9.2), and the selection was made by the person who designed the artefact under discussion; Section 9.3.4 names which interviewee raised which point precisely so that the selection is at least attributable.

9.6 Discussion

The instruments do not tell one story, and the places they part company are more informative than the places they coincide.

The first apparent disagreement is between SUS and the UX questionnaire. A grade D mean of 56.73 sits alongside medians of 5 on being able to reject generated output, on confidence that a step could be undone, and on the acceptability of waiting for generation. That is only a contradiction if both are read as verdicts on the same object: SUS asks about the system as a whole, three of its items specifically about complexity, about needing to learn a great deal and about needing technical support, while the UX items ask about individual mechanisms. The free text resolves the two in the same direction: nine of the thirteen answers to what was most confusing name density, orientation or navigation rather than any AI mechanism, in terms such as “too many things in the screen at the begging not sure where to click” (P1) and “too much happening at the same time” (P6). The SUS sub-scales point the same way, Learnability at 44.23 against Usability at 59.86. The reading all four sources support is that the mechanisms the framework requires are individually legible while the surface assembling them is not, and that the cost

is concentrated at first contact. This is a finding about the instantiation and, for the reason given in Section 9.1, it does not transfer to the framework.

The second cross-read is an agreement between instruments not designed to meet. Error and feedback surfacing is the weakest area the UX questionnaire identifies, D1 having the lowest closed-item median. Independently, the two heaviest tasks by workload are Design and Packs, and Packs carries the highest frustration score of any task at 26.2 despite its predicted moderate load (Section 9.4.2). A participant's free text supplies the mechanism connecting them, describing a mistake in one stage which "Apex did not clearly show what the error was" and then completing the following stages correctly but on that incorrect assumption (P8). A low D1 is not a cosmetic complaint about message wording: it describes a state in which a gate passes over an error that was never surfaced.

The third is the one place an instrument, a free-text pattern and an independently recorded defect converge. Task 8's Performance score, the highest of any task at 21.5 (Section 9.4.2), means participants rated their own deployment success furthest from *Perfect*. Four participants' free text centres on the same surface, Phase 5's pre-flight reasoning about the repository, from three different angles: one read the "infra changes required" verdict as unexpected friction on a demonstration project, and two named the absence of a prior deployment as the resolving fact, which is the framework's own documented bootstrap rule rather than a fault. Recorded separately, and not by the instruments, is the defect described in Section 9.5: that check read a stale context file as current and reported at high confidence. Three sources of evidence, one of which is not a questionnaire, point at the same mechanism, which is why Table 9.4 rates governance *partially met* rather than *met*.

Where the instruments genuinely disagree, they do so about trust, and the disagreement is not resolvable within this design. B4 and B2 both return a median of 4. One participant's free text reports the uncalibrated state directly, having trusted completely and not reviewed in detail. The questionnaire cannot say whether that participant was alone or typical, because task 4's key observation was blanket acceptance of generated output and, nothing having been observed, no data on it exists. The honest position is that the instrument found no divergence and the study is not capable of finding one.

Finally, a disagreement running opposite to the one usually anticipated. The expected tension is a favourable usability score alongside interview scepticism about process overhead; here the tool score is below average while the framework's reception in the interviews is broadly favourable, including an unprompted endorsement of its central bet and an expert reading the phase and hat structure as a genuine departure from closed-role methodology (Section 9.3.4). As Section 9.1 already establishes, that differential is not evidence against the method.

9.6.1 Answering the Research Questions

A - DRQ1, how can AI-supported activities be mapped to SDLC phases so that the point, purpose and boundary of AI participation are unambiguous? They can be, at phase granularity, and Table 6.3 is the answer: every phase names what AI may contribute, what it must produce, which hat is accountable, and the control that must be satisfied before exit. Two independent checks support it rather than the author's assertion. The decomposition survived instantiation with no phase merged, split or skipped, the alignment criterion rated *met* in Table 9.4; and an expert who had not designed the framework read it back in interview, comparing its hats and its refusal to bind roles to individuals against the closed-role structures of organisational methodology (Section 9.3.4). The qualification is the clarity

rating: boundaries are unambiguous at phase level, and at least one construct carried a precondition the framework never stated, which only building it exposed.

B - DRQ2, which governance mechanisms, quality gates and human-in-the-loop practices keep AI contributions traceable, validated and accountable? Those named in Sections 6.6 and 6.6.3, and the evidence separates them by strength. Traceability and validation are the strongest: gates are enforced against specification content rather than against a signature, stable identifiers make every artefact resolve to a requirement, and generated claims about coverage are re-checked server-side because self-reporting was found untrustworthy in practice (Section 7.6). Human-in-the-loop support is rated *met* on the structural test, no phase being exitable by generation alone, and participants independently confirm its preconditions (Table 9.4, Table B.5). Accountability is the weaker half and is rated *partially met*: the framework's own measure of AI-attributable defects is instantiable only as a proxy, and a governance surface was observed reporting confidently on stale evidence. The mechanisms make contributions traceable and reviewable; they do not yet make the consequences of a contribution measurable.

C - DRQ3, to what extent can those constructs be operationalised in a working system, and which resist it? Of the twelve constructs mapped in Table 7.1, ten are realised as specified, one exists only as a named proxy, and one is partially realised with the resisting part identified. That count is the smaller part of the answer; the larger part is what the two exceptions revealed. Bolt Cycle Time was implemented, displayed and believed while measuring something else, caught only by comparing the implementation against its written definition (Section 7.3). Spec-drift detection failed because the framework assumes a per-story specification granularity it nowhere states, so the instantiation discovered an omission in the method rather than a defect in the tool. Both are reported as findings, and the existence of Apex on its own is not offered as an answer to this question.

That the ten realised constructs hold under real use, not only at the scale of a unit test, is the part the construct mapping alone cannot show and the demonstration supplies (Section 9.4.5). Carried across a fifty-story project, Context Traceability Rate resolved for 50 of 54 deployed stories with its one failure mode traced to a single cause, and Spec Conformance's finding-level detail showed one shared-component defect detected consistently across nine stories rather than missed in any of them, which is the behaviour Spec-Anchored Continuity is meant to produce rather than a coincidence of the sample. The same evidence answers the question in the other direction. The Testing Gate, a realised construct by Table 7.1's own count, was shown passable by a suite that had stopped measuring the system it tested, and the same stale-context failure recurred twice more (Section 9.4.5); realised, on this evidence, does not yet mean reliably realised.

D - DRQ4, does the framework, through that implementation, give practitioners usable and cognitively sustainable support in a real setting? The honest answer is split, and the two halves do not contradict. On cognitive sustainability the evidence is favourable: a grand workload aggregate of 21.5 out of 100, frustration among the lowest subscales at 16.7, and a heaviest single task at 30.4. The burden is concentrated in mental demand and falls on the phases with the most generated content to review, which is where the framework intends to put it. On usability it is not favourable: a SUS mean of 56.73, grade D, with one participant at 15 and a Portuguese subgroup mean of 44.17 at grade F; error

legibility is the weakest UX item; and nine of thirteen named density or orientation as the most confusing thing about the tool. The two readings sit together because they measure different things: the tool asks little of the practitioner once they know where they are, and does too little to get them there. Both halves are claims about the instantiation, at one session of about an hour, with the sample limits stated in Section 9.5; the framework's own component of DRQ4 rests on the demonstration in Chapter 8. The demonstration carries no equivalent instrument and answers a different question, sustained solo operation across nine days rather than one measured session, but its result is consistent with the orientation difficulty the SUS and UX data name: the same governance surfaces nine of thirteen participants found confusing in an hour were, over nine days of real use, the ones found reporting confidently on evidence that was no longer current (Section 9.4.5).

10

Conclusion

Contents

10.1 Summary and Contributions	81
10.2 Communication	83
10.3 Limitations	84
10.4 Future Work	85
Declaration of Generative AI and AI-Assisted Technologies in the Writing Process	87

10.1 Summary and Contributions

This dissertation began from a problem that is methodological rather than technical. AI assistance is now present in every phase of the SDLC, adoption is close to universal and trust is not, and the evidence assembled in Chapter 1 and characterised systematically in Chapter 4 shows the value of that assistance to be conditional on the process surrounding it rather than a property of the model: functional correctness and security do not improve together, measured productivity and perceived productivity diverge, and at organisational scale output rises while delivery outcomes do not. Established delivery methodologies do not close that gap, having been designed for lifecycles in which every contributor is human, and the approaches proposed to replace them remain fragmented across isolated tools with no common position on lifecycle integration, oversight or evaluation. The research problem formalised in Chapter 5 follows directly: the absence of structured, repeatable guidance and standardised evaluation practice for integrating AI across the SDLC while preserving human oversight, software quality and security, and accountability.

Two artefacts were produced against that problem, and the distinction between them is load-bearing rather than presentational. The first, specified in Chapter 6, is a process framework: six lifecycle phases preceded by a Strategic Alignment stage, each fixing what the model may contribute, what must leave the phase, which hat is accountable and which control must be satisfied before exit (Table 6.3); a governance set comprising Bolts as the unit of execution, the Consistency Factor, Spec-Anchored Continuity, five quality gates and three governance metrics (Section 6.6); responsibility expressed as functions rather than positions, so that accountability survives a team of one; and three named loop-backs out of Maintenance, each re-entering a phase whose gate then applies in full. The second, presented in Chapter 7, is Apex, a working system instantiating those constructs, built not as the contribution itself but as evidence that the constructs are implementable and as the vehicle through which the framework could be demonstrated and evaluated.

What the evaluation established is set out in Chapters 8 and 9 and is deliberately of two kinds. The framework was carried through a complete product over nine calendar days, growing from a pre-committed six epics to nine delivered, fifty stories and fifty-four deployed passages through all six phases, with all three loop-back routes exercised more than once each and every metric resolving to a checkable number rather than a placeholder (Section 9.4.5); it was assessed analytically against five criteria fixed before the results were seen, two rated met and three partially met with the shortfall named in each case (Table 9.4); and it was put to five practitioners and one Agile and Scrum expert in semi-structured interviews (Section 9.3.4). The instantiation was measured separately, through a fixed nine-task script administered unmoderated to thirteen participants, returning a SUS mean of 56.73 and a Raw NASA-TLX grand aggregate of 21.5 out of 100. That separation is itself part of the result: a usability score for the tool is not evidence that the method is sound, and the evaluation was designed so that neither result could be quietly borrowed for the other.

The four research questions of Chapter 1 are answered in Section 9.6.1 against that evidence, and what follows summarises those answers rather than re-deriving them. DRQ1, on mapping AI-supported activities to SDLC phases so that the point, purpose and boundary of AI participation are unambiguous, is answered affirmatively at phase granularity: the decomposition survived instantiation with no phase merged, split or skipped, and was read back in interview by an expert who had not designed it. The qualification is that unambiguity holds at phase level and not everywhere below it, since at least one construct carried a precondition the framework never stated. DRQ2, on the governance mechanisms, quality gates and human-in-the-loop practices that keep AI contributions traceable, validated and accountable, is answered by the set named in Sections 6.6 and 6.6.3, with the evidence separating those mechanisms by strength: traceability and validation are the strongest; human-in-the-loop support is met on the structural test, no phase being exitable by generation alone; and accountability is the weaker half, because the framework's own measure of AI-attributable defects is instantiable only as a proxy and a governance surface was observed reporting confidently from stale evidence.

DRQ3, on how far those constructs can be operationalised and which resist it, is answered by the construct-by-construct mapping in Table 7.1: of twelve constructs, ten are realised as specified, one exists only as a named proxy and one is partially realised with the resisting part identified: one metric was implemented, displayed and believed while measuring something else, caught only by comparing the implementation against its written definition, and that spec-drift detection failed because the framework assumes a specification granularity it nowhere states, so the instantiation discovered an omission in the

method rather than a defect in the tool. The demonstration then showed that realised does not yet mean reliably realised: the Testing Gate was found passable by a suite that had stopped measuring the system it tested, and no gate in Table 6.5 is defined at the boundary a destructive command issued inside a phase crosses. DRQ4, on whether the framework, through that implementation, gives practitioners usable and cognitively sustainable support, is answered split and the two halves do not contradict. On cognitive sustainability the evidence is favourable, with workload concentrated in mental demand and falling on the phases carrying the most generated content to review, which is where the framework intends to put it. On usability it is not: a grade D SUS mean below the published corpus, error legibility the weakest questionnaire item, and nine of thirteen participants naming density or orientation as the most confusing thing about the tool. The tool asks little of a practitioner who knows where they are and does too little to get them there.

The contributions are therefore the four stated in Chapter 1, now discharged rather than promised: the synthesis of the evidence on AI participation in the SDLC and the systematic review supporting it; the framework itself, treating AI as a first-class collaborator with phases, gates, function-based responsibilities, metrics and a versioned specification anchoring every artefact to the requirement it derives from; Apex, which shows those constructs implementable rather than merely describable, and which reports honestly on the two that were not; and an empirical evaluation separating what can be claimed about the method from what can be claimed about the tool. What the work does not claim is equally definite. No evidence here establishes that the framework improves delivered software quality, because no instrument in this study measures delivered quality, and the strongest results reported are perception measures over one session and one designer-operated project.

10.2 Communication

This section addresses the Communication activity of the DSRM.

Dissemination follows three vehicles, each addressing a different audience and each at a different stage. The first is a scientific article reporting the SLR of Chapter 4, covering its protocol, the attrition from 646 retrieved articles to the 31 retained for analysis, and the five review questions it answers; the review is the component of this work that stands independently of the artefacts and is therefore the component most directly useful to other researchers. A draft exists, but submission was deprioritised in favour of completing this dissertation, and the target venue remains to be finalised.

The second is the extended abstract accompanying this dissertation, written as a self-contained technical article presenting the framework, its instantiation and the results of its evaluation to a reader who will not read the full document. It is submitted alongside this dissertation and is assessed as part of it.

The third is this dissertation itself, presented and defended before an academic examination committee. The defence is the point at which the artefacts are exposed to structured, adversarial examination by reviewers who did not design them, which no instrument in Chapter 9 supplies: none of its participants or interviewees was positioned to interrogate the framework's design decisions against the alternatives engaged in Section 6.9. The defence is reported here as part of the DSRM Communication activity for that reason and not merely as an academic formality.

10.3 Limitations

The limitations below concern the artefacts and the research as a whole, kept separate from Section 9.5, which concerns what the evaluation's design permits to be concluded from its data: a study can be internally sound and still be a study of something provisional.

- **The framework's core constructs are an original synthesis, not an adaptation of a single validated source.** The literature grounds individual pieces, the Bolt, the Unit of Work and Mob Elaboration from AI-DLC [42], the Trio's structural precedent from the Spotify model and Team Topologies [43, 46], Spec-Anchored Continuity from spec-driven development [40, 41], but no published source offers the joint blueprint assembled here, and the assembly itself has been evaluated only by the means in Chapter 9 (Section 6.9). Two unstated premises are already established, not hypothesised: AI Defect Escape Rate is not instantiable at the observation point its own definition presumes, since nothing says who owns production telemetry for generated logic, so only the weaker linked-post-deployment-issue proxy is realisable (Section 7.7); and spec-drift detection presumes a per-story specification granularity the framework nowhere states, which remains open for the technical specification. The demonstration added three findings of the same kind to the two already named in Section 10.1's DRQ2 and DRQ3 answers, the destructive-command boundary gap and the stale-instrument gate: the sharpest is that an advisory governance surface can report high confidence while wrong in either direction, with nothing distinguishing that case from being right. Whether an in-phase control can be defined without abandoning the gate model is left open.
- **Several grounding sources are recent, vendor-reported, or not yet peer-reviewed,** identified as such at every point of use rather than laundered by citation. AI-DLC's productivity claims are vendor-reported and unreplicated, and its vocabulary is adopted while explicitly declining to carry its results [42]. The principal spec-driven-development source describes the practice as emerging with unsettled scaling behaviour [40], and the strongest independent argument for specification discipline as the binding constraint on dependability is a preprint [41]. Much of Chapter 1's empirical case is industry telemetry and survey data rather than peer-reviewed study [6, 7, 9, 12], and the most-cited favourable productivity figure in the field was never peer-reviewed [10]. The argument rests on consistency of direction across independent sources, not the magnitude of any one figure.
- **The vocabulary borrowed from AI governance standards is shared language and nothing more.** Section 6.6.4 maps its principles, phases, metrics and gates onto the NIST AI Risk Management Framework's functions [48], with comparable vocabulary in ISO/IEC 42001:2023 [49]. Neither the framework nor Apex has been audited against either standard, no certification was sought, and nothing here asserts compliance with either: the claim is about where the work sits, not what it satisfies.
- **Everything reported concerns early adoption in a single setting.** The demonstration is nine days of solo operation by the person who designed both artefacts, one project, one technology stack; the task-based study is one session of roughly an hour per participant, one seeded project, one project management tool, a convenience sample from a single professional network skewed junior; the third planned demonstration setting, an independent organisation, did not run. Nothing in that evidence

base can speak to sustained use: whether gates stay exercised once the process's novelty wears off, whether the specification remains the source of truth after several amendments, whether Bolt Cycle Time and Context Traceability Rate stay informative once reported upward to someone with an interest in their value, and whether the framework survives contact with an organisation that did not choose it are all open, and none is answered here.

10.4 Future Work

The directions below are derived from specific findings in this dissertation rather than assembled as a list of plausible extensions; each bullet names the finding that motivates it.

- **Application across multiple organisations and team sizes.** The framework's accountability-by-function design is meant to survive a team of one (Section 6.5), but the only end-to-end operation of it so far *is* that limiting case: one developer holding every hat (Chapter 8). The evaluation's thirteen participants ran a fixed script on a seeded project rather than their own work, and the weakest-supported UX item was exactly the one asking whether the process fits a real team (Table B.5). Needed: application in organisations that did not design the framework, with hats actually distributed across people.
- **Support for project management backends beyond the first-class one.** Adding a second and third adapter (Section 7.2) showed the abstraction holds at the level of operations and fails at the level of guarantees, with three of the second tool's limitations permanent rather than unfinished. Both the demonstration and the evaluation still ran only against the first-class tool, so the parity claim is architectural, not evidenced. Needed: a demonstration against a different backend, which would also separate unfamiliarity with the tool from unfamiliarity with the board (Section 9.5).
- **Replacing proxy metrics with direct measurement.** AI Defect Escape Rate's proxy only sees defects someone later linked to a story on the board; nineteen Fix-Bolts were logged across fourteen stories while the proxy itself registered one (Section 8.4). Section 6.6.4 assigns this metric the job of catching a gate that has gone ceremonial, a job a same-session-blind proxy cannot do. Needed: direct measurement from production telemetry, once the framework says who owns it for generated logic specifically.
- **Operationalising governance metrics for AI token and compute cost.** An Agile/Scrum expert interviewed for this work (Section 9.3.4) described an organisation that centrally governs and meters AI tool use with no role owning that governance, and asked, unprompted, whether the collaboration nets out as cost or benefit. The framework can currently answer that question for defect escape but not for cost (Section 6.6.4). Three concrete extensions, proposed the way the framework proposes its other constructs:
 - *A fourth governance metric, **AI Token and Compute Cost**, defined at Bolt granularity like Bolt Cycle Time (Section 6.6.4) so it composes with the existing three rather than sitting beside them: cost against Bolt Cycle Time prices flow, cost against AI Defect Escape Rate prices the defects that escaped anyway. Its observation point must be provider usage reporting at the point of*

invocation, named from the outset rather than discovered missing later, the way AI Defect Escape Rate's was (Section 7.7).

- *A named accountable hat for AI cost governance*, plus a general mechanism for detecting an unstaffed hat. The demonstration showed the failure live: the Security Reviewer hat went unexercised through an entire Fix-Bolt cascade with nothing detecting it (Section 8.4). A gate whose evidence names the function that examined it, with a signal when that function never acted, would serve both cases at once.
- *Risk- and phase-tiered model selection*, extending the framework's own Risk-Proportional principle, already operational in the Fast Lane/Secure Lane split (Table 6.2), to a policy Table 6.3 does not yet fix.
- **Support for the Design phase specifically**, the point the evidence marks hardest to get right. Task 4 (produce and lock the design) carries the study's highest workload on every subscale but frustration and performance: mental demand 47.7 against a next-highest of 26.9, temporal demand 40.8, aggregate 30.4 (Table 9.2). The asymmetry is visible in the gates themselves: Testing checks generated scripts against a locked scenario that already exists; Design produces the first concrete artefact from an epic-level intent, with no earlier phase's lock to check against (Table 6.5). Needed, not a heavier gate but a lighter one: worked examples anchoring what an adequate specification looks like for a given story type, and interim structured self-critique (usability, scenario coherence, architectural fit) before the human reviews the whole artefact at once.
- **Redesigning the tool's interface for first-contact adoption, not sustained use.** SUS Usability averages 59.86 against Learnability's 44.23, a full grade lower and far more dispersed (0-100 versus 12.50-87.50, Section 9.4.1): the difficulty concentrates at first contact rather than spreading through use. Corroborated twice over: the density/orientation confusion already named in Section 10.1's DRQ4 answer above, and error legibility as the weakest individual UX item, alongside mental demand as the largest NASA-TLX subscale overall (Section 9.4.3, Table 9.2). The Portuguese subgroup's 44.17 against the English subgroup's 60.50 points the same way, though $n = 3$ cannot carry it alone. This is a direct barrier to the adoption the first future-work item above calls for, and a redesign aimed specifically at orientation and error legibility, not the tool broadly, is what the data argues for.

Declaration of Generative AI and AI-Assisted Technologies in the Writing Process

During the preparation of this work, the author used Claude Code, with the Sonnet 5, Opus 5 and Fable 5 models, together with Google Gemini and Gemini Notebook, to improve the wording and clarity of the manuscript and to support the organisation and review of source material. After using these tools, the author reviewed and edited the text as needed and takes full responsibility for the content of the publication. The final manuscript reflects the author's own analysis, results, and conclusions.

This declaration concerns the use of these technologies in the writing process only. The use of generative AI as the object of study of this research, and within the artefacts it produces, is described in Chapter 6 and Chapter 7.

Bibliography

- [1] J. H. Moore and N. Tatonetti, “Vibe coding: a new paradigm for biomedical software development,” *BioData Mining*, vol. 18, no. 1, pp. 1–3, 2025.
- [2] A. Zamfiroiu, C.-B. Păștinică, C. Crăciun, C. Ciutacu, and T. Luca, “Exploring Conversational Programming through the CHOP Paradigm and Artificial Intelligence,” *Informatica Economica*, vol. 29, no. 2, pp. 5–17, 2025.
- [3] I. Ahmed, A. Aleti, H. Cai, A. Chatzigeorgiou, P. He, X. Hu, M. Pezzè, D. Poshyvanyk, and X. Xia, “Artificial Intelligence for Software Engineering: The Journey So Far and the Road Ahead,” *ACM Transactions on Software Engineering & Methodology*, vol. 34, no. 5, pp. 1–27, 2025.
- [4] S. Zhang, Z. Xing, R. Guo, F. Xu, L. Chen, Z. Zhang, X. Zhang, Z. Feng, and Z. Zhuang, “Empowering Agile-Based Generative Software Development through Human-AI Teamwork,” *ACM Transactions on Software Engineering & Methodology*, vol. 34, no. 6, pp. 1–46, 2025.
- [5] W. Hou and Z. Ji, “Comparing Large Language Models and Human Programmers for Generating Programming Code,” *Advanced Science*, vol. 12, no. 8, pp. 1–12, 2025.
- [6] Stack Overflow, “2025 Developer Survey: AI,” Annual practitioner survey, industry-reported, not peer-reviewed, 2025. [Online]. Available: <https://survey.stackoverflow.co/2025/ai/>
- [7] DORA and Google Cloud, “State of AI-assisted Software Development 2025,” Industry research report, not peer-reviewed, 2025. [Online]. Available: <https://dora.dev/dora-report-2025/>
- [8] S. Zhao, D. Wang, K. Zhang, J. Luo, Z. Li, and L. Li, “Is Vibe Coding Safe? Benchmarking Vulnerability of Agent-Generated Code in Real-World Tasks,” arXiv:2512.03262, preprint, not yet peer-reviewed, 2025. [Online]. Available: <https://arxiv.org/abs/2512.03262>
- [9] Veracode, “2025 GenAI Code Security Report: Assessing the Security of Using LLMs for Coding,” Industry report, not peer-reviewed. Assessment of over 100 large language models across Java, Python, C# and JavaScript, July 2025. [Online]. Available: https://www.veracode.com/wp-content/uploads/2025_GenAI_Code_Security_Report_Final.pdf
- [10] S. Peng, E. Kalliamvakou, P. Cihon, and M. Demirer, “The Impact of AI on Developer Productivity: Evidence from GitHub Copilot,” arXiv:2302.06590, preprint, not published in a peer-reviewed venue, 2023. [Online]. Available: <https://arxiv.org/abs/2302.06590>

- [11] J. Becker, N. Rush, E. Barnes, and D. Rein, "Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity," arXiv:2507.09089, METR, preprint, not yet peer-reviewed, 2025. [Online]. Available: <https://arxiv.org/abs/2507.09089>
- [12] Faros AI, "The AI Productivity Paradox Report 2025," Industry telemetry report, not peer-reviewed. Sample of over 10,000 developers across 1,255 teams, July 2025. [Online]. Available: <https://www.faros.ai/blog/ai-software-engineering>
- [13] L. Banh, F. Holldack, and G. Strobel, "Copiloting the future: How generative AI transforms Software Engineering," *Information and Software Technology*, vol. 183, 2025.
- [14] M. Farhanna, N. Paramitha, M. Susi, H. Surya, A. Yanti, P. Dea, and S. Deni, "Exploring the Use of Generative AI in Software Development: A Preliminary Study," in *E3S Web of Conferences*, vol. 645, 2025, p. 04002.
- [15] A. Hemmat, M. Sharbaf, S. Kolaheidouz-Rahimi, K. Lano, and S. Y. Tehrani, "Research directions for using LLM in software requirement engineering: a systematic review," *Frontiers in Computer Science*, vol. 7, 2025.
- [16] C. Gao, X. Hu, S. Gao, X. Xia, and Z. Jin, "The Current Challenges of Software Engineering in the Era of Large Language Models," *ACM Transactions on Software Engineering & Methodology*, vol. 34, no. 5, pp. 1–30, 2025.
- [17] H. A. Al-Hashimi, R. A. Khan, H. S. Alwageed, A. M. Algarni, S. Ayouni, and A. O. Almagrabi, "Exploring the role of generative AI in enhancing cybersecurity in software development life cycle," *Array*, vol. 28, p. 100509, 2025.
- [18] X. Bai, S. Huang, C. Wei, and R. Wang, "Collaboration between intelligent agents and large language models: A novel approach for enhancing code generation capability," *Expert Systems With Applications*, vol. 269, 2025.
- [19] M. Basha and G. Rodríguez-Pérez, "Trust, transparency, and adoption in generative AI for software engineering: insights from twitter discourse," *Information and Software Technology*, 2025.
- [20] X. Chen, C. Gao, C. Chen, G. Zhang, and Y. Liu, "An Empirical Study on Challenges for LLM Application Developers," *ACM Transactions on Software Engineering & Methodology*, vol. 34, no. 7, pp. 1–37, 2025.
- [21] Y. Dong, X. Jiang, Z. Jin, and G. Li, "Self-Collaboration Code Generation via ChatGPT," *ACM Transactions on Software Engineering & Methodology*, vol. 33, no. 7, pp. 1–38, 2024.
- [22] U. K. Durrani, M. Akpınar, M. F. Adak, A. T. Kabakus, M. M. Ozturk, and M. Saleh, "A Decade of Progress: A Systematic Literature Review on the Integration of AI in Software Engineering Phases and Activities (2013-2023)," *IEEE Access*, vol. 12, pp. 171 185–171 204, 2024.
- [23] Y. Han and C. Lyu, "Multi-stage guided code generation for Large Language Models," *Engineering Applications of Artificial Intelligence*, vol. 139, 2025.

- [24] M. A. Haque, "LLMs: A game-changer for software engineers?" *BenchCouncil Transactions on Benchmarks, Standards & Evaluations*, vol. 5, no. 1, pp. 30–41, 2025.
- [25] J. R. V. Jan Pries-Heje, R. Baskerville, "Strategies for Design Science Research Evaluation," *European Conference on Information Systems (ECIS)*, no. 1, pp. 1–13, 2008.
- [26] S. K. Jabrw and Q. I. Sarhan, "A Systematic Survey on Large Language Models for Code Generation," *ARO-The Scientific Journal of Koya University*, vol. 13, no. 2, 2025.
- [27] V. V. Jensen, A. Alami, A. R. Bruun, and J. S. Persson, "Managing expectations towards AI tools for software development: a multiple-case study," *Information Systems & e-Business Management*, pp. 1–33, 2025.
- [28] U. Karlovs-Karlovskis, "Generative Artificial Intelligence Use in Optimising Software Engineering Process: A Systematic Literature Review," *Applied Computer Systems*, vol. 29, no. 1, pp. 68–77, 2024.
- [29] D.-K. Kim and H. Ming, "Assessing output reliability and similarity of large language models in software development: A comparative case study approach," *Information and Software Technology*, vol. 185, 2025.
- [30] D.-K. Kim, "Improving Software Development Traceability with Structured Prompting," *Journal of Computer Information Systems*, pp. 1–21, 2025.
- [31] J. Li, G. Li, Y. Li, and Z. Jin, "Structured Chain-of-Thought Prompting for Code Generation," *ACM Transactions on Software Engineering & Methodology*, vol. 34, no. 2, pp. 1–23, 2025.
- [32] V. S. Pendyala and N. B. Thakur, "Performance and interpretability analysis of code generation large language models," *Neurocomputing*, vol. 656, 2025.
- [33] K. Qiu, N. Puccinelli, M. Ciniselli, and L. Di Grazia, "From Today's Code to Tomorrow's Symphony: The AI Transformation of Developer's Routine by 2030," *ACM Transactions on Software Engineering & Methodology*, vol. 34, no. 5, pp. 1–17, 2025.
- [34] T. Ramos, A. Dean, and D. McGregor, "AI-Augmented Software Engineering: Revolutionizing or Challenging Software Quality and Testing?" *Journal of Software: Evolution & Process*, vol. 37, no. 2, pp. 1–7, 2025.
- [35] V. Terragni, A. Vella, P. Roop, and K. Blincoe, "The Future of AI-Driven Software Engineering," *ACM Transactions on Software Engineering & Methodology*, vol. 34, no. 5, pp. 1–20, 2025.
- [36] A. Tkachuk, "Determining the Capabilities of Generative Artificial Intelligence Tools to Increase the Efficiency of Refactoring Process," *Technology Audit & Production Reserves*, vol. 3, no. 2(83), pp. 6–11, 2025.
- [37] Q. Wang, J. Wang, M. Li, Y. Wang, and Z. Liu, "A Roadmap for Software Testing in Open-Collaborative and AI-Powered Era," *ACM Transactions on Software Engineering & Methodology*, vol. 34, no. 5, pp. 1–17, 2025.

- [38] M. Weyssow, X. Zhou, K. Kim, D. Lo, and H. Sahraoui, "Exploring Parameter-Efficient Fine-Tuning Techniques for Code Generation with Large Language Models," *ACM Transactions on Software Engineering & Methodology*, vol. 34, no. 7, pp. 1–25, 2025.
- [39] S. H. Yoo and H. J. Kim, "Security Analysis of Automated Code Generation: Structural Vulnerabilities in AI-Generated Code," *Tehnički Glasnik*, vol. 19, no. 4, pp. 560–574, 2025.
- [40] Thoughtworks, "Spec-driven development," Technology Radar, Vol. 33, November 2025. [Online]. Available: <https://www.thoughtworks.com/en-us/radar/techniques/spec-driven-development>
- [41] S. E. Farrag, "The Productivity-Reliability Paradox: Specification-Driven Governance for AI-Augmented Software Development," arXiv:2605.01160, preprint, not yet peer-reviewed, 2026. [Online]. Available: <https://arxiv.org/abs/2605.01160>
- [42] Amazon Web Services, "AI-Driven Development Lifecycle (AI-DLC)," Vendor methodology announced at AWS DevSphere 2025. Reported productivity figures are vendor-reported and not independently verified, 2025. [Online]. Available: <https://aws.amazon.com/blogs/devops/ai-driven-development-life-cycle/>
- [43] H. Kniberg and A. Ivarsson, "Scaling Agile @ Spotify with Tribes, Squads, Chapters & Guilds," Whitepaper, industry-reported, not peer-reviewed. A description of Spotify's organisation at the time of writing, not a prescriptive method, November 2012. [Online]. Available: <https://blog.crisp.se/2012/11/14/henrikkniberg/scaling-agile-at-spotify>
- [44] Leading Complexity, "Summary of Henrik Kniberg's session, about his thoughts on the Spotify Model, in the Leading Complexity programme 2022," Crisp's Blog. Third-party published summary of Henrik Kniberg's session in the Leading Complexity programme 2022, not written by Kniberg himself, February 2023. [Online]. Available: <https://blog.crisp.se/2023/02/06/leadingcomplexity/leading-complexity-series-summary-1-henrik-knibergs-thoughts-on-the-spotify-model>
- [45] J. Lee, "Failed #SquadGoals," Practitioner account by a former Spotify employee, industry-reported, not peer-reviewed, April 2020. [Online]. Available: <https://www.jeremiahlee.com/posts/failed-squad-goals/>
- [46] M. Skelton and M. Pais, *Team Topologies: Organizing Business and Technology Teams for Fast Flow*. Portland, OR: IT Revolution Press, 2019.
- [47] C. Bryar and B. Carr, *Working Backwards: Insights, Stories, and Secrets from Inside Amazon*. New York, NY: St. Martin's Press, 2021.
- [48] National Institute of Standards and Technology, "Artificial Intelligence Risk Management Framework (AI RMF 1.0)," National Institute of Standards and Technology, U.S. Department of Commerce, Technical Report NIST AI 100-1, January 2023. [Online]. Available: <https://doi.org/10.6028/NIST.AI.100-1>
- [49] ISO/IEC, "Information technology - Artificial intelligence - Management system," International Organization for Standardization/International Electrotechnical Commission, International Standard ISO/IEC 42001:2023, December 2023.

- [50] S. Siddeeq, M. Abbasi, J. Rasku, Z. Zhang, F. Christophe, T. Mikkonen, and P. Abrahamsson, "Epic-Organized vs. Requirement-Aligned Gherkin: An Empirical Evaluation of LLM-Based Acceptance Criteria Generation," Accepted at the International Conference on Software Engineering of Emerging Technologies (SEET 2026), Springer Nature. arXiv:2607.01980, 2026. [Online]. Available: <https://arxiv.org/abs/2607.01980>
- [51] N. Forsgren, M.-A. Storey, C. Maddila, T. Zimmermann, B. Houck, and J. Butler, "The SPACE of Developer Productivity: There's More to it than You Think," *ACM Queue*, vol. 19, no. 1, pp. 20–48, 2021.
- [52] J. Brooke, "SUS: A 'quick and dirty' usability scale," in *Usability Evaluation in Industry*, P. W. Jordan, B. Thomas, B. A. Weerdmeester, and I. L. McClelland, Eds. Taylor and Francis, 1996, pp. 189–194.
- [53] A. I. Martins, A. F. Rosa, A. Queirós, A. Silva, and N. P. Rocha, "European Portuguese validation of the System Usability Scale (SUS)," *Procedia Computer Science*, vol. 67, pp. 293–300, 2015. [Online]. Available: <https://doi.org/10.1016/j.procs.2015.09.273>
- [54] A. Bangor, P. Kortum, and J. Miller, "Determining what individual SUS scores mean: Adding an adjective rating scale," *Journal of Usability Studies*, vol. 4, no. 3, pp. 114–123, 2009.
- [55] J. Sauro and J. R. Lewis, *Quantifying the User Experience: Practical Statistics for User Research*, 2nd ed. Morgan Kaufmann, 2016.
- [56] J. R. Lewis and J. Sauro, "The factor structure of the System Usability Scale," in *Human Centered Design, HCII 2009, Lecture Notes in Computer Science*, vol. 5619, 2009, pp. 94–103.
- [57] S. G. Hart and L. E. Staveland, "Development of NASA-TLX (Task Load Index): Results of empirical and theoretical research," in *Human Mental Workload*, P. A. Hancock and N. Meshkati, Eds. North-Holland, 1988, pp. 139–183.
- [58] S. G. Hart, "NASA-Task Load Index (NASA-TLX); 20 years later," in *Proceedings of the Human Factors and Ergonomics Society Annual Meeting*, vol. 50, 2006, pp. 904–908.



Extended Results for the Systematic Literature Review

This appendix holds the evidence tables underlying the review reported in Chapter 4. They are reproduced here rather than in the chapter itself so that the chapter can state what the review found without the reader having to read four tabulations to reach it, while the tabulations remain available in full for anyone auditing the synthesis. Kitchenham's guidelines require the review to be reconstructible from the protocol together with this material.

A.1 Stakeholder opinions reported in the literature

The fourth research question is answered in Chapter 4 from the three tables below, each separating the perspectives of one of the groups named in its caption.

Table A.1: Public / Researchers Opinions on AI and LLMs in Software Development

Opinion Category	Description	#	References
General Trust / Perception	People's perception of systems powered by artificial intelligence as a mixture of "software and sorcery." [3]	6	[2, 3, 18, 19, 32, 34]
Misinformation Risk	"Users highlight its potential to misguide or hallucinate to students." [19]	4	[13, 19, 22, 26]
Bias and Impartiality Concerns	Users express dissatisfaction over perceived "leftist bias and propaganda in ChatGPT." [19]	3	[3, 19, 34]

Continued on next page

Opinion Category	Description	#	References
Output Diversity / Similarity	"57% similarity overall with 43% variability, presenting that each model has its unique characteristics." [29]	2	[4, 29]
Legal, Copyright, and Licensing Issues	"Questions also surround the copyrightability/ownership of generated code [...]" [3]	2	[3, 19]
Evaluation Urgency	Urgent need to "establish an effective, objective, and complete evaluation system" for code-generation LLMs. [3]	2	[3, 32]
Job Automation	"Many users fear that AI will eventually replace human developers [...]" [19]	1	[19]

Table A.2: Developers / Practitioners Opinions on AI and LLMs in Software Development

Opinion Category	Description	#	References
Productivity and Efficiency Gains	"The knowledge and reasoning capabilities of LLMs have the potential to result in significant efficiency gains." [13]	6	[2, 13, 19, 23, 24, 35]
Reliability Issues / Hallucinations	"LLMs sometimes produce highly convincing content that is factually wrong [...] the 'hallucination phenomenon'." [18]	6	[13, 18, 19, 22, 26, 32]
Output Variability and Trust Decline	"I get a different code each time [...] I lose a bit of confidence in the work." [13]	5	[13, 19, 32, 34, 35]
Quality of Generated Code	"Several quality issues: undefined variables, insecure comments, and code requiring significant revision." [26]	4	[4, 26, 27, 35]
Augmentation of Human Capabilities	"LLMs are not merely a product of overhyped marketing; they represent a profound shift in how software is engineered. They should be seen as powerful tools that augment human capabilities rather than replace them." [24]	3	[19, 24, 35]
Learning and Skill Improvement	LLMs "[...] directly support learning processes through interaction and conceptual clarification." [2]	3	[2, 16, 24]
Knowledge Acquisition / Search Replacement	"LLMs present data in a concise and contextually relevant manner." [13]	2	[2, 13]
Controllability	Participants praised its "controllability [...] allowing modification and feature adjustment after generation." [4]	2	[4, 19]
Risk of Over-reliance and Cognitive Decline	"There is almost the danger that you stop thinking for yourself [...] people will tend to become lazy." [13]	2	[13, 19]
Investment of time with prompts	"You spend ages trying to cut the prompt so that it comes out right [...]" [13]	2	[13, 19]

Table A.3: Organizations / Management Opinions on AI and LLMs in Software Development

Opinion Category	Description	#	References
Need for Human Oversight	LLM outputs "still necessitate extensive human review before implementation." [29]	7	[4, 17, 19, 29, 30, 33, 35]

Continued on next page

Opinion Category	Description	#	References
Data Security, Privacy, and Corporate Bans	"Due to concerns regarding data privacy, when considering real-world practice, most software organizations tend to avoid using commercial LLMs and would prefer to adopt open source ones with training or fine-tuning using organization-specific data." [37]	6	[13, 15, 19, 35, 37, 39]
Operational Costs and Resource Constraints	Organizations must "consider the current limitations in terms of computational power or pay close attention to energy consumption," leading them to "tend to fine-tune medium-sized models". [37]	3	[13, 15, 37]
Strategic Goal for Developer Satisfaction	"We have to be able to compete on efficiency and job satisfaction [...] It is difficult to find software developers." [27]	2	[24, 27]
Expectation Management Requirement	"SDOs must carefully manage their expectations towards AI tools." [27]	2	[19, 27]
Strategic Competitiveness	The "early adoption of LLMs in software engineering is crucial to stay competitive in this rapidly evolving landscape." [24]	1	[24]

A.2 Industry sectors adopting AI and LLMs

The evidence behind the fifth research question, answered in Chapter 4.

Table A.4: Industry sectors adopting AI and LLMs

Industry Sector	Description	#	References
Software Engineering, Software Development & Consulting Services	Software Development and Consulting is mentioned a lot, since they are the main sectors where AI and LLMs are being utilized in many different ways, such as AIOps, Cloud Management, Requirements Engineering, Software Testing, Quality Assurance, Code Generation and Development, Cybersecurity, Vulnerability Management, Refactoring, Code Optimization, Specialized Hardware, Scientific Computing and Emerging Technologies (like Blockchain/Crypto).	25	[2–4, 13–20, 22–24, 26–29, 32, 33, 35–39]
Finance, Banking, and Regulated Industries (Including Insurance)	Specialized LLMs can understand the "regulatory requirements, security needs, and performance constraints" especially in blockchain, cryptocurrency, smart contracts, and regulated environments. [24, 27, 29]	8	[4, 13, 17, 19, 24, 27, 29, 34]
Transportation and Safety-Critical Domains	The lack of transparency of LLMs is problematic, particularly in "safety-critical applications like healthcare, finance, or aerospace, where understanding the reasoning behind code is essential for ensuring security and correctness". [3]	7	[3, 15, 19, 24, 34, 37, 39]
Healthcare, Medical, and Biomedical Research	AI "accelerates the coding process by converting natural language or abstract research intent (a vibe) into functioning software modules, dramatically shortening development cycles in biomedical research and clinical application." [1].	5	[1, 4, 5, 18, 19]
Education and Training	LLMs "directly support learning processes through interaction and conceptual clarification". Students should be taught "how to specify AI, including functional, non-functional, and 'happy and unhappy path scenarios'" and learn "prompt engineering workarounds" and to "research and fact-check AI responses". [34]	5	[2, 4, 5, 15, 34]

Continued on next page

Industry Sector	Description	#	References
Legal Reasoning and Regulatory Compliance	LLMs demonstrate success across multiple fields, such as education, health-care, and legal reasoning. Specifically, prompt engineering is used to "tailor LLMs for the US legal system, enabling the model to identify relevant cases and predict judicial outcomes with greater precision". [26]	4	[17, 18, 26, 34]
Digital Ecosystems (E-Commerce and Services)	Case studies involve the development of systems such as a food order and delivery system (FODS) and a smart wallet system (SWS). LLMs are applied in contexts like Online Retailers and the service industry. [4, 29]	3	[4, 17, 29]

A.3 Final set of studies retained by the review

Table A.5 lists the thirty-one studies retained after the filtering process described in Chapter 4, with the venue of each individually so that the concentration reported there can be checked study by study. The last column states which of the five research questions listed in Table 4.1 each study was drawn on to answer, following the same assignment used to build the tables above; a study contributing to several appears once, with every question it answers listed.

Table A.5: Studies retained by the systematic literature review

Reference	Venue	Year	RQs answered
[3]	ACM Transactions on Software Engineering & Methodology	2025	1, 2, 3, 4, 5
[17]	Array	2025	1, 2, 4, 5
[18]	Expert Systems With Applications	2025	1, 2, 3, 4, 5
[13]	Information and Software Technology	2025	1, 2, 3, 4, 5
[19]	Information and Software Technology	2025	1, 2, 4, 5
[20]	ACM Transactions on Software Engineering & Methodology	2025	1, 2, 3, 5
[21]	ACM Transactions on Software Engineering & Methodology	2024	1, 3
[22]	IEEE Access	2024	1, 2, 3, 4, 5
[14]	E3S Web of Conferences	2025	1, 2, 5
[16]	ACM Transactions on Software Engineering & Methodology	2025	1, 2, 3, 4, 5
[23]	Engineering Applications of Artificial Intelligence	2025	1, 2, 3, 4, 5
[24]	BenchCouncil Transactions on Benchmarks, Standards & Evaluations	2025	1, 2, 3, 4, 5
[15]	Frontiers in Computer Science	2025	1, 2, 3, 4, 5
[5]	Advanced Science	2025	2, 5
[26]	ARO-The Scientific Journal of Koya University	2025	1, 2, 3, 4, 5
[27]	Information Systems & e-Business Management	2025	1, 2, 4, 5
[28]	Applied Computer Systems	2024	1, 2, 5
[29]	Information and Software Technology	2025	1, 2, 3, 4, 5
[30]	Journal of Computer Information Systems	2025	1, 2, 3, 4
[31]	ACM Transactions on Software Engineering & Methodology	2025	1, 2, 3
[1]	BioData Mining	2025	1, 3, 5
[32]	Neurocomputing	2025	2, 3, 4, 5
[33]	ACM Transactions on Software Engineering & Methodology	2025	1, 2, 3, 4, 5
[34]	Journal of Software: Evolution & Process	2025	1, 2, 3, 4, 5
[35]	ACM Transactions on Software Engineering & Methodology	2025	1, 2, 3, 4, 5

Continued on next page

Reference	Venue	Year	RQs answered
[36]	Technology Audit & Production Reserves	2025	1, 2, 5
[37]	ACM Transactions on Software Engineering & Methodology	2025	1, 2, 3, 4, 5
[38]	ACM Transactions on Software Engineering & Methodology	2025	2, 3, 5
[39]	Tehnički Glasnik	2025	1, 2, 4, 5
[2]	Informatica Economica	2025	1, 2, 3, 4, 5
[4]	ACM Transactions on Software Engineering & Methodology	2025	1, 2, 3, 4, 5



Evaluation Instruments

This appendix reproduces every instrument administered during the evaluation described in Chapter 9, in the order a participant encountered them: the consent and demographics form, the workload questionnaire, the usability questionnaire, the author-designed UX questionnaire, and the two interview guides used in the follow-up sessions. Participants chose English or Portuguese for the session (item G6 on the demographics form); every instrument was offered in both languages, and each is reproduced here in English, the language of this dissertation, with the source of the Portuguese wording stated alongside it. Scoring procedures, interpretation scales and results are reported in Chapter 9, not repeated here; this appendix is the instrument as administered, not its analysis.

B.1 Informed consent and demographics

Completed first, before any task or questionnaire; the ethics position it rests on is stated in Section 9.2.

Study: Evaluation of Apex, a tool supporting a process framework for human-AI collaboration in software development.

Researcher: Tomás dos Santos Taborda, MEIC-T, Instituto Superior Técnico, Universidade de Lisboa. **Supervisors:** Prof. Miguel Leitão Bignolas Mira da Silva, Hugo de Sousa.

What this involves. You will use a software tool for about 60 minutes to complete nine predefined tasks, working on your own and at your own pace. After six of those tasks you will fill in a short workload questionnaire, which takes about a minute each time. At the end of the session you will fill in two further short questionnaires, taking about 20 minutes in total. You may optionally be invited to a separate interview of 30 to 45 minutes.

What is being tested. The tool and the process behind it are being tested, not you. There is no way for you to perform badly. Difficulties you have are findings about the tool and are the most useful thing you can give us.

What is recorded. Only your answers to the questionnaires, and the times at which you submit each form. Nobody watches the session. Your screen is not recorded, your audio is not recorded, and no video is taken at any point. If you take part in the follow-up interview, that is not recorded either; the researcher takes written notes.

Anonymity. You are identified only by a code such as P3. Your name, your employer and any project details you mention are not recorded in the results. Quotations used in the dissertation are anonymised, and any detail that could identify you or your

organisation is removed or generalised.

Data handling. There are no recordings to store or delete, because none are made. Anonymised questionnaire responses and anonymised interview notes are stored, accessible only to the researcher, and are kept so that reported results can be independently recomputed. They may be reproduced in an appendix of the dissertation in anonymised form.

Voluntary participation. Participation is voluntary. You may take a break at any time, skip any task or question, or stop and withdraw entirely without giving a reason and without any consequence. If you withdraw, tell the researcher whether your data so far should be deleted or kept.

Risks. There are no anticipated risks beyond mild frustration with software.

I have read and understood the above, and I agree to take part.

- ☐ I confirm I have read and agree to the above. *(required)*
- ☐ I consent to anonymised quotations being used in the dissertation.
- ☐ I agree to be contacted about a follow-up interview.

Participant code (from your card): _____

Demographics

Filled in immediately after consent, on the same form, in the language the participant chose. The participant code is the join key back to the workload, usability and UX forms.

Table B.1: Demographics items, collected on the consent form

Item	Question	Options
G1	Years of professional software development experience	under 1 / 1-3 / 4-7 / 8-15 / over 15 / student
G2	Current role	developer / tester or QA / product owner / designer / engineering manager / student / other
G3	Frequency of use of AI coding assistants	never / tried once or twice / monthly / weekly / daily
G4	Familiarity with Gherkin or BDD	none / have seen it / use it regularly
G5	Familiarity with the project management tool used in the session	none / some / use it regularly
G6	Preferred language for the session	English / Português

B.2 Raw NASA Task Load Index

The Raw NASA-TLX [57, 58] was administered once after each of the six marked tasks (script tasks 3, 4, 6, 7, 8 and 9), identified by a required first question naming the task and a second question for the participant code. Workload was collected on a linear scale from 0 to 10 and rescaled by multiplying by ten, and the subscales were presented in the order below, placing the reversed Performance scale last rather than fourth; both departures, and the reason for each, are stated in Section 9.3.

Table B.2: Raw NASA Task Load Index subscales, in the order presented

Subscale	Description	Scale
Mental Demand	How much mental and perceptual activity was required? Was it easy or demanding, simple or complex, forgiving or exacting?	Very Low - Very High
Physical Demand	How much physical activity was required? Was the task easy or demanding, slow or brisk, restful or laborious?	Very Low - Very High
Temporal Demand	How much time pressure did you feel due to the pace of the task? Slow and leisurely or rapid and frantic?	Very Low - Very High
Effort	How hard did you have to work, mentally and physically, to accomplish your level of performance?	Very Low - Very High
Frustration Level	How insecure, discouraged, irritated, stressed and annoyed, versus secure, gratified, content and relaxed, did you feel?	Very Low - Very High
Performance (<i>reversed anchors</i>)	How successful were you in accomplishing this task's goals?	Perfect - Failure

Administered in Portuguese to participants selecting that language on G6, using the direct translations in docs/thesis/evaluation/instrument-nasa-tlx.md: same six subscales, same order, same reversed Performance anchors, differing only in wording.

B.3 System Usability Scale

The SUS [52] was administered after Raw NASA-TLX, self-completed and unwatched, scale 1 = Strongly disagree ... 5 = Strongly agree. The only adaptation from the standard wording is substituting *Apex* for *the system*. Participants selecting Portuguese on G6 were given the European Portuguese version validated by Martins et al. [53] verbatim, with the same system-name substitution; that validation's psychometric limits, and what they mean for the Portuguese figures, are reported in Section 9.3.1.

Table B.3: System Usability Scale items, as administered

#	Item
1	I think that I would like to use Apex frequently.
2	I found Apex unnecessarily complex.
3	I thought Apex was easy to use.
4	I think that I would need the support of a technical person to be able to use Apex.
5	I found the various functions in Apex were well integrated.
6	I thought there was too much inconsistency in Apex.
7	I would imagine that most people would learn to use Apex very quickly.
8	I found Apex very cumbersome to use.
9	I felt very confident using Apex.
10	I needed to learn a lot of things before I could get going with Apex.

B.4 Apex UX Questionnaire

Not a validated instrument; its rationale and its five constructs are stated in Section 9.3.3. Administered last, scale 1 = Strongly disagree ... 5 = Strongly agree plus an explicit *N/A - did not use this*.

Items E2 and E3 evaluate the framework rather than the instantiation and are analysed separately from the rest. Seventeen items were administered; the eighteenth was removed before administration for the reason given in Section 9.3.3 and is not reproduced below.

Table B.4: Apex UX questionnaire items, by construct

A. Understanding where you are	
A1	At any moment, I knew which phase of the process I was in.
A2	I understood what the tool expected me to do next.
A3	I could tell the difference between work the tool had finished and work it had not.
B. Working with AI-generated content	
B1	I could tell which parts of the content were generated by AI and which were mine.
B2	Reviewing the AI's output took less effort than writing it myself would have.
B3	I felt able to reject or change the AI's output when I disagreed with it.
B4	I trusted the generated content enough to use it in a real project.
B5	I could see what information the AI had been given as a basis for its output.
C. Control and reversibility	
C1	I felt in control of what the tool did on my behalf.
C2	I was confident I could undo or redo a step if I made a mistake.
C3	I understood the consequences of locking a phase before I did it.
C4	Waiting for the AI to finish was acceptable.
D. Feedback and errors	
D1	When something went wrong, the tool told me clearly what had happened.
D2	The messages the tool showed me told me what to do next, not only what failed.
E. Fit to real work	
E1	The artefacts the tool produced are ones my team would actually use.
E2	The process the tool imposes fits the way my team works.
E3	The effort of following the process is worth what it produces.

Open questions

1. What was the single most confusing thing about using Apex?
2. Was there a point where you did not trust what the tool had produced? What made you doubt it?
3. If you could change one thing about the interface, what would it be?

Administered in Portuguese to participants selecting that language on G6, using the direct translations in docs/thesis/evaluation/instrument-apex-ux.md: same items, same five constructs, same three open questions, differing only in wording.

Full item-by-item responses

Response counts for all seventeen administered items, with the medians Section 9.4.3 computes from them.

Table B.5: Apex UX questionnaire responses, item by item, $N = 13$. Scale 1 = strongly disagree to 5 = strongly agree; the *N/A* column counts participants who marked *did not use this*, which is excluded from the median. Items E2 and E3 evaluate the framework, not the instantiation.

Item	1	2	3	4	5	N/A	Mdn
<i>A. Understanding where you are</i>							
A1 Knew which phase of the process I was in	0	0	3	4	6	0	4
A2 Understood what the tool expected next	0	0	4	5	4	0	4
A3 Could tell finished work from unfinished	0	0	4	7	2	0	4
<i>B. Working with AI-generated content</i>							
B1 Could tell generated content from my own	0	2	0	3	7	1	5
B2 Reviewing took less effort than writing it	0	0	2	6	5	0	4
B3 Felt able to reject or change the output	0	0	0	1	11	1	5
B4 Trusted it enough to use in a real project	0	0	3	8	2	0	4
B5 Could see what information the AI was given	0	1	2	6	4	0	4
<i>C. Control and reversibility</i>							
C1 Felt in control of what the tool did	0	1	3	8	1	0	4
C2 Confident I could undo or redo a step	0	1	1	2	8	1	5
C3 Understood the consequences of locking a phase	0	1	4	2	6	0	4
C4 Waiting for the AI to finish was acceptable	0	0	0	4	9	0	5
<i>D. Feedback and errors</i>							
D1 Was told clearly what had happened when something went wrong	0	1	6	4	2	0	3
D2 Messages said what to do next, not only what failed	0	0	4	3	6	0	4
<i>E. Fit to real work</i>							
E1 The artefacts are ones my team would use	0	1	3	5	1	3	4
E2 (framework) The process fits how my team works	0	2	2	6	0	3	4
E3 (framework) The effort is worth what it produces	0	0	2	5	6	0	4

B.5 Interview guides

Two guides, two populations, both evaluating the framework rather than the tool. Scheduled as a separate 30-45 minute session, not immediately after the task session, and not recorded; the questions below are prompts for a semi-structured conversation rather than a script read verbatim, and the order and depth of follow-up varied per participant.

Guide 1 - Practitioners

1. Describe how your team currently goes from an idea to code in production, where AI already appears in that (if at all, and who decided that), and what goes wrong most often.
2. The framework splits the lifecycle into six phases with a human decision at the end of each. Which of those decisions would genuinely be made in your team, which rubber-stamped, and which phase

would your team resist most?

3. The framework requires a specification to exist before code is generated. What does your team do today instead, what would change, and where would the framework slow you down for no benefit?
4. What would have to be true for your team to adopt this for one project, and what would make you abandon it after two sprints?
5. Who in your team would have to own this for it to survive, and does that person exist?
6. If the tool disappeared tomorrow but you kept the framework, would you still use it? Which parts?
7. When AI writes code that ships and later fails, who is accountable in your organisation today, and does the framework change that?
8. Does the framework's record of what was decided, and by whom, have value to anyone outside the team?
9. What is missing from the framework entirely, and is there anything you expected me to ask and I did not?

Guide 2 - Agile and Scrum experts

Target participants with recognised depth in Agile delivery methodology, not necessarily users of Apex. Given the framework document in advance; this interview concerns the method on paper, not the tool.

1. How would you describe what this framework is, in your own words?
2. Where does this align with Scrum and Agile principles, and where does it depart from them? Is the departure justified by what AI changes, or is it a reversion to stage-gated delivery?
3. The framework has explicit gates between phases. Is that a guardrail or a phase gate in the Waterfall sense, and does the framework avoid the mistake heavyweight processes made in the 2000s?
4. The framework treats roles as hats rather than as people. Does that work in a real team, or does it collapse into one person wearing all of them?
5. Which of the required artefacts would you drop or add, and is the amount of documentation proportionate to what it buys?
6. What kind of team is this framework wrong for, and what would a coach have to teach for a team to run it correctly?
7. Would you recommend this to a client? Under what conditions would you refuse?
8. If you had to defend this framework to a sceptical engineering manager, what would your strongest argument be, and your weakest point?

B.6 Per-participant raw scores

Every SUS score is printed per participant, with both sub-scales, in Table B.6 below; workload is reported as subscale means by task in Table 9.2 in Chapter 9, and the UX questionnaire as full response counts

per item in Table B.5 above (Appendix B.4). The per-participant responses behind the latter two are not printed (a thirteen by six by six workload matrix and a thirteen by seventeen response matrix, out of proportion to their reporting value here).

Table B.6: SUS score and the Usability and Learnability sub-scales, per participant, by language of administration. All three quantities run from 0 to 100 and none is a percentage.

Participant	Language	SUS	Usability	Learnability
P0	EN	42.5	50.00	12.5
P1	EN	75.0	81.25	50.0
P5	EN	80.0	87.50	50.0
P6	EN	15.0	12.50	25.0
P7	EN	62.5	71.88	25.0
P8	EN	60.0	59.38	62.5
P9	EN	60.0	65.63	37.5
P10	EN	75.0	75.00	75.0
P11	EN	65.0	56.25	100.0
P12	EN	70.0	75.00	50.0
P2	PT	37.5	46.88	0.0
P3	PT	40.0	46.88	12.5
P4	PT	55.0	50.00	75.0
Mean, all 13		56.73	59.86	44.23
Mean, English ($n = 10$)		60.50		
Mean, Portuguese ($n = 3$)		44.17		